The W.M. Keck Observatory Near-Infrared Laser Frequency Comb Astrocomb System Documentation

Jinhao Ge, Stephanie Leifer, Greg Doppmann, Daniel Hickstein, David Carlson, YooJung Kim

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Last update: January 14, 2026

Table of Contents

Introduction ..... **VII**

0.1 General Operating Principle VIII

0.2 Key Performance Parameters VIII

1 GUI introduction **1**

1.1 GUI layout and introduction 1

1.1.1 Device control panel 2

1.1.2 Status monitor 3

1.1.3 Confirm & Log panel 4

1.1.4 Interaction rules and safety logic 6

2 Operation of the Menlo Flattener, ICE Server, and Dispatcher **7**

2.1 Introduction 7

2.2 System Overview 7

2.3 Operating the Menlo Laptop 8

2.3.1 Remote Access via Google Remote Desktop 8

2.3.2 Launching the Flattener Control Software 8

2.3.3 Slider Control and Power Attenuation 9

2.4 Starting the ICE Server and Dispatcher 10

2.4.1 Preparing Terminals on the LFC Laptop 10

2.4.2 Starting the ICE Server 11

2.4.3 Starting the Dispatcher on `irastrocomb` 12

2.4.4 Connecting the Dispatcher to the ICE Server 13

2.5 Turning the Full Comb On and Off 14

2.5.1 Enabling the Full Comb 14

2.5.2 Flattener Modes: Filter OR Flatten 15

2.5.3 Returning the Comb to Standby ... 17
2.5.4 Disconnecting the Dispatcher and Stopping the ICE Server ... 18
2.6 Additional Functions ... 18
2.6.1 Checking Full-Comb Versus Standby Status ... 18
2.6.2 HK Shutter Control ... 19
2.7 Errors and Troubleshooting ... 20
2.7.1 Accidentally Closed Terminals ... 20
2.7.2 Dispatcher Write Errors ... 20
2.7.3 Maximum Number of Terminals ... 21
2.7.4 VISA error ... 21

3 Instrument Control Summary ... 23

3.1 Amonics EDFA (`AmonicsEDFA`) ... 23
3.1.1 Introduction ... 23
3.1.2 Functions ... 23
3.2 Agilent 86142B optical spectrum analyzer (`Agilent_86142B`) ... 26
3.2.1 Introduction ... 26
3.2.2 Functions ... 26
3.3 Arduino relay module (`Arduino_relay`) ... 30
3.3.1 Introduction ... 30
3.3.2 Functions ... 30
3.4 Clarity wavelength locker (`Clarity`) ... 33
3.4.1 Introduction ... 33
3.4.2 Functions ... 33
3.5 Eaton PDU (`EatonPDU`) ... 35
3.5.1 Introduction ... 35
3.5.2 Functions ... 36
3.6 hk shutter controller (`hk_shutter`) ... 40
3.6.1 Introduction ... 40
3.6.2 Functions ... 41
3.7 Instek GPD-4303S DC power supply (`InstekGPD_4303S`) ... 42
3.7.1 Introduction ... 42
3.7.2 Functions ... 42
3.8 Instek GPP DC power supply (`InstekGppDCSupply`) ... 47
3.8.1 Introduction ... 47
3.8.2 Functions ... 47
3.9 Keysight 33500 function generator (`KeysightFG_33500`) ... 50
3.9.1 Introduction ... 50
3.9.2 Functions ... 50

3.10 ORION laser module (`ORIONLaser`) ... 53
3.10.1 Introduction ... 53
3.10.2 Functions ... 53
3.11 OZ Optics VOA (`OZopticsVOA`) ... 56
3.11.1 Introduction ... 56
3.11.2 Functions ... 56
3.12 Pendulum CNT-90 frequency counter (`PendulumCNT90`) ... 57
3.12.1 Introduction ... 57
3.12.2 Functions ... 57
3.13 Pritel optical amplifier (`PritelAmp`) ... 59
3.13.1 Introduction ... 59
3.13.2 Functions ... 59
3.14 Stanford Research System SIM900 mainframe (`SRS_SIM900`) ... 62
3.14.1 Introduction ... 62
3.14.2 Functions ... 62
3.15 SRS PID controller SIM960 (`SRS_PIDcontrol_SIM960`) ... 63
3.15.1 Introduction ... 63
3.15.2 Functions ... 63
3.16 SRS isolated voltage source SIM928 (`SRS_VoltSorc_SIM928`) ... 67
3.16.1 Introduction ... 67
3.16.2 Functions ... 68
3.17 Tektronix TDS2024C oscilloscope (`TDS2024C`) ... 68
3.17.1 Introduction ... 68
3.17.2 Functions ... 68
3.18 TE Technology TC-720 temperature controller (`TEC_TC720`) ... 71
3.18.1 Introduction ... 71
3.18.2 Functions ... 72
3.19 Measurement Computing USB-2408 DAQ (`USB2408`) ... 78
3.19.1 Introduction ... 78
3.19.2 Functions ... 78
3.20 Finisar Waveshaper (`Waveshaper`) ... 79
3.20.1 Introduction ... 79
3.20.2 Functions ... 80
3.21 Example integrated control script ... 84
3.21.1 Overview ... 84
3.21.2 Step 1: OSA startup comb spectrum ... 85
3.21.3 Step 2: EDFAs and RF power supplies ... 85
3.21.4 Step 3: Safety relay and repetition-rate check ... 86
3.21.5 Step 4: Waveshaper dispersion and band-pass ... 86
3.21.6 Step 5: Minicomb intensity-lock servo ... 87
3.21.7 Step 6: Pritel amplifier, shutters, and VOAs ... 88

3.21.8 Step 7: Scope, function generators, ORION laser, and TEC 88
3.21.9 Step 8: Rack and table temperature monitoring 89

4 LFC KTL keyword function, ICE server, and Dispatcher 91

4.1 Comb KTL service 91
4.1.1 Overview 91
4.1.2 Establishing the ICE connection 92
4.1.3 Notes for developers 93
4.1.4 Details on keywords and type handling 95
4.1.5 How the ICE connection works 96
4.1.6 KTL keywords and the KeckLFC class 97
4.2 ICE-based control interface 98
4.2.1 Overview 98
4.2.2 LFC ICE server (`server.py`) 98
4.2.3 Reference ICE client (`client.py`) 100
4.3 KeckLFC keyword controller (code realization level) 102
4.3.1 Environment and configuration 102
4.3.2 Core infrastructure 102
4.3.3 Representative keyword functions 104
4.4 KTL functions and keywords (GUI level and basic level concept) 110
4.4.1 GUI-level KTL functions 110
4.4.2 Low-level KTL keywords (maybe no direct GUI button, depend on demand) 113
4.5 LFC `.sin` files and keyword definition 123
4.5.1 `LFC.xml.sin`: XML keyword list 124
4.5.2 `combd.sin`: dispatcher source 126

5 Instrument Description 129

5.1 Facility 129
5.2 Mapping of numbered components in the assembly diagram 132
5.2.1 Rack interface connection introduction 144
5.2.2 Rack router IP information 145
5.3 Detailed Instrument Design 146
5.3.1 Comb Generator 147
5.3.2 Frequency Stabilization of the Pump Laser 150
5.3.3 Electronics Rack Small component chassis 154
5.3.4 Filter cavity 156
5.3.5 Control system 157

5.3.6 Thermal control 158
5.3.7 Spectral Flattener 159
5.3.8 Fiber Feed 161
5.3.9 Electrical 161
5.4 Instrument operation details and comb operation process 166
5.4.1 Individual instrument operation 166
5.4.2 Combined operation details 175

Introduction

An astrocomb is a Laser Frequency Comb (LFC) calibration source for astronomical spectrographs. Like gas cells, arc lamps, and etalons, it provides light at well-defined frequencies to serve as a "spectral ruler" to compare against target object spectral signatures and to monitor shifts in spectrograph pixel positions due to thermo-mechanical changes in the instrument. As such, it is an important tool for performing precision radial velocity studies of exoplanet-hosting stars. Unlike gas cells and arc lamps, LFCs provide light "markers" at very uniform spacing optimized for a particular spectrograph's resolution. And unlike etalons, the spacing between the lines is traceable to a GPS-disciplined RF signal. Astrocombs are therefore the highest precision and accuracy calibration sources available to astronomical spectrographs. They are also the most complex of these instruments.

This document describes the operating principles, signal chain, components, configuration, and software control of an electro-optic (EO) near-infrared (NIR) LFC designed and built by the California Institute of Technology and Aerospace Corporation for the W.M. Keck Observatory under a grant from the Heising-Simons Foundation (HSF). The comb is intended for use initially with the NIRSPEC spectrograph, and eventually with the HISPEC spectrograph under development at the time of this writing. Its light is to be injected through the Keck Planet Imager and Characterizer (KPIC) fiber injection unit, although it can also be directly coupled to the spectrograph integrating sphere(s). The comb assembly is largely composed of commercial, off-the-shelf (COTS) parts developed for the optical telecommunications industry, and thus spare parts are generally available.

This manual is organized by user roles and access levels for operating the LFC (laser fiber comb) system. It begins with high level operating instructions for the end-user, and progresses through a detailed instrument design description for use by M.W. Keck Observatory staff.

Chapter 1 — GUI Control (general access). For observers who operate the astrocomb through the graphical user interface. The GUI is generally open to all users, with certain functions gated by password-protected, limited-access controls.

Chapter 2 — System Start-Up/Shutdown (elevated access). For users responsible for powering the full comb on/off and establishing or disabling system connections. These higher-level procedures require access to the LFC laptop.

Chapters 3–5 — Expert Tools & Fundamentals (highest access). Intended for the expert team to understand and utilize the astrocomb at a fundamental level:

- **Chapter 3:** Low-level control code for each device in the astrocomb.
- **Chapter 4:** LFC KTL keywords functions used by the dispatcher to control the LFC via the GUI.
- **Chapter 5:** Detailed description of the physical instruments and components that comprise the astrocomb.

Troubleshooting guides for software and hardware are contained in Chapter 2 and Chapter 5, respectively.

0.1 General Operating Principle

An electro-optic (EO) Laser Frequency Comb generates a comb of calibration lines by creating sidebands on a continuous-wave (cw) laser through electro-optic modulation at a prescribed frequency. This is accomplished by using an EO intensity modulator to carve out pulses at a specific radio frequency (RF), followed by cascaded EO phase modulators that temporally chirp the pulses to provide additional optical bandwidth. The result is a "'minicomb" of spectral bandwidth and number of lines governed by the number of cascaded phase modulators and the RF-drive voltage. The frequencies of both the pump laser and the RF drive oscillator must be locked to stable reference signals to provide the comb stability needed for spectrograph calibration. When optimized, the NIR LFC minicomb bandwidth is about 11 nm. The minicomb is then temporally compressed and then spectrally broadened to provide coverage over the desired spectrograph bandpass and spectrally flattened to provide uniform line intensity. The full, flattened comb calibration signal can then be attenuated to a level appropriate for the desired integration time to match the intensity of a target of interest. A shutter isolates the LFC system from the observatory instruments which are fed by optical fiber.

0.2 Key Performance Parameters

Table 1. Key performance parameters of the astro-comb.

Parameter	Value	Notes
Mode spacing	16 GHz	Repetition rate is set by the drive oscillator, which is fixed for this assembly. However, the instrument is capable of producing lines spaced between 12 and 18 GHz (TBC – check RF amplifier and modulator specs.)
Minimum wavelength	1500 nm	The comb produces light well blue of 1500 nm, but the spectral flattener only operates between 1500 nm and 2500 nm. It has the potential to direct shorter-wavelength light to a separate flattener unit.
Maximum wavelength	2500 nm	Comb light is produced as far as 2500 nm, but the intensity and SNR of the comb lines is lower than the rest of the bandpass (to be quantified during test).
Dynamic range	60 dB	Achieved in 10 dB steps using neutral-density filters.
Power per mode	TBD	To be determined from system-level power measurements.
Frequency stability	10^{-10}	Depending on the pump reference.

CHAPTER 1
GUI introduction

1.1 GUI layout and introduction

All screenshots in this chapter are demo versions of the graphical user interface (GUI). The final implementation may differ slightly from what is shown here; in all cases, the real hardware and software behaviour take precedence.

Figure 1.1 shows the overall layout of the *LFC STATUS and CONTROL* GUI. The window is divided into three main regions:

- The **device control panel** on the left, which contains the high-level control buttons for the laser frequency comb (LFC).
- The **status monitor** in the centre, which is automatically updated and displays the health of key subsystems and the current comb spectrum.
- The **Confirm & Log** panel on the right, which records recent operations, provides a confirmation layer for all commands, and exposes advanced controls that require a password.

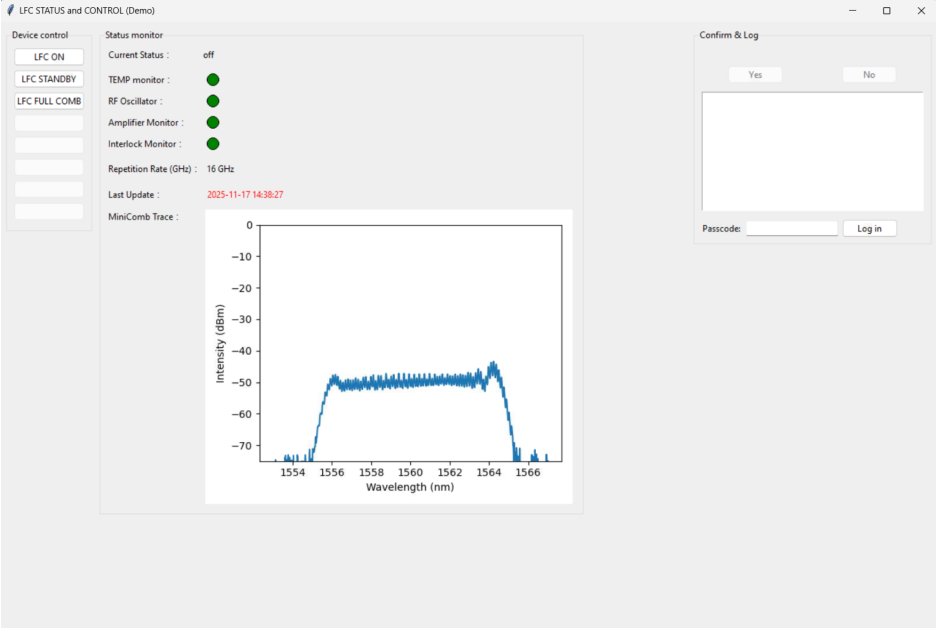


Figure 1.1. Overall layout of the LFC status and control GUI (demo).

The GUI polls the astrocomb system automatically every 5min. During each polling cycle the software queries the status and key parameters of the astrocomb devices and updates the indicators in the central panel. If all indicators are green, the corresponding subsystems are operating within their expected ranges. If any indicator turns red, the system is automatically driven to a safe **OFF** state, and an expert must inspect the astrocomb before it is turned on again.

In addition to hardware status, the GUI also reads out the **repetition rate** of the comb. This parameter is reported in GHz and corresponds to the frequency spacing between adjacent comb lines. Each automatic update is timestamped; the timestamp is displayed as *Last Update* in the status monitor and serves as a time reference for the displayed information.

The lower central area can display a real-time minicomb trace from the spectrum analyser. In the final GUI this view may be omitted or replaced by a simplified status indicator, depending on performance and operational needs.

1.1.1 Device control panel

The column on the left contains the primary device control buttons, such as:

- **LFC ON**
- **LFC STANDBY**
- **LFC FULL COMB**

Each button represents a predefined sequence of operations on multiple devices in the astrocomb system. The detailed behaviour of these functions is described in later sections of this manual.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

To minimise accidental operations, the GUI enforces a strict rule that **only one button can be active at any given time**. When one function is running, all other buttons are temporarily disabled and greyed out. This prevents conflicting commands from being sent to the hardware.

Furthermore, during the automatic status polling (which typically lasts 2–3 minutes), *all* buttons are disabled. This ensures that no state-changing commands are issued while the GUI is synchronising with the astrocomb devices.

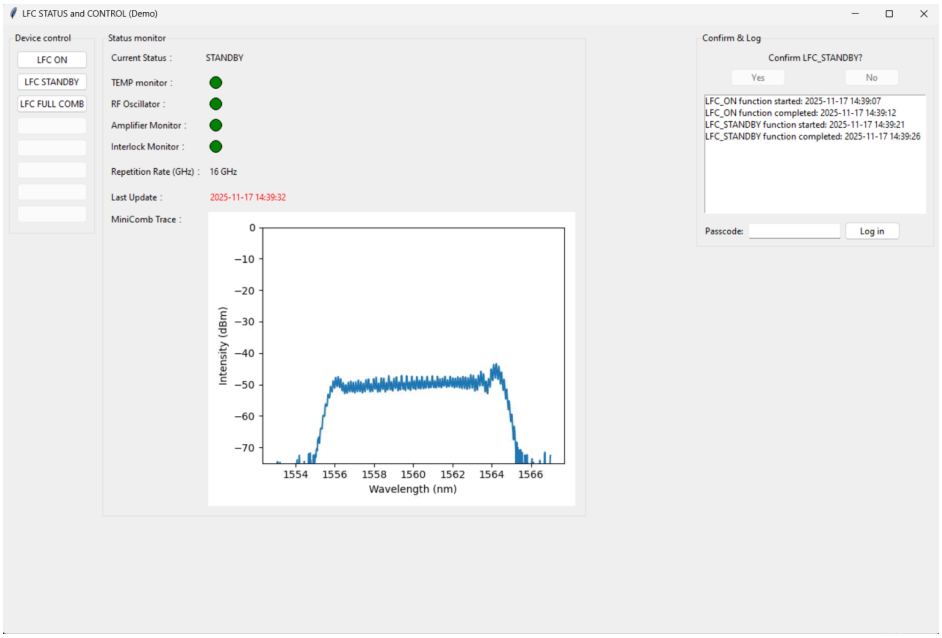


Figure 1.2. Device control panel and status monitor (demo).

1.1.2 Status monitor

The central status monitor summarises the health of the system. In the demo GUI the following indicators are shown:

- **Current Status**: text label indicating whether the LFC is in *OFF*, *STANDBY*, or *FULL COMB* mode.
- **TEMP monitor**: temperature-related status of critical components.
- **RF Oscillator**: status of the RF oscillator chain.
- **Amplifier Monitor**: status of the optical amplifiers.
- **Interlock Monitor**: status of safety interlocks.
- **Repetition Rate (GHz)**: measured comb repetition rate.
- **Last Update**: timestamp of the most recent automatic query.
- **MiniComb Trace**: a real-time plot of comb power versus wavelength (if enabled).

Each of the monitor items is accompanied by a coloured indicator:

- **Green**: subsystem is within normal operating range.
- **Red**: subsystem is out of range or has reported an error.

If any of the indicators turns red, the control logic immediately forces the system into a safe **OFF** status. The GUI will not allow a new start until the underlying issue has been diagnosed and cleared by qualified personnel.

1.1.3 Confirm & Log panel

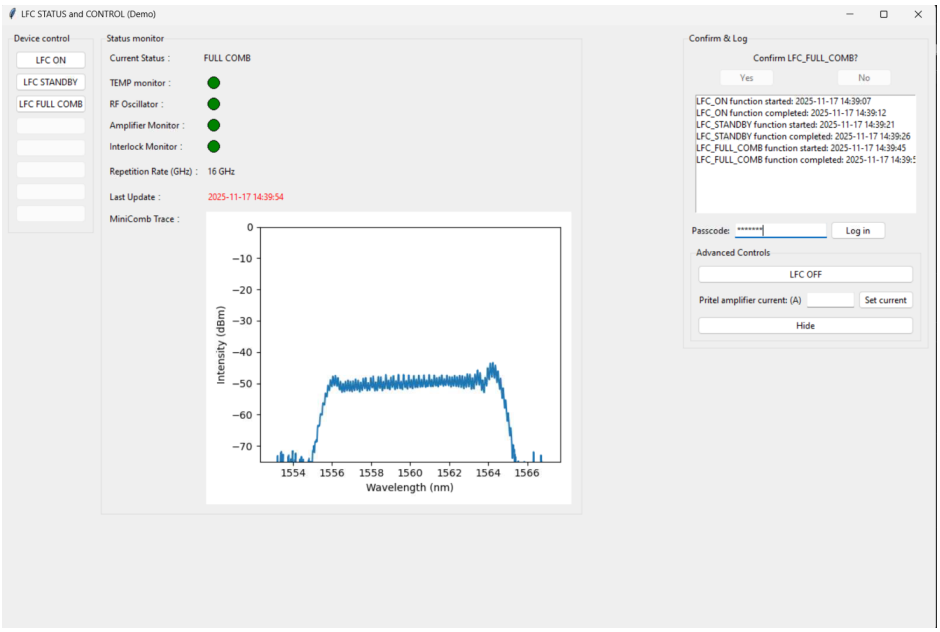


Figure 1.3. Confirm & Log panel and advanced controls (demo).

The right-hand side of the GUI contains the **Confirm & Log** panel. This panel serves three purposes:

(1) To record recent operations and events.
(2) To provide a confirmation layer for all user commands.
(3) To expose advanced controls that require a password.

1.1.3.1 Operation log

The log window lists the most recent commands and status changes, together with timestamps. In the current design, the last twenty events are kept visible. Typical entries include:

- Start and completion of `LFC_ON`, `LFC_STANDBY`, and `LFC_FULL_COMB` functions.
- Transitions to **OFF** status due to errors.
- Manual changes to advanced parameters such as amplifier current.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

1.1.3.2 Command confirmation

Above the log window is a compact confirmation area with two buttons, **Yes** and **No**, labelled with the operation that is about to be executed (for example, "Confirm LFC_FULL_COMB?").

To avoid accidental actions, **every** function button in the GUI requires explicit confirmation:

(1) When the user clicks a function button (e.g. **LFC FULL COMB**), the system does not immediately apply the change.
(2) Instead, the confirmation message appears, and the user must click **Yes** to proceed or **No** to cancel.

Only after a positive confirmation is the corresponding sequence sent to the astrocomb devices.

1.1.3.3 Advanced access

The lower part of the panel contains the advanced control section. Access to these controls requires a password:

- The user enters the passcode in the *Passcode* field and presses **Log in**. After **Log in** is pressed, the contents of the passcode field are cleared for security.
- After successful login, the **Advanced Controls** area is enabled.

Two key advanced functions are available:

LFC OFF This button immediately shuts down all relevant outputs of the astrocomb system. It should be used whenever the comb must be safely turned off, for example before planned power cuts or hardware reconfiguration. The user must understand that this command stops all light output and should be issued deliberately.

Pritel amplifier current (A) This field sets the drive current of the Pritel amplifier, which controls the comb power and, consequently, the spectral broadening. The normal operating range is 3.8 to 4.2A. A higher current gives higher output power and a broader comb. Currents below 3.8A are allowed, but the comb bandwidth may be insufficient. Currents above 4.2A are considered unsafe. If the comb bandwidth requirement cannot be met at 4.2A, the user should contact the Octave team for inspection. The current must *not* be increased above 4.2A without understanding the root cause of the problem, as this may cause serious damage to the astrocomb hardware.

A **Hide** button may be provided to collapse the advanced section when not in use, reducing clutter and preventing accidental interaction.

1.1.4 Interaction rules and safety logic

To ensure safe and predictable behaviour, the GUI enforces several global rules:

- **Single active command**: at any moment only a single function can be in progress. While a command is executing, all other buttons are disabled and greyed out.
- **Polling lockout**: during the automatic status update sequence (typically 2–3 minutes), all buttons are disabled to prevent conflicts between user actions and device polling.
- **Error-induced shutdown**: if any monitor indicator turns red, the system automatically transitions to the **OFF** state and blocks further operation until the issue has been resolved by qualified staff.

These rules, combined with the confirmation layer and password-protected advanced controls, are designed to minimise operator error and protect the astrocomb hardware from unintended or unsafe commands.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

CHAPTER 2

Operation of the Menlo Flattener, ICE Server, and Dispatcher

2.1 Introduction

This chapter describes how to operate the ICE server, the dispatcher, and the Menlo flattener used for controlling and shaping the Keck LFC (laser frequency comb). It is based on the internal operation slides and is intended as a practical step–by–step guide for day–to–day use.

2.2 System Overview

The system consists of three main elements:

(1) The **Menlo laptop**, which is an isolated computer hosting the Menlo flattener and slider control software locally.
(2) The **ICE server**, running on the LFC laptop, which exposes control of the comb hardware to external clients.
(3) The **dispatcher**, running on `irastrocomb`, which sends high–level commands to the comb via the ICE server.

To operate the comb, you typically:

(1) Log into the LFC laptop and use Google Remote Desktop to control the Menlo laptop.
(2) Use the Menlo software to monitor and flatten the comb spectrum.
(3) Start the ICE server on the LFC laptop.
(4) Start the dispatcher on `irastrocomb` and connect it to the ICE server.
(5) Turn the comb to full power (“full comb”), adjust flattening/filtering, then eventually return it to standby and shut down the software.

2.3 Operating the Menlo Laptop

2.3.1 Remote Access via Google Remote Desktop

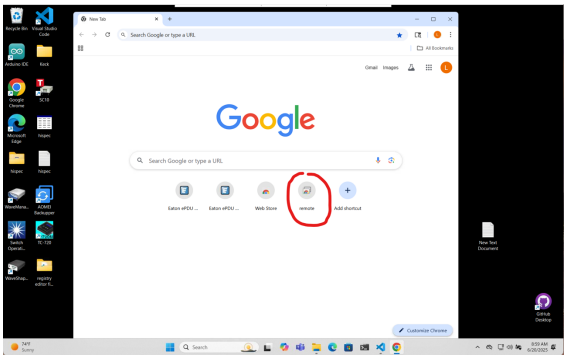


Figure 2.1. Google Remote Desktop

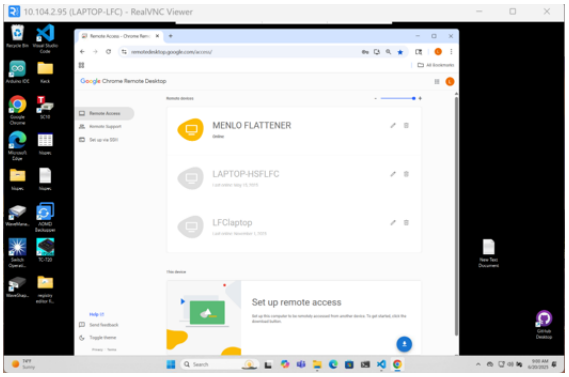


Figure 2.2. Menlo flattener

Because the Menlo laptop is isolated and runs the control software locally, it is accessed remotely from the LFC laptop:

(1) Log in to the LFC laptop.
(2) Open a web browser and navigate to Google.
(3) Either click on the Google Remote Desktop icon or search for "Google Remote Desktop".
(4) In the list of available machines, click on the entry corresponding to the **Menlo flattener** to start a remote session.

Once connected, you will see the Menlo desktop, which provides access to:

- **Flattener Control** — flattens the comb spectrum using a spatial light modulator (SLM).
- **Slider Control** — attenuates the comb output using a hardware slider with discrete positions.

2.3.2 Launching the Flattener Control Software

On the Menlo laptop, the Flattener Control software is started as follows:

(1) Click the icon labeled **1** in the original screenshot to launch the Flattener Control.
(2) Wait approximately 10 seconds for a simplified control panel to appear.
(3) A simplified panel with limited functionality may appear. You can use this panel or, preferably, proceed to the full panel.
(4) Click the control labeled **2** in the screenshot to bring up the **full** flattener panel.
(5) Once the full panel appears and the blue spectral trace is visible, click buttons **3** and **4** in order:

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

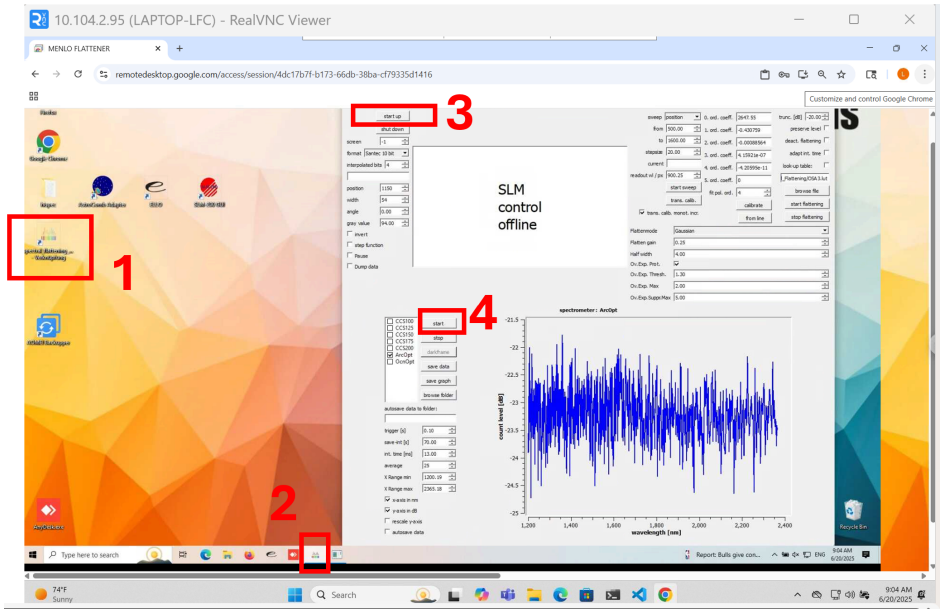


Figure 2.3. Launching the Flattener Control Software

(a) Button **3** launches the spatial light modulator (SLM).
(b) Button **4** launches the optical spectrum analyzer (OSA).

The SLM assigns one pixel per comb line, so each SLM pixel corresponds to a specific comb–line frequency and can attenuate that line individually. The OSA shows the comb spectrum in real time.

If the spectrum does not update on the OSA:

- Repeat the steps from 1 to 4 to relaunch the panels, or
- Reduce the OSA averaging (e.g. set the average to 1 or 2).

2.3.3 Slider Control and Power Attenuation

To launch the slider control on the Menlo laptop:

(1) Open the application named **ELLO**.
(2) Connect to the slider device from within ELLO.
(3) A window will appear showing slider positions and controls.

The slider offers six discrete attenuation settings. Using **Position 6** as the 0 dB reference, the approximate insertion losses are:

- Position 1: ~ 5 dB
- Position 2: ~ 10 dB

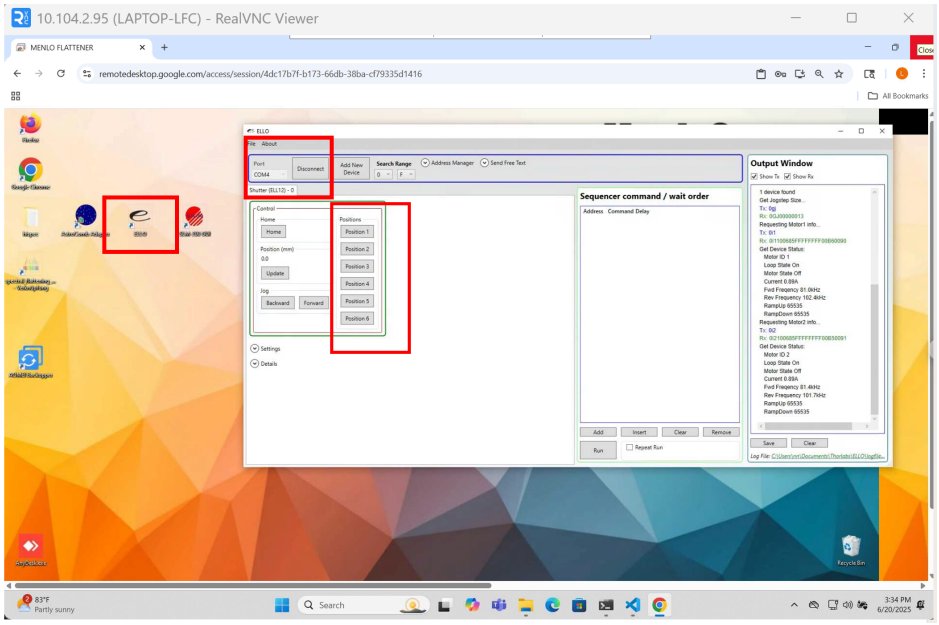


Figure 2.4. Slider Control and Power Attenuation

- Position 3: ~ 20 dB
- Position 4: > 20 dB
- Position 5: > 20 dB
- Position 6: 0 dB reference

The slider is typically used *after* the full comb has been brought up, to attenuate the comb power to a suitable level for observation.

An example measurement for an input power of 55 mW:

- Position 1: ~ 15 μW
- Position 2: ~ 4.8 μW
- Position 3: ~ 0.5 μW
- Position 4: 0 μW
- Position 5: 0 μW
- Position 6: ~ 48 μW

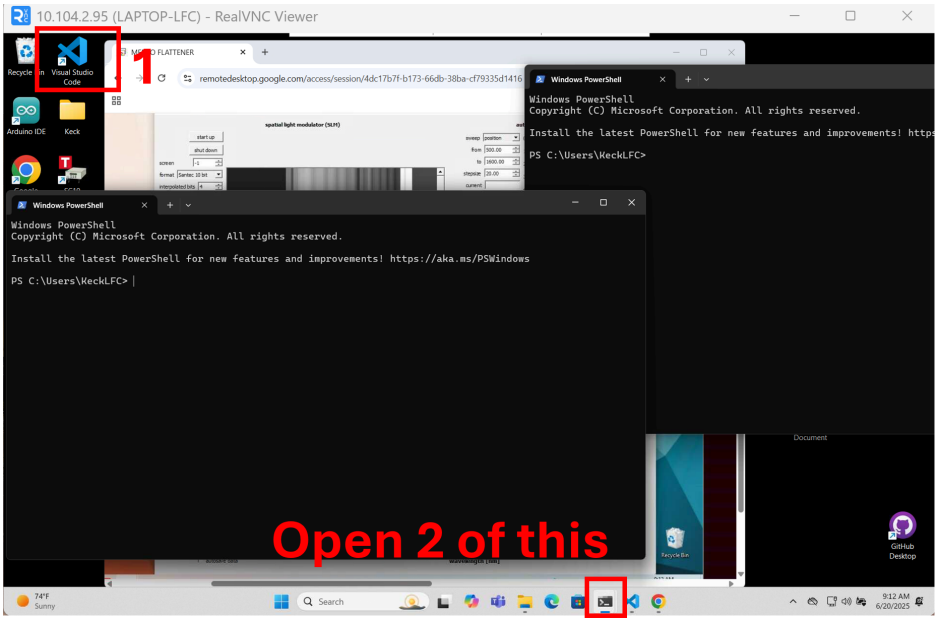
2.4 Starting the ICE Server and Dispatcher

To control the comb from `irastrocomb`, the ICE server on the LFC laptop must be running, and the dispatcher must be started and connected to the ICE server.

2.4.1 Preparing Terminals on the LFC Laptop

Open **two** terminal windows and Visual Studio Code (VS Code) on the LFC laptop:

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

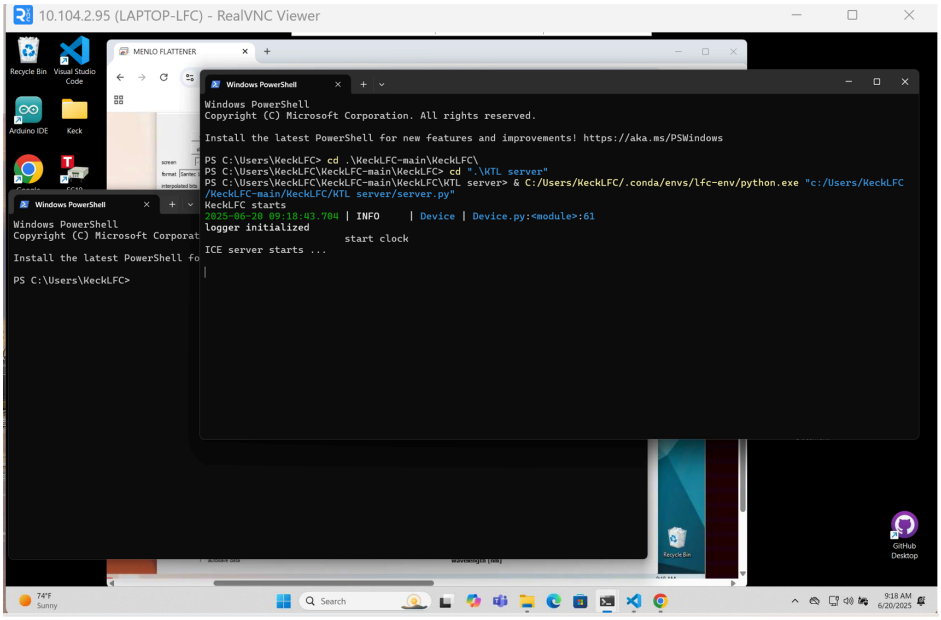


(1) Launch two terminal windows.
(2) Open VS Code, then minimize it for now.

We will refer to these terminals as:

- The **right** terminal: used to start the ICE server (*ICE terminal*).
- The **left** terminal: used to start and interact with the dispatcher (*dispatcher terminal*).

2.4.2 Starting the ICE Server



In the ICE terminal (right terminal), start the ICE server using the following commands:

(1) Change into the main KeckLFC directory:

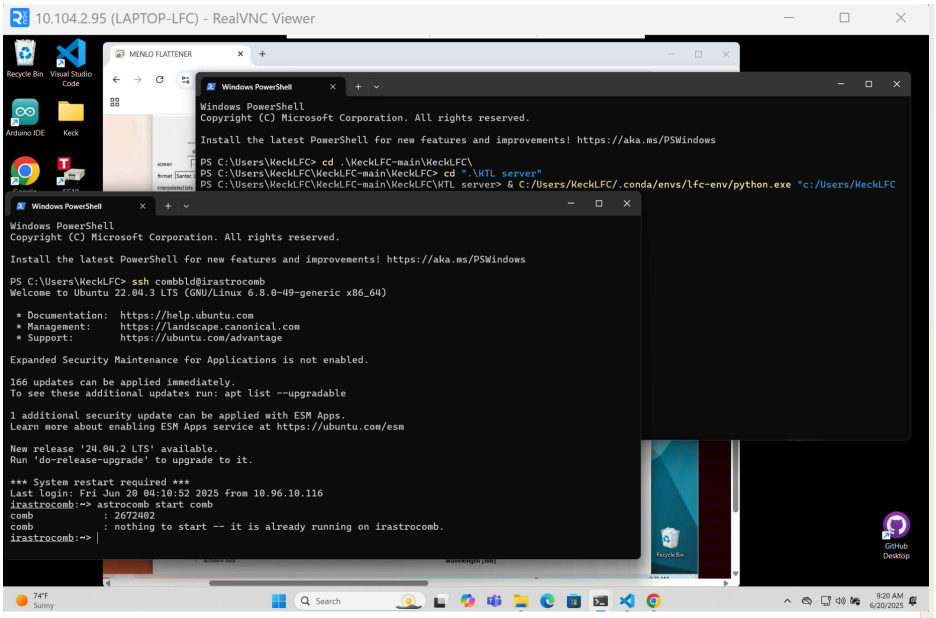
```
cd .\KeckLFC-main\KeckLFC\
```

(2) Start the KTL server:

```
cd ".\KTL server" &
C:/Users/KeckLFC/.conda/envs/lfc-env/python.exe \
  "c:/Users/KeckLFC/KeckLFC-main/KeckLFC/KTL server/server.py"
```

Once the output matches the reference screenshot from the slides, the ICE server is running. This terminal must remain open while the dispatcher is connected.

2.4.3 Starting the Dispatcher on `irastrocomb`



In the dispatcher terminal (left terminal), log into `irastrocomb` and start the dispatcher:

(1) From the LFC laptop, run:

```
ssh combbld@irastrocomb
```

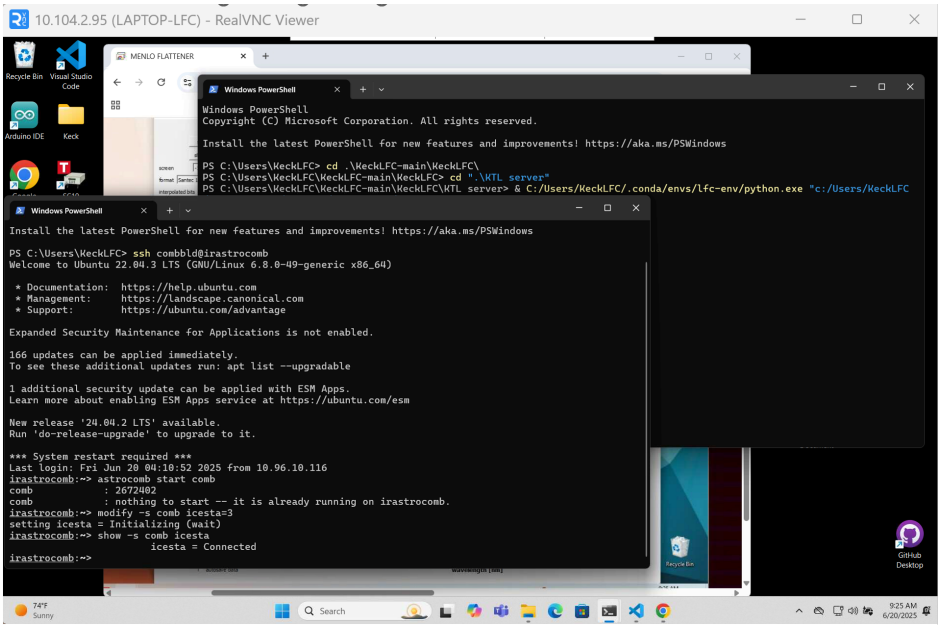

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

(2) Once logged into `irastrocomb`, start the comb control process:

```
astrocomb start comb
```

When the screen output matches the reference picture, the dispatcher is running and ready to connect to the ICE server.

2.4.4 Connecting the Dispatcher to the ICE Server



To connect the dispatcher to the ICE server:

(1) In the dispatcher terminal, type:

```
modify -s comb icesta = 3
```

This command requests that the dispatcher connect to the ICE server.

(2) Wait about 20 seconds.

(3) Check the connection status with:

```
show -s comb icesta
```

You should see a status indicating that the dispatcher is *connected*.

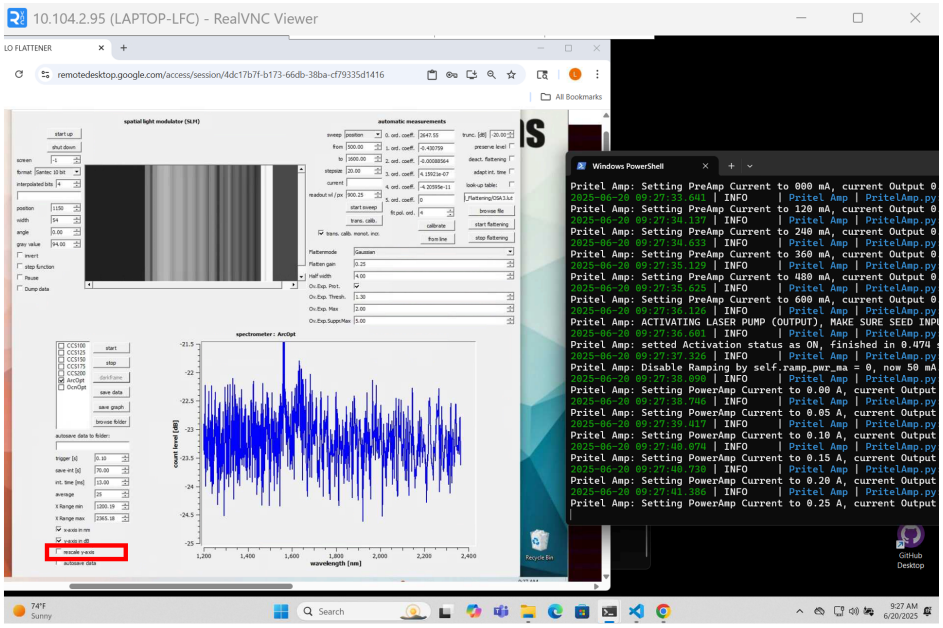
If this connection step is missed or fails, and you later send commands to the dispatcher, you will encounter an error of the form:

```
ERR_WRITE_SW_ERROR (-5401): There was an error in the device-specific
write routine for this keyword: check the log files.
```

This indicates that the dispatcher is not properly connected to the ICE server.

2.5 Turning the Full Comb On and Off

2.5.1 Enabling the Full Comb



Once the dispatcher is connected to the ICE server, you can turn on the full comb:

(1) In the dispatcher terminal, type:

```
modify -s comb lfc_set_full_comb = 1
```

(2) Wait about 1 minute for the comb power to ramp up.
(3) On the Menlo OSA display, you should see the full comb spectrum appear.

You can:

- Choose to filter out the pump, or
- Perform flattening at a chosen dB level.

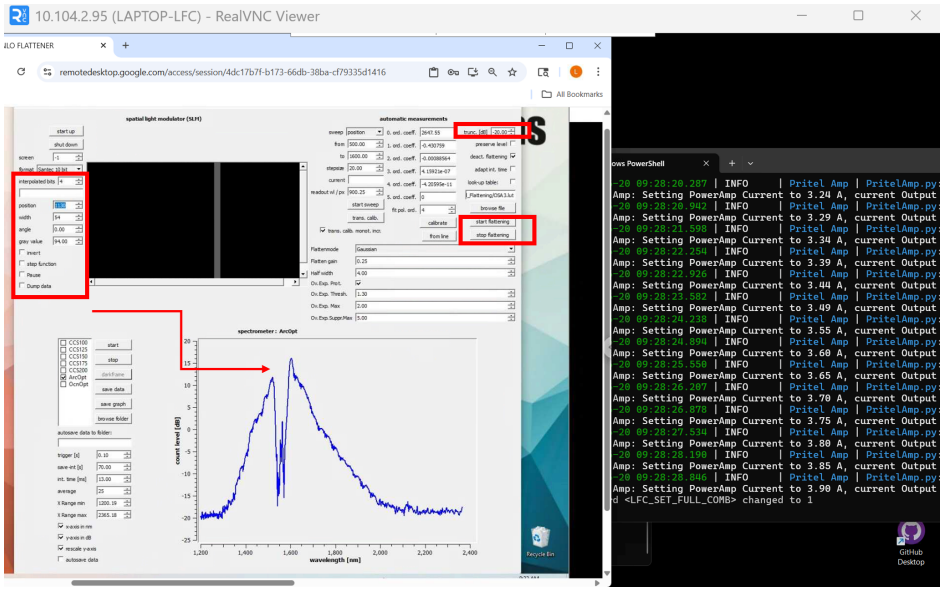
However, only one of these modes should be used at a time.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

2.5.2 Flattener Modes: Filter OR Flatten

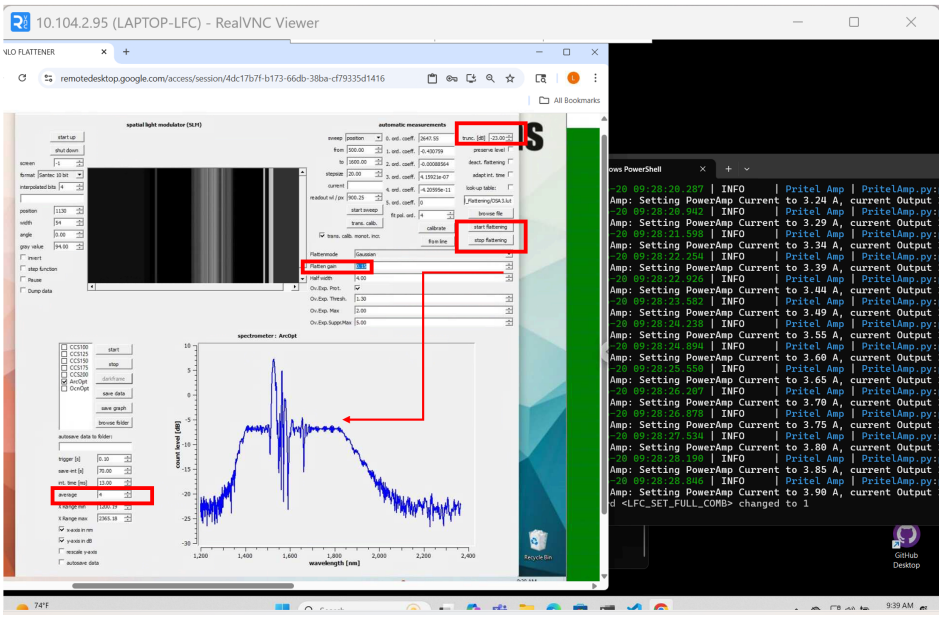
The Menlo flattener offers two primary modes of operation: filter mode and flatten mode. **However, only one of these modes could be used at a time.**

2.5.2.1 Filter Mode



- Select **Filter** mode by clicking the left–hand button on the Flattener interface.
- A gray band appears on the SLM display, indicating which comb–line frequencies will be attenuated.
- You can adjust:
  - The **width** of the gray band (filter bandwidth),
  - The **position** of the band (center frequency),
  - The **gray level** (depth of attenuation).

2.5.2.2 Flatten Mode



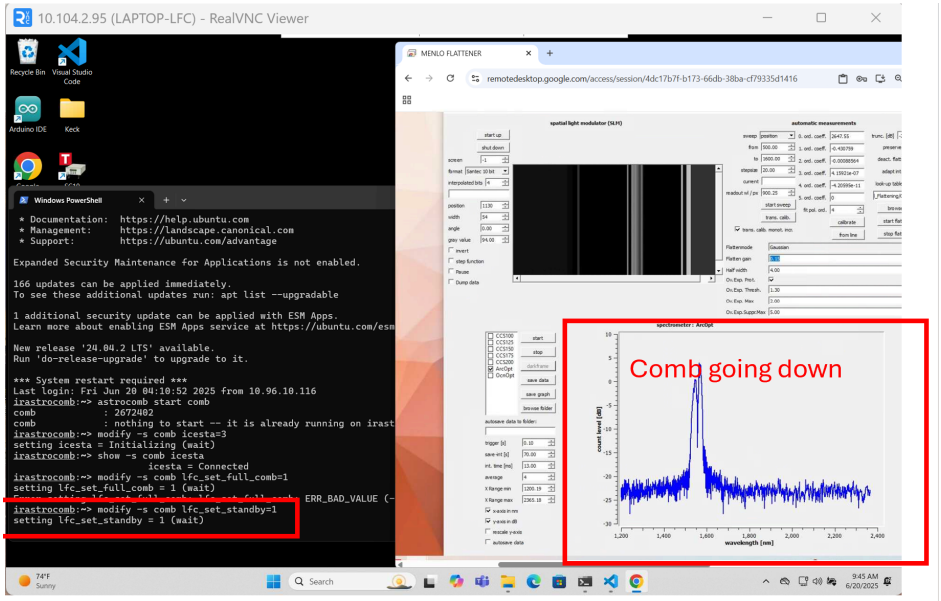
- Select **Flatten** mode by clicking the right–hand button.
- Set a target truncation (flattening) level (for example, −23, −20, −17, −15, −13, −10 dB, etc.).
- Click **Start**, and the system will automatically adjust the SLM to flatten the comb spectrum to the target level.

2.5.2.3 Recommended Parameter Settings

- OSA averaging can be set to 4, 2, or 1.
- If the spectrum is oscillating during flattening, reduce the flatten gain (for example, from the default 0.25 to 0.15).
- If the spectrum is not fully displayed, use the **rescale Y axis** function on the Menlo OSA panel.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

2.5.3 Returning the Comb to Standby



After you have finished using the comb, you must return it to standby:

(1) In the dispatcher terminal, type:

```
modify -s comb lfc_set_standby = 1
```

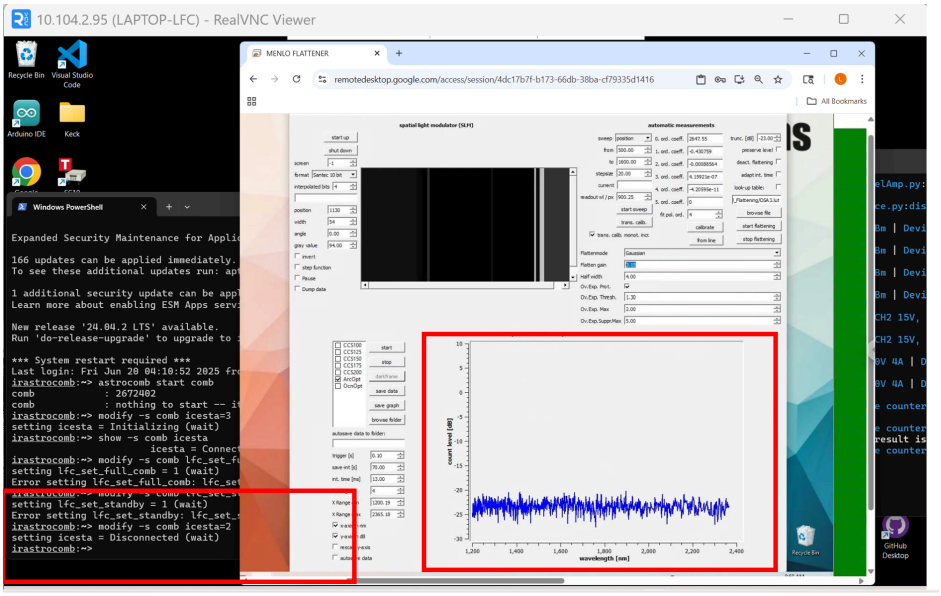
(2) Observe the Menlo spectrum: the full comb should decay to zero over time.

At present, there is no dedicated status keyword such as

```
show -s comb lfc_set_full_comb
```

or a direct full–comb/standby status readback. The primary way to judge whether the comb is in full–comb or standby mode is by observing the spectrum on the Menlo OSA. A GUI check status feature is planned for future implementation.

2.5.4 Disconnecting the Dispatcher and Stopping the ICE Server



Once the comb is in standby:

(1) In the dispatcher terminal, disconnect from the ICE server:

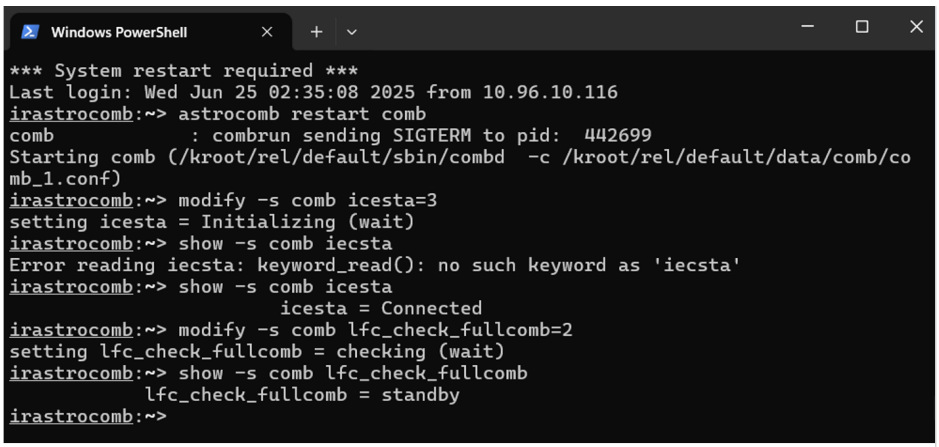
```
modify -s comb icesta = 2
```

(2) In the ICE terminal, stop the ICE server by pressing `Ctrl+C`.

(3) When the terminal output matches the final state shown in the reference slides, both the dispatcher connection and ICE server are properly stopped.

2.6 Additional Functions

2.6.1 Checking Full-Comb Versus Standby Status



jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

For users who cannot see the Menlo spectrum (for example, if they are remote without OSA access), there is an auxiliary status check:

(1) In the dispatcher terminal, after setting the comb to the desired mode, type:

```
modify -s comb lfc_check_fullcomb = 2
```

(2) Wait about 5 seconds.
(3) Then type:

```
show -s comb lfc_check_fullcomb
```

(4) The returned value indicates whether the comb is in full–comb or standby, as shown in the reference screenshot.

Important notes:

◇ The *value* you write must be exactly 2:

```
modify -s comb lfc_check_fullcomb = 2
```

◇ This function is primarily intended for situations where the Menlo spectrum cannot be seen. If you can see the Menlo OSA display, always judge the comb status directly from the spectrum.

2.6.2 HK Shutter Control

The hardware shutter (HK shutter) can be controlled from the dispatcher terminal:

(1) To set the shutter state, type:

```
modify -s comb lfc_hk_shutter = 1   % pass light
modify -s comb lfc_hk_shutter = 0   % shut light
```

(2) Wait about 5 seconds.
(3) Then check the shutter status with:

```
show -s comb lfc_hk_shutter
```

2.7 Errors and Troubleshooting

2.7.1 Accidentally Closed Terminals

If you accidentally close one of the terminals:

- Do not panic. Simply reopen the terminal and repeat the corresponding procedure.

Examples:

- If you close the ICE terminal:

  (1) Open a new terminal.
  (2) Restart the ICE server as described in Section 2.4.2.
  (3) Reconnect the dispatcher if needed.

- If you close the dispatcher terminal:

  (1) Open a new terminal.
  (2) SSH into `irastrocomb` again.
  (3) Restart the dispatcher and reconnect to the ICE server.

2.7.2 Dispatcher Write Errors

If you see an error of the form:

```
ERR_WRITE_SW_ERROR (-5401):  There was an error in the device-specific
write routine for this keyword:  check the log files.
```

this typically indicates that:

- The dispatcher is not connected to the ICE server (for example, `icesta` was not set to 3), or
- The ICE server is not running.

In this case:

(1) Verify that the ICE server is running in the ICE terminal.
(2) In the dispatcher terminal, run:

```
modify -s comb icesta = 3
```


jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

and recheck using:

```
show -s comb icesta
```

(3) Once the connection status is correct, reissue your comb control commands.

2.7.3 Maximum Number of Terminals

As a general rule:

- **Do not open more than two terminals** for this workflow on the LFC laptop.

Keeping exactly one ICE terminal and one dispatcher terminal helps avoid confusion and ensures that the correct processes are controlled from the expected locations.

2.7.4 VISA error

If the ICE server prints an error message related to a VISA failure, it may look similar to the screenshots below.

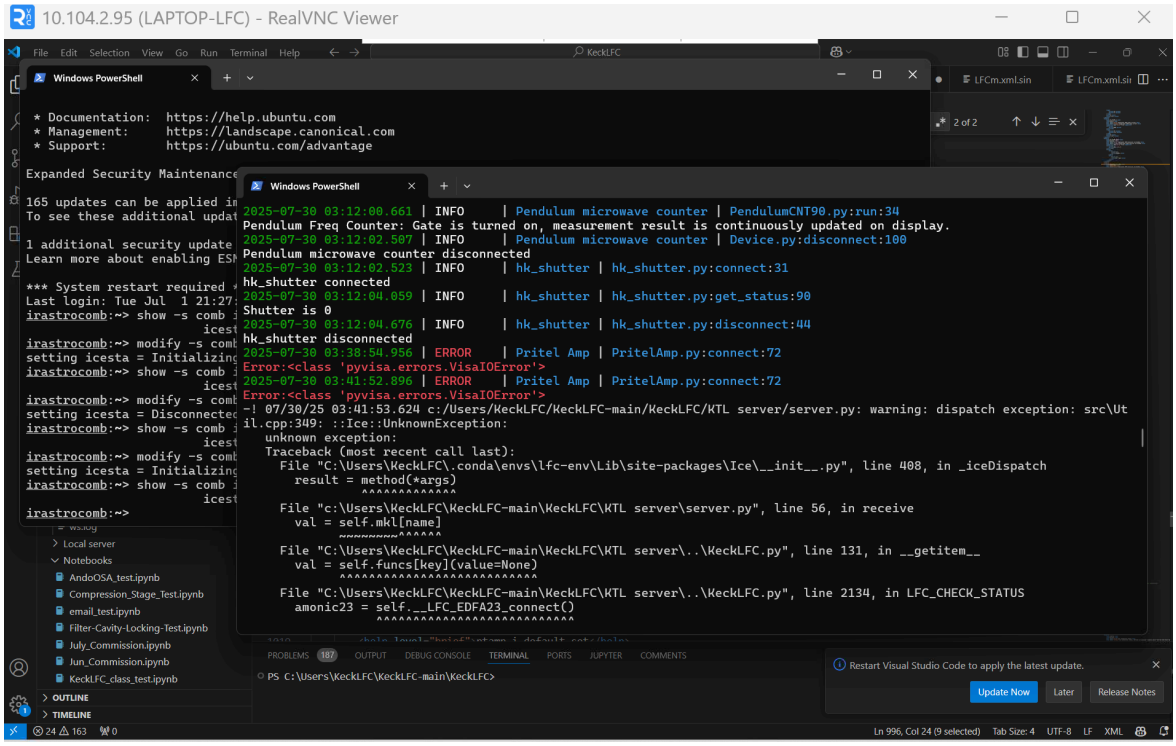


Figure 2.5. Example of a VISA error reported by the ICE server.

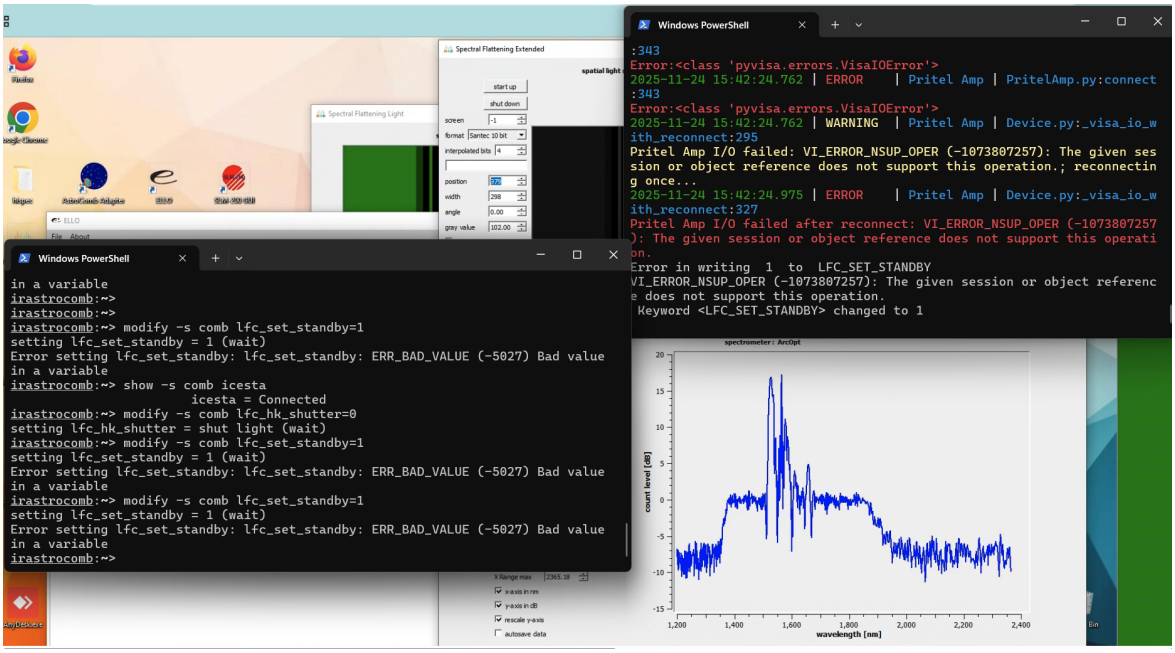


Figure 2.6. Additional VISA error details from the ICE server log.

In most cases this problem can be cleared by restarting both the ICE server and the KTL dispatcher. The recommended procedure is:

(1) **Stop the ICE server and disconnect the dispatcher.**
In the ICE server terminal press `Ctrl+C` to stop the server. From the KTL side, disconnect the dispatcher using `modify -s comb icesta=2`.

(2) **Restart the ICE server.**
Wait about 10s. Then restart the ICE server using the same command as before (you can use the ↑ key to recall the previous command in the terminal history). After the server has been running for roughly 10s, reconnect the dispatcher with `modify -s comb icesta=3`.

(3) **Verify that the dispatcher–server connection is restored.**
Wait another 5s for the dispatcher to complete the handshake with the ICE server. Then issue `modify -s comb lfc_set_standby=1`. Confirm that the comb power is successfully reduced (this should be visible on the Menlo spectrum display). It is important to verify this step before proceeding.

(4) **Clean shutdown after recovery.**
Once the VISA error has been cleared and the command has executed correctly, stop the ICE server again with `Ctrl+C`, and disconnect the dispatcher one more time with `modify -s comb icesta=2`.

In summary, the typical recovery path for a VISA error is to restart the ICE server, reconnect the dispatcher, re-issue the desired command, verify the comb response, and finally perform a clean shutdown of both server and dispatcher.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

CHAPTER 3

Instrument Control Summary

3.1 Amonics EDFA (`AmonicsEDFA`)

3.1.1 Introduction

The `AmonicsEDFA` driver controls an Amonics erbium-doped fiber amplifier (EDFA). It provides high-level access to key status signals (interlock, temperature, powers) and allows the user to configure the operation mode (ACC/APC), channel setpoints, channel on/off state, and the global (master) activation. The class inherits from the generic `Device` base class, which provides the underlying VISA/serial connection and logging.

3.1.2 Functions

Connection and status summary

`connect()` Open the VISA/serial connection if it is not already open.
Input: none.
Return: integer status (implementation dependent; typically nonzero on success).

`disconnect()` Close the connection if it is open.
Input: none.
Return: integer status (implementation dependent).

`printStatus()` Query the EDFA and generate a formatted multi-line status summary, including model number, serial number, interlock state, case temperature, activation status, channel mode, setpoint, and power readings. The summary is logged and returned as a string.
Input: none.
Return: `str`, human-readable summary.

Global status properties

`interLock` Read-only property reporting the hardware interlock state.
Get: `edfa.interLock` returns `'UNLOCKED'` or `'LOCKED'`.
Set: not supported (read-only).
Type: `str`.

`caseTemperature` Read-only property reporting the EDFA case temperature.

Get: edfa.caseTemperature returns the temperature in degrees Celsius.
Set: not supported (read-only).
Type: float.

modelNumber Read-only property for the model identifier string.
Get: edfa.modelNumber returns the model number.
Set: not supported (read-only).
Type: str.

serialNumber Read-only property for the serial number.
Get: edfa.serialNumber returns the serial number.
Set: not supported (read-only).
Type: str.

Channel 1 power monitors

All powers are expressed in milliwatts (mW).

outputPowerCh1 Read-only property for the optical output power of channel 1.
Get: edfa.outputPowerCh1 returns the output power in mW.
Set: not supported (read-only).
Type: float.

inputPowerCh1 Read-only property for the optical input power of channel 1, if supported by the hardware. If the instrument does not provide this monitor, the driver logs a warning and returns NaN.
Get: edfa.inputPowerCh1 returns the input power in mW, or numpy.nan.
Set: not supported (read-only).
Type: float.

PDPowerCh1 Read-only property for the internal photodiode reading on channel 1.
Get: edfa.PDPowerCh1 returns the internal PD power in mW.
Set: not supported (read-only).
Type: float.

Channel 1 mode, setpoint, and status

modeCh1 Property for the operation mode of channel 1: ACC (constant current) or APC (constant power). If a query fails, the getter defaults to 'ACC' as a safe mode.
Get: edfa.modeCh1 returns 'ACC' or 'APC'.
Set: edfa.modeCh1 = 'ACC' or edfa.modeCh1 = 'APC'; other values raise ValueError.
Type: str (for both get and set).

accCh1Cur Property for the channel 1 setpoint. In ACC mode this is the drive current (mA); in APC mode this is the target output power (mW). The driver enforces the hardware maximum and logs a warning when clipping.
Get: edfa.accCh1Cur returns the setpoint in mA or mW, depending on modeCh1.
Set: edfa.accCh1Cur = value, where value can be a numeric type (interpreted

as mA/mW) or a string with units (for example, "250mA" or "0.3A"). Invalid strings raise ValueError.
Type (get): float.
Allowed types (set): int, float, str.

accCh1Status Property for the channel 1 status flag. The driver maps internal codes to human-readable strings.
Get: edfa.accCh1Status returns one of 'OFF', 'ON', 'BUSY', or 'LOCK'.
Set: edfa.accCh1Status = v, where v is 0, 1, 'on', or 'off' (case-insensitive). Other values raise an exception. When setting the status, the driver polls the instrument until the requested state is reached or a timeout occurs; on timeout a runtime error is raised. If channel 1 is turned OFF while the global activation is ON, the driver also turns activation OFF and issues a warning.
Type (get): str.
Allowed types (set): int, str.

Channel 2 setpoint and status

If the specific EDFA model includes a second channel, the following properties mirror the channel 1 behaviour for channel 2.

accCh2Cur Property for the channel 2 setpoint (current or power, depending on mode). The semantics and unit rules are identical to accCh1Cur.
Get: edfa.accCh2Cur returns the setpoint in mA or mW.
Set: edfa.accCh2Cur = value, same accepted types as accCh1Cur.
Type (get): float.

accCh2Status Property for the channel 2 status flag. The semantics are identical to accCh1Status.
Get: edfa.accCh2Status returns one of 'OFF', 'ON', 'BUSY', or 'LOCK'.
Set: edfa.accCh2Status = v, where v is 0, 1, 'on', or 'off'.
Type (get): str.
Allowed types (set): int, str.

Master activation

activation Property for the global master activation state, equivalent to the front-panel "Activate" control. This determines whether the EDFA output is enabled. The driver enforces that channel 1 must already be ON before activation can be switched ON.
Get: edfa.activation returns 'OFF' or 'ON'.
Set: edfa.activation = v, where v is 0, 1, 'off', or 'on' (case-insensitive). If activation is set to ON while accCh1Status is not ON, the driver raises a ValueError. When changing activation, the driver polls the instrument until the requested state is reached or a timeout occurs; on timeout, a runtime error is raised. A safety warning is logged every time activation is turned ON.
Type (get): str.
Allowed types (set): int, str.

For further implementation details (including low-level SCPI commands and internal helper functions), please refer to the source file `AmonicsEDFA.py`.

3.2 Agilent 86142B optical spectrum analyzer (`Agilent_86142B`)

3.2.1 Introduction

The `Agilent_86142B` driver controls an Agilent 86142B optical spectrum analyzer (OSA). It provides high-level access to key configuration parameters (wavelength range, span, resolution, reference level, sensitivity), trace visibility, and data acquisition. The class inherits from the generic `Device` base class, which supplies the underlying VISA/GPIB connection and logging.

3.2.2 Functions

Connection and status summary

`connect()` Open the VISA/GPIB connection if it is not already open.
Input: none.
Return: integer status (implementation dependent; typically nonzero on success).

`disconnect()` Close the connection if it is open.
Input: none.
Return: integer status (implementation dependent).

`printStatus()` Generate and log a formatted status header for the Agilent 86142B OSA. At present, this is a placeholder summary string mainly for identification and logging.
Input: none.
Return: `str`, multi-line status block.

Configuration properties

All wavelength-related values are in nanometres (nm); power-related values are in dBm.

`resolution` Property for the resolution bandwidth of the OSA. Internally mapped to the resolution SCPI commands.
Get: `osa.resolution` returns the resolution in nm.
Set: `osa.resolution = value`, where `value` is converted to a floating-point value in nm.
Type (get): `float`.
Allowed types (set): `int`, `float`, `str`.

`sensitivity` Property for the lower end of the OSA power range, expressed as a sensitivity floor in dBm.
Get: `osa.sensitivity` returns the lower power limit in dBm.
Set: `osa.sensitivity = value`, where `value` is converted to a floating-point

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

dBm value.
Type (get): `float`.
Allowed types (set): `int`, `float`, `str`.

`reflevel` Property for the reference level of the displayed trace, in dBm.
Get: `osa.reflevel` returns the current reference level in dBm.
Set: `osa.reflevel = value`, where `value` is converted to a floating-point dBm value; the driver then confirms the setting and logs it.
Type (get): `float`.
Allowed types (set): `int`, `float`, `str`.

`wlstart` Property for the scan start wavelength.
Get: `osa.wlstart` returns the start wavelength in nm.
Set: `osa.wlstart = value`, where `value` is converted to a floating-point nm value. The driver logs the updated setting.
Type (get): `float`.
Allowed types (set): `int`, `float`, `str`.

`wlstop` Property for the scan stop wavelength.
Get: `osa.wlstop` returns the stop wavelength in nm.
Set: `osa.wlstop = value`, where `value` is converted to a floating-point nm value. The driver logs the updated setting.
Type (get): `float`.
Allowed types (set): `int`, `float`, `str`.

`wlspan` Property for the wavelength span, equal to `wlstop - wlstart`.
Get: `osa.wlspan` returns the span in nm.
Set: `osa.wlspan = value`, where `value` is interpreted as the desired span in nm. The driver keeps the current centre wavelength unchanged and adjusts start/stop accordingly; the new span is logged.
Type (get): `float`.
Allowed types (set): `int`, `float`, `str`.

`wlcenter` Property for the centre wavelength of the scan.
Get: `osa.wlcenter` returns the centre wavelength in nm.
Set: `osa.wlcenter = value`, where `value` is converted to a floating-point nm value. The driver keeps the current span unchanged and symmetrically updates start and stop wavelengths; the new centre is logged.
Type (get): `float`.
Allowed types (set): `int`, `float`, `str`.

Sweep control

`Run()` Start continuous spectrum acquisition (equivalent to the "Repeat" softkey on the front panel).
Input: none.
Return: none (status messages are logged).

`Single()` Trigger a single sweep.
Input: none.
Return: none (status messages are logged).

`Stop()` Stop continuous spectrum acquisition (stop "Repeat" mode).
Input: none.
Return: none (status messages are logged).

Trace visibility and feed mode

The OSA supports multiple traces identified by letters `'A'` through `'F'`. The driver also accepts the special keyword `'all'` to apply an operation to all traces at once. Trace names are case-insensitive.

`blank_trace(trace)` Hide one or more traces from the display.
Input: `trace` (`str`), one of `'A'`, `'B'`, `'C'`, `'D'`, `'E'`, `'F'`, or `'all'`. Invalid names raise a `ValueError`.
Return: none.

`disp_trace(trace)` Show one or more traces on the display.
Input: `trace` (`str`), same rules as `blank_trace`.
Return: none.

`fix_trace(trace)` Set the feed mode of the specified trace to "never" (the trace is frozen and no longer updated on new sweeps). This is equivalent to fixing the trace for reference.
Input: `trace` (`str`), one of `'A'`–`'F'` or `'all'`.
Return: none.

`update_trace(trace)` Set the feed mode of the specified trace to "always" (the trace is updated on each sweep).
Input: `trace` (`str`), one of `'A'`–`'F'` or `'all'`.
Return: none.

`write_trace(trace)` Compatibility wrapper; in this implementation it is marked as deprecated and simply calls `update_trace(trace)` after issuing a warning through the logger.
Input: `trace` (`str`), one of `'A'`–`'F'` or `'all'`.
Return: none.

Live update in Jupyter notebooks

The driver includes a simple live viewer intended for use inside a Jupyter environment. It repeatedly calls `get_trace()` in a background thread and updates a matplotlib figure.

`startosa(trace)` Start live acquisition of a given trace in a separate thread and display updates inline. If a live acquisition is already running, the call is ignored.
Input: `trace` (`str`), trace name (`'A'`–`'F'`).
Return: none.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

stoposa() Stop the live acquisition loop started by startosa().
Input: none.
Return: none.

Data acquisition

get_trace(trace, plot=True, filename=None) Acquire a single trace from the OSA and return the wavelength and power arrays. Optionally create and display a plot, and optionally save a PNG file. Wavelengths are constructed as a linear grid between the current start and stop wavelengths.
Input:
trace (str): trace name ('A'–'F'). Invalid names raise ValueError.
plot (bool): if True, create a matplotlib figure and show the spectrum.
filename (str or None): base filename; if not None, a PNG figure is saved using this base name.
Return: a tuple (wl, intensity), where both elements are numpy.ndarray objects: wl is the wavelength grid in nm and intensity is the power in dBm.

Saving traces to files

save_trace(trace, filename, extensions=['.mat', '.txt'], plot=True) Acquire a trace and save it to one or more file formats. If the target directory does not exist, it is created. If a file with the same name already exists, it is renamed with a timestamp and _bak suffix before the new file is written. Optionally plot the data during acquisition.
Input:
trace (str): trace name ('A'–'F').
filename (str): base path without extension.
extensions (list or str): one or more of '.mat', '.txt', '.fits', '.csv'. The special value 'all' enables all available formats.
plot (bool): if True, plot the trace while acquiring.
Return: none. Depending on extensions, the driver writes one or more of:

- a MATLAB .mat file with arrays OSAWavelength, OSAPower, and metadata such as resolution and timestamp;
- a plain-text .txt file with two columns (wavelength, power);
- a FITS .fits file with two-column data;
- a .csv file with two columns (wavelength, power).

For additional implementation details, including the exact SCPI commands used and internal helper logic, please refer to the source file Agilent_86142B.py.

3.3 Arduino relay module (Arduino_relay)

3.3.1 Introduction

The Arduino_relay driver controls a custom Arduino-based relay and shutter board used for safety interlock of the amplifier chain. It monitors an analog voltage proportional to input power, compares it to configurable low and high thresholds, and decides whether the system is safe to send an OK_to_Amplify signal. The board also controls a YJ shutter path and reports detailed status strings. The class inherits from the generic Device base class, but uses a direct serial interface instead of VISA.

3.3.2 Functions

Connection and status summary

`connect()` Open the serial connection to the Arduino relay module if it is not already open. The method configures the port, timeout, baud rate, and internal state.
Input: none.
Return: integer status (`1` on success, `0` if already connected, `-1` on failure).

`disconnect()` Close the serial connection if it is open.
Input: none.
Return: integer status (`1` on success, `0` if already disconnected).

`printStatus()` Query the relay module and assemble a formatted multi-line status summary. The summary includes the relay thresholds and voltages, the relay state, and the YJ shutter state. Certain keywords in the status string (such as `OK_to_Amplify` and `STOPPING`) are highlighted using terminal color codes. The summary is logged and returned as a string.
Input: none.
Return: `str`, human-readable status summary.

Information properties

`info_str_relay` Read-only property returning the raw multi-line status string reported by the relay module. Internally this is obtained by sending the `"GET"` command.
Get: `relay.info_str_relay` returns the raw status text for thresholds and voltages.
Set: not supported (read-only).
Type: `str`.

`info_str_YJ` Read-only property returning the raw status string for the YJ shutter state, obtained by sending the `"YJState"` command.
Get: `relay.info_str_YJ` returns the YJ state text.
Set: not supported (read-only).
Type: `str`.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Relay state control and information

reset_relay_latch() Reset the internal latch that holds a stopping state. This is used after the input power returns to a safe range to allow the relay to send an OK_to_Amplify signal again. Internally this sends the "reset" command.
Input: none.
Return: str response from the module, or None on communication failure.

get_relay_info() Query the relay module for a detailed multi-line description of thresholds, voltages, and other diagnostic information. Internally this sends the "get" command.
Input: none.
Return: str, raw diagnostic text.

get_relay_status() Return a high-level relay status string interpreted from the current thresholds and voltages. The method parses info_str_relay and computes whether the relay is sending OK_to_Amplify, is stopping the amplifier but could be reset, or is stopping due to too low or too high input power.
Input: none.
Return: str, one of:

- "relay sending OK_to_Amplify signal to amplifier"
- "relay is STOPPING amplifier, but will be OK_to_Amplify after reset_relay_la
- "relay is STOPPING amplifier, because input power is too low"
- "relay is STOPPING amplifier, because input power is too high"
- "relay is STOPPING amplifier"

YJ shutter control and information

shut_YJ() Command the board to close the YJ shutter. Internally this sends the "YJShut" command.
Input: none.
Return: str response from the module, or None on communication failure.

pass_YJ() Command the board to open the YJ shutter (pass state). Internally this sends the "YJPass" command.
Input: none.
Return: str response from the module, or None on communication failure.

get_YJ_info() Query the current YJ shutter state. Internally this sends the "YJState" command.
Input: none.
Return: str, human-readable YJ state.

Threshold configuration

Thresholds are expressed in ADC counts from 0 to 1023, corresponding to an input range of 0 V to 5 V.

`set_relay_low_threshold(low_threshold)` Set the low threshold used to evaluate the relay decision. If the requested value is outside $[0, 1023]$, the driver logs an error and returns `None`. For very low values (below 100) it logs a warning and interactively prompts on the console for user confirmation (`y/n`); if the user declines, the operation is cancelled. On acceptance, the method sends the `"THRESHOLD"` command with the given value.
Input: `low_threshold` (`int`), desired threshold between 0 and 1023.
Return: `str` response from the module, or `None` if the value is out of range or cancelled by the user.

`set_relay_high_threshold(high_threshold)` Set the high threshold used to evaluate the relay decision. If the requested value is outside $[0, 1023]$, the driver logs an error and returns `None`. For very high values (above 900) it logs a warning and prompts for user confirmation; if the user declines, the operation is cancelled. On acceptance, the method sends the `"HIGH"` command with the given value.
Input: `high_threshold` (`int`), desired threshold between 0 and 1023.
Return: `str` response from the module, or `None` if the value is out of range or cancelled by the user.

`get_relay_low_threshold()` Parse the relay status string to extract the configured low threshold.
Input: none.
Return: `int`, low threshold value in ADC counts.

`get_relay_high_threshold()` Parse the relay status string to extract the configured high threshold.
Input: none.
Return: `int`, high threshold value in ADC counts.

Voltage monitors

These methods parse the relay status string and report the current voltages used in the decision logic.

`get_current_low_voltage()` Return the internal transformed value labelled as `"Now voltage to judge (low)"` in the relay status text. This is used for the low-side decision.
Input: none.
Return: `int`, ADC counts corresponding to the current low-side decision voltage.

`get_current_high_voltage()` Return the internal transformed value labelled as `"Now voltage to judge (high)"` in the relay status text. This is used for the high-side decision.
Input: none.
Return: `int`, ADC counts corresponding to the current high-side decision voltage.

`get_current_voltage()` Return the current raw voltage reading labelled as `"Now`

voltage is" in the relay status text.
Input: none.
Return: int, ADC counts corresponding to the instantaneous measured voltage.

API help

get_API_help() Request and return the built-in help string from the Arduino relay firmware, which describes the available text commands and their meanings. Internally this sends the "help" command.
Input: none.
Return: str, multi-line help text.

For further implementation details, including low-level serial I/O and string parsing logic, please refer to the source file Arduino_relay.py.

3.4 Clarity wavelength locker (Clarity)

3.4.1 Introduction

The Clarity driver controls a Clarity wavelength locker used to stabilize a laser at a fixed optical frequency or wavelength. It configures the serial communication parameters, provides functions to start automatic calibration, enables or disables the output, configures periodic calibration, controls the lock status and lock position, and reads back the locked frequency and wavelength. The class inherits from the generic Device base class.

3.4.2 Functions

Connection and status

connect() Mark the device as connected and log a message. For this driver the connect method does not open a new VISA session; it only sets the internal connected flag and prints an informational message.
Input: none.
Return: none.

printStatus() Placeholder for a future status summary implementation. In the current version this method is not implemented and does nothing.
Input: none.
Return: none.

Automatic calibration and on/off control

auto_on() Start the internal automatic calibration and lock procedure. Internally this sends the CAL:INT command to the device.
Input: none.
Return: none.

`get_status()` Query the overall system status. The driver interprets the last character of the response as an integer code.
Input: none.
Return: `int`, where typical values are `0` (off), `1` (calibrating), `2` (locking), `3` (locked).

`get_onoff()` Query whether the source output is enabled. The driver inspects the last two characters of the response to determine the state.
Input: none.
Return: `int`, `1` for ON, `0` for OFF.

`set_onoff(onoff)` Enable or disable the source output. The argument is passed directly into the corresponding SCPI command. After writing, the method waits briefly and then reads back the state using `get_onoff()`.
Input: `onoff` (string or integer), typically `'ON'`, `'OFF'`, `1`, or `0`.
Return: `int`, the current state as reported by `get_onoff()` (`1` for ON, `0` for OFF).

Periodic calibration

`get_periodic_calibration()` Query the periodic calibration interval. The driver inspects the last two characters of the response; if they are `'FF'`, periodic calibration is treated as disabled and the method returns zero. Otherwise the last two characters are converted to an integer.
Input: none.
Return: `int`, `0` if disabled, or a positive integer representing the configured interval (device-specific units).

`set_periodic_calibration(t)` Configure the periodic calibration interval. If the argument is zero, the driver sends the keyword `OFF` to disable periodic calibration; otherwise it sends the integer value. After writing, it waits briefly and returns the new interval as reported by `get_periodic_calibration()`.
Input: `t` (`int`), `0` to disable or a positive integer interval.
Return: `int`, the effective interval from `get_periodic_calibration()`.

Lock status and lock position

`get_lock_status()` Query whether the system is currently in lock mode. The driver inspects the last two characters of the response; `'ON'` is interpreted as locked, `'FF'` as unlocked.
Input: none.
Return: `int`, `1` if locked, `0` if unlocked.

`set_lock_status(onoff)` Enable or disable lock mode. The argument is passed directly to the corresponding SCPI command, and the method returns the new status as reported by `get_lock_status()`.
Input: `onoff` (string or integer), typically `'ON'`, `'OFF'`, `1`, or `0`.
Return: `int`, `1` if locked, `0` if unlocked.

set_lock_position(pos) Select the lock position used by the internal algorithm. The driver supports two discrete positions and maps them to the appropriate SCPI keywords.
Input: pos (int), 0 for "LEFT" mode or 1 for "CENT" (center) mode. Other values are ignored.
Return: none.

Frequency and wavelength readout

get_frequency() Query the current source frequency. The driver sends the query command, waits for the instrument to respond, then reads until the termination token 'THz' and parses the trailing numeric part.
Input: none.
Return: float, optical frequency in terahertz (THz).

get_wavelength() Query the current source wavelength. The driver sends the query command, waits briefly, then reads until the termination token 'nm' and parses the trailing numeric part.
Input: none.
Return: float, optical wavelength in nanometres (nm).

Password management

enter_password(password=5000) Enter a password to unlock protected functions on the device. The password is passed directly to the appropriate SCPI command.
Input: password (int or str), password value; default is 5000.
Return: none.

set_password(pass1, pass2) Change the device password. The instrument expects the old and new password in a specific colon-separated format; the driver passes the two arguments directly into that format.
Input: pass1, pass2 (int or str), password fields as required by the device (typically old and new password).
Return: none.

For additional implementation details, including low-level SCPI command sequences and serial configuration, please refer to the source file Clarity.py.

3.5 Eaton PDU (EatonPDU)

3.5.1 Introduction

The EatonPDU driver controls an Eaton ePDU DC power distribution unit. It manages login/logout on the command-line interface, queries input power metrics (frequency, voltage, current, active and apparent power), and provides high-level access to the status and control of each outlet (present status, friendly name, switchable flag). The

class inherits from the generic Device base class and uses a serial session configured for the Eaton ePDU command-shell.

3.5.2 Functions

Connection, login, and status summary

connect(login=True) Open the underlying Device connection and optionally log in to the PDU shell. The method first calls the base-class connect and then, if login is True, calls login().
Input: login (bool), whether to perform a shell login immediately after connecting (default True).
Return: tuple (cnt, lgin), where cnt is the status from Device.connect(), and lgin is the status from login().

disconnect(logout=True) Optionally log out from the PDU shell and then close the underlying connection.
Input: logout (bool), whether to issue logout() before closing the connection (default True).
Return: tuple (dcnt, lgot), where dcnt is the status from Device.disconnect(), and lgot is the status from logout().

login() Log in to the PDU command-shell using the stored username and password. If already logged in, a message is logged and no further action is taken. Multiple attempts are made up to max_login_attempt.
Input: none.
Return: 1 on successful login, 0 if already logged in, -1 if all attempts fail.

logout() Log out of the PDU command-shell if currently logged in.
Input: none.
Return: 1 on successful logout, 0 if already logged out.

loggedin Read-only property indicating whether the driver is currently logged in to the PDU shell. Internally this uses test_loggedin().
Get: pdu.loggedin returns True if logged in, False otherwise.
Set: not supported (read-only).
Type: bool.

test_loggedin(relogin=False) Check whether the current shell session is logged in. If relogin is True and the test fails, the method attempts to log in again.
Input: relogin (bool), whether to automatically re-login on failure.
Return: True if a valid prompt is detected, otherwise False.

printStatus() Build and log a formatted summary of the PDU status, including IPv4 address, input system metrics, and outlet system metrics. The summary is logged and also returned as a string.
Input: none.
Return: str, multi-line status summary.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

get_all_status() Query a snapshot of the full system status. This includes the IPv4 address, a dictionary for the input system, and a dictionary for the outlet system.
Input: none.
Return: dict with keys 'ipv4', 'InputSystem', and 'OutletSystem'.

Network information

get_ipv4_address(test_loggedin=True) Query the IPv4 address of the PDU. Optionally verify that the driver is logged in before sending the command.
Input: test_loggedin (bool), whether to test (and possibly re-login) before the query (default True).
Return: str, IPv4 address (for example, "192.168.0.160").

Input system measurements

All input power quantities are for Input[1] of the PDU.

get_input_frequency(test_loggedin=True) Get the input line frequency.
Input: test_loggedin (bool), whether to ensure a valid login before querying.
Return: float, frequency in hertz (Hz).

get_input_phase_count(test_loggedin=True) Get the number of input phases.
Input: test_loggedin (bool).
Return: int, number of phases.

get_input_active_power_onphase(phase=1, test_loggedin=True) Get the per-phase active power.
Input: phase (int), phase index starting from 1; test_loggedin (bool). The method checks that phase does not exceed the phase count.
Return: float, active power in watts (W).

get_input_apparent_power_onphase(phase=1, test_loggedin=True) Get the per-phase apparent power.
Input: phase (int); test_loggedin (bool).
Return: float, apparent power in volt-amperes (VA).

get_input_current_onphase(phase=1, test_loggedin=True) Get the per-phase current.
Input: phase (int); test_loggedin (bool).
Return: float, current in amperes (A).

get_input_voltage_onphase(phase=1, test_loggedin=True) Get the per-phase voltage.
Input: phase (int); test_loggedin (bool).
Return: float, voltage in volts (V).

get_input_active_power(test_loggedin=True) Get the total active power at the input.

Input: test_loggedin (bool).
Return: float, active power in watts (W).

get_input_apparent_power(test_loggedin=True) Get the total apparent power at the input.
Input: test_loggedin (bool).
Return: float, apparent power in volt-amperes (VA).

get_input_system_status() Collect a full snapshot of the input system, including frequency, total active and apparent power, phase count, and a per-phase dictionary for each phase (active power, apparent power, current, voltage). The method also logs a formatted status string.
Input: none.
Return: dict with keys: 'Frequency_Hz', 'ActivePower_W', 'ApparentPower_VA', 'PhaseCount', and nested dictionaries 'Phase1', 'Phase2', ...

Outlet system overview

get_outlet_count(test_loggedin=True) Get the total number of outlets supported by the PDU.
Input: test_loggedin (bool).
Return: int, number of outlets.

check_outlet_available(outlet, test_loggedin=True) Verify that the requested outlet index is within the valid range. If it is out of range, a ValueError is raised.
Input: outlet (int), outlet index starting from 1; test_loggedin (bool).
Return: True if the outlet index is valid.

get_outlet_status(outlet) Get a snapshot of the status for a single outlet. This includes present on/off state, switchable flag, friendly name, and electrical measurements. A formatted description is logged.
Input: outlet (int), outlet index starting from 1.
Return: dict with keys 'PresentStatus', 'Switchable', 'FriendlyName', 'Current_A', 'ActivePower_W', 'ApparentPower_VA', 'PowerFactor'.

get_outlet_status_all() Get a snapshot of the status for all outlets. The method first logs a brief summary, then iterates over each outlet and calls get_outlet_status().
Input: none.
Return: dict with keys 'OutletCount' and 'Outlet1', 'Outlet2', ..., *eachcontaininganout*

Outlet naming

assign_outlet_name() Assign user-defined friendly names from outlet_name_list to all outlets. The list length must match the number of outlets; if not, a ValueError is raised. For each outlet, the old name is logged, a new name is written, and a small delay is inserted between commands.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Input: none (uses the `outlet_name_list` attribute).
Return: none.

`get_outlet_friendly_name(outlet, test_loggedin=True, test_available=True)`
Get the configured friendly name of an outlet.
Input: `outlet` (`int`); optional `test_loggedin` and `test_available` flags (normally left at their default `True`).
Return: `str`, friendly name.

`set_outlet_friendly_name(outlet, name, test_loggedin=True, test_available=True)`
Set the friendly name of an outlet. Names longer than 31 characters are truncated and a warning is logged. The new name is written to the device and confirmed.
Input: `outlet` (`int`), outlet index; `name` (`str`), new friendly name (will be truncated to at most 31 characters); optional flags `test_loggedin`, `test_available`.
Return: `str`, the device response string.

Outlet measurements

`get_outlet_current(outlet, test_loggedin=True, test_available=True)` Get the RMS current of an outlet.
Input: `outlet` (`int`); optional `test_loggedin`, `test_available`.
Return: `float`, current in amperes (A).

`get_outlet_active_power(outlet, test_loggedin=True, test_available=True)`
Get the active power of an outlet.
Input: `outlet` (`int`); optional flags as above.
Return: `float`, active power in watts (W).

`get_outlet_apparent_power(outlet, test_loggedin=True, test_available=True)`
Get the apparent power of an outlet.
Input: `outlet` (`int`); optional flags.
Return: `float`, apparent power in volt-amperes (VA).

`get_outlet_power_factor(outlet, test_loggedin=True, test_available=True)`
Get the power factor of an outlet.
Input: `outlet` (`int`); optional flags.
Return: `float`, power factor (unitless).

Outlet status and switching

`get_outlet_present_status(outlet, test_loggedin=True, test_available=True)`
Get the present on/off status of an outlet. The method queries the `PresentStatus.SwitchOnOff` field.
Input: `outlet` (`int`); optional flags `test_loggedin`, `test_available`.
Return: `int`, `1` if the outlet is ON, `0` if it is OFF.

`get_outlet_switchable(outlet, test_loggedin=True, test_available=True)`
Check whether switching is allowed on the given outlet. The method returns a boolean independent of whether the PDU hardware itself supports switching;

this is a user-configurable permission flag.
Input: `outlet` (`int`); optional flags.
Return: `True` if the outlet is marked switchable, `False` if it is not; an invalid device response raises `ValueError`.

`set_outlet_switchable(outlet, switchable)` Set the user-configurable switchable flag of an outlet.
Input: `outlet` (`int`); `switchable` (`bool`), desired permission state.
Return: `str`, device response string.

`check_passcode(passcode)` Check whether the given passcode matches the preset `passcode` attribute. This is used as a simple safety guard before switching outlets.
Input: `passcode` (`str`).
Return: `True` if the passcode matches, `False` otherwise.

`set_outlet_on(outlet)` Interactively turn an outlet ON. If the outlet is already ON, the method logs this and returns immediately. Otherwise it prompts on the console for user confirmation (`y/n`) and a passcode; only if both are accepted does it proceed to switch on the outlet (after verifying that the outlet is switchable).
Input: `outlet` (`int`).
Return: `True` if the command is accepted and executed, `False` if aborted, or an integer present status from the underlying command; the exact return depends on the path taken.

`set_outlet_off(outlet)` Interactively turn an outlet OFF. The logic mirrors `set_outlet_on`: if the outlet is already OFF, the method returns immediately; otherwise it prompts for confirmation and a passcode before switching off.
Input: `outlet` (`int`).
Return: `True` if the command is accepted and executed, `False` if aborted, or an integer present status from the underlying command.

For additional implementation details, including low-level parsing of device responses and private helper methods, please refer to the source file `EatonPDU.py`.

3.6 hk shutter controller (`hk_shutter`)

3.6.1 Introduction

The `hk_shutter` driver controls a serial-based shutter controller (Heimann/Thorlabs-style HK shutter) over a direct COM port. It configures the serial port parameters (baud rate, parity, stop bits, timeout) and provides high-level methods to query and change the shutter status (open/closed) and the operating mode (manual, auto, single, repeat, external gate). The class inherits from the generic `Device` base class but uses a raw `pyserial` connection instead of VISA.

3.6.2 Functions

Connection and status

`connect()` Open the serial connection to the shutter controller if it is not already open. The method configures the COM port, timeout, baud rate, parity, stop bits, and data bits. On success it sets the internal `connected` flag and logs a message.
Input: none.
Return: `1` on success, `0` if already connected, `-1` on failure.

`disconnect()` Close the serial connection if it is open and clear the internal `connected` flag. A message is logged on success.
Input: none.
Return: `1` on success, `0` if already disconnected.

Shutter status

`get_status()` Query the current shutter status. The driver flushes the serial buffers, sends the command `"ens?\r"`, reads the response bytes, and interprets a single status character. If the device returns `'0'` the method reports status `0`; if it returns `'1'` it reports status `1`. Any other response is treated as an error.
Input: none.
Return: `0` if the shutter reports state "0", `1` if the shutter reports state "1", `-1` if the device does not respond as expected.

`set_status(status)` Change the shutter status if needed. The method first flushes the buffers and calls `get_status()` to read the current state. If the requested `status` is already active, it prints a message and returns the current value. If the requested `status` differs from the current one, the method sends the command `"ens\r"`, reads the reply, and then calls `get_status()` again to confirm the new state.
Input: `status` (`int`), desired state (`0` or `1`). The method does not enforce a specific mapping between numeric values and physical open/closed state; this mapping depends on the shutter hardware.
Return: `0` or `1` reflecting the shutter status after the command, or `-1` if the status cannot be determined.

Shutter mode

`get_mode()` Query the current operating mode of the shutter controller. The driver flushes the buffers, sends the command `"mode?\r"`, reads the reply, and inspects a single mode character. The following mappings are implemented:

- `1`: manual mode
- `2`: auto mode
- `3`: single mode
- `4`: repeat mode
- `5`: external gate mode

Any other value is treated as an error. For convenience, the method prints a textual description of the mode before returning.
Input: none.
Return: `int` mode code (1–5) on success, `-1` if the response is not one of the supported mode codes.

`set_mode(mode)` Change the operating mode of the shutter controller. The method flushes the buffers and calls `get_mode()` to obtain the current mode. If `mode` equals the current mode, it logs a message and returns the current mode code. If `mode` differs, the method sends the command `"mode=<mode>\r"`, reads back the reply bytes, then calls `get_mode()` again to confirm the new mode and logs a human-readable message.
Input: `mode` (`int`), desired mode code (`1` for manual, `2` for auto, `3` for single, `4` for repeat, `5` for external gate). Other values will be sent to the controller but are not recognised by the decoder and will typically result in `-1` being returned by `get_mode()`.
Return: `int` mode code reported by `get_mode()` after the command, or `-1` on error.

For additional implementation details, including low-level serial buffer handling and timing, please refer to the source file `hk_shutter.py`.

3.7 Instek GPD-4303S DC power supply (`InstekGPD_4303S`)

3.7.1 Introduction

The `InstekGPD_4303S` driver controls a GW Instek GPD-4303S four-channel DC power supply. It configures the serial communication parameters and provides high-level access to the master output activation, tracking/operation mode, per-channel voltage and current setpoints, and measured outputs. Safety limits for each channel are enforced in software to prevent programming voltages or currents beyond the instrument capabilities. The class inherits from the generic `Device` base class.

3.7.2 Functions

Connection and status

`connect()` Open the underlying VISA/serial connection if it is not already open.
Input: none.
Return: integer status (implementation dependent; typically nonzero on success).

`disconnect()` Close the connection if it is open.
Input: none.
Return: integer status (implementation dependent).

`printStatus()` Query the supply and build a multi-line status summary. The sum-

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

mary includes model and serial number, master activation state, global operation mode, per-channel mode (CC/CV), set voltages and currents, measured output voltages and currents, and computed output power for each channel. The summary is logged and returned as a string.
Input: none.
Return: `str`, human-readable status summary.

`modelNumber` Read-only property for the device model identifier.
Get: `dcsp.modelNumber` returns the model string (for example, `"GPD-4303S"`).
Set: not supported (read-only).
Type: `str`.

`serialNumber` Read-only property for the device serial number.
Get: `dcsp.serialNumber` returns the serial number.
Set: not supported (read-only).
Type: `str`.

Master activation and operation mode

`activation` Property for the global output activation, equivalent to the front panel output ON/OFF control. Internally this is derived from the device status code.
Get: `dcsp.activation` returns `'OFF'` if the outputs are disabled, `'ON'` if enabled.
Set: `dcsp.activation = status`, where `status` is one of `0`, `1`, `'off'`, or `'on'` (case-insensitive). Invalid values raise `ValueError`. The setter sends the appropriate command to switch all outputs on or off and logs a message indicating the new state.
Type (get): `str`.
Allowed types (set): `int`, `str`.

`operationMode` Property for the tracking mode of channels 1 and 2. The supply supports independent, series, and parallel tracking modes.
Get: `dcsp.operationMode` returns one of `'indep'`, `'series'`, or `'parallel'` based on the status code.
Set: `dcsp.operationMode = mode`, where `mode` is one of `'indep'`, `'series'`, or `'parallel'` (case-insensitive). Invalid values raise `ValueError`. The setter writes the corresponding tracking code and logs the selected mode.
Type (get): `str`.
Allowed types (set): `str`.

Channel 1 properties

All voltages are in volts (V) and currents are in amperes (A).

`Iout1` Property for the measured output current on channel 1.
Get: `dcsp.Iout1` returns the measured current in A.
Set: `dcsp.Iout1 = I` forwards to `Iset1 = I`; this convenience setter exists for clarity but in normal use the setpoint should be adjusted via `Iset1`.

Type (get): `float`.
Allowed types (set): numeric.

`Iset1` Property for the programmed current limit (setpoint) of channel 1. The driver enforces the hardware limit of 3 A on this channel.
Get: `dcsp.Iset1` returns the set current in A.
Set: `dcsp.Iset1 = Iset1` sets the channel 1 current to `Iset1` (A). If the requested current exceeds 3 A a `ValueError` is raised. The driver writes the value, polls the instrument until the read-back matches within 0.001 A, or raises a `RuntimeError` on timeout.
Type (get): `float`.
Allowed types (set): numeric.

`Vout1` Property for the measured output voltage on channel 1.
Get: `dcsp.Vout1` returns the measured voltage in V.
Set: `dcsp.Vout1 = V` forwards to `Vset1 = V`; in normal use the setpoint should be adjusted via `Vset1`.
Type (get): `float`.
Allowed types (set): numeric.

`Vset1` Property for the programmed voltage setpoint of channel 1. The driver enforces the hardware limit of 30 V on this channel.
Get: `dcsp.Vset1` returns the set voltage in V.
Set: `dcsp.Vset1 = Vset1` sets the channel 1 voltage to `Vset1` (V). If the requested voltage exceeds 30 V a `ValueError` is raised. The driver writes the value, polls the instrument until the read-back matches within 0.001 V, or raises a `RuntimeError` on timeout.
Type (get): `float`.
Allowed types (set): numeric.

Channel 2 properties

Channel 2 behaves identically to channel 1 with the same limits (30 V, 3 A).

`Iout2` Measured output current on channel 2. The setter forwards to `Iset2`.
Get: `dcsp.Iout2` returns current in A.
Set: `dcsp.Iout2 = I` is equivalent to `Iset2 = I`.
Type (get): `float`.

`Iset2` Programmed current setpoint for channel 2, limited to 3 A.
Get: `dcsp.Iset2` returns the set current in A.
Set: `dcsp.Iset2 = Iset2` with the same safety and timeout behaviour as `Iset1`.
Type (get): `float`.

`Vout2` Measured output voltage on channel 2. The setter forwards to `Vset2`.
Get: `dcsp.Vout2` returns voltage in V.
Set: `dcsp.Vout2 = V` is equivalent to `Vset2 = V`.
Type (get): `float`.

`Vset2` Programmed voltage setpoint for channel 2, limited to 30 V.
Get: `dcsp.Vset2` returns the set voltage in V.
Set: `dcsp.Vset2 = Vset2` with the same safety and timeout behaviour as `Vset1`.
Type (get): `float`.

Channel 3 properties

Channel 3 has a combined voltage–current limit: for 0–5 V the current may be up to 3 A, while for 5–10 V the current must not exceed 1 A.

`Iout3` Measured output current on channel 3. The setter forwards to `Iset3`.
Get: `dcsp.Iout3` returns current in A.
Set: `dcsp.Iout3 = I` is equivalent to `Iset3 = I`.
Type (get): `float`.

`Iset3` Programmed current setpoint for channel 3. The driver enforces the combined limits:

- absolute maximum 3 A;
- if the current is above 1 A, the voltage setpoint must be below or equal to 5 V;
- if the voltage setpoint is above 5 V, the current must be less than or equal to 1 A.

Violating these constraints raises `ValueError`.
Get: `dcsp.Iset3` returns the set current in A.
Set: `dcsp.Iset3 = Iset3` with the same polling and timeout behaviour as other channels.
Type (get): `float`.

`Vout3` Measured output voltage on channel 3. The setter forwards to `Vset3`.
Get: `dcsp.Vout3` returns voltage in V.
Set: `dcsp.Vout3 = V` is equivalent to `Vset3 = V`.
Type (get): `float`.

`Vset3` Programmed voltage setpoint for channel 3, limited to 10 V. If the requested voltage is above 10 V a `ValueError` is raised. If the requested voltage is above 5 V and the current setpoint exceeds 1 A, a `ValueError` is raised advising to decrease `Iset3` first.
Get: `dcsp.Vset3` returns the set voltage in V.
Set: `dcsp.Vset3 = Vset3` with the same polling and timeout behaviour as other channels.
Type (get): `float`.

Channel 4 properties

Channel 4 is a low-voltage auxiliary output with maximum 5 V and 1 A.

`Iout4` Measured output current on channel 4. The setter forwards to `Iset4`.
Get: `dcsp.Iout4` returns current in A.

Set: `dcsp.Iout4 = I` is equivalent to `Iset4 = I`.
Type (get): `float`.

`Iset4` Programmed current setpoint for channel 4, limited to 1 A.
Get: `dcsp.Iset4` returns the set current in A.
Set: `dcsp.Iset4 = Iset4`; values above 1 A raise `ValueError`, and the driver uses the same polling and timeout scheme as other channels.
Type (get): `float`.

`Vout4` Measured output voltage on channel 4. The setter forwards to `Vset4`.
Get: `dcsp.Vout4` returns voltage in V.
Set: `dcsp.Vout4 = V` is equivalent to `Vset4 = V`.
Type (get): `float`.

`Vset4` Programmed voltage setpoint for channel 4, limited to 5 V.
Get: `dcsp.Vset4` returns the set voltage in V.
Set: `dcsp.Vset4 = Vset4`; values above 5 V raise `ValueError`, and the driver uses the same polling and timeout scheme as other channels.
Type (get): `float`.

Convenience methods

`setAllZero()` Set all four channels to zero voltage and zero current. Internally, this calls the private setpoint methods for both current and voltage on channels 1–4.
Input: none.
Return: none.

`setIMaxAll()` Set all channels to convenient high currents within their safe limits:

- channel 1: 3 A,
- channel 2: 3 A,
- channel 3: 1 A (safe default across the full voltage range),
- channel 4: 1 A.

This is intended for quick setup; voltage limits still apply.
Input: none.
Return: none.

`beep(status=1)` Control or trigger the power supply beeper. If `status` is `0` or `'off'`, the method sends a command to turn off the beep and returns 0. If `status` is `1` or `'on'`, the method first turns the beep off, then on again, in order to generate an audible beep, and returns 1. Any other value raises `ValueError`.
Input: `status` (`int` or `str`), one of `0`, `1`, `'off'`, `'on'` (case-insensitive).
Return: `0` if the beep is turned off, `1` if a beep is generated.

For further implementation details, including low-level SCPI commands, status code decoding, and internal helper methods, please refer to the source file `InstekGPD_4303S.py`.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

3.8 Instek GPP DC power supply (InstekGppDCSupply)

3.8.1 Introduction

The InstekGppDCSupply driver controls a GW Instek GPP series programmable DC power supply (for example GPP-1326, GPP-2323, GPP-3323, GPP-4323). It configures the serial communication parameters and provides high-level access to identification, over-voltage and over-current protection (OVP/OCP), per-channel output status, and per-channel voltage and current setpoints. Software limits are enforced to prevent programming currents beyond the instrument capabilities (6.2A maximum). The class inherits from the generic Device base class.

3.8.2 Functions

Status and identification

printStatus() Query the supply and build a multi-line status summary for all valid channels. For each channel the summary reports activation state, voltage and current setpoints (VSET, ISET), measured output voltage and current (VOUT, IOUT), calculated output power, and the OVP/OCP status and levels. The status is logged and also returned as a string.
Input: none.
Return: str, human-readable status summary.

modelNumber Read-only property for the device model identifier.
Get: psu.modelNumber returns the model string (for example "GPP-4323").
Set: not supported (read-only).
Type: str.

serialNumber Read-only property for the device serial number.
Get: psu.serialNumber returns the serial number.
Set: not supported (read-only).
Type: str.

Channel 1 basic output properties

All voltages are in volts (V), currents in amperes (A), power in watts (W).

Iset1 Property for the programmed current setpoint on channel 1. The driver enforces a maximum of 6.2A. The current is set with 1 mA resolution and verified by polling until the read-back matches within 0.001A, or a timeout is reached.
Get: psu.Iset1 returns the set current in A.
Set: psu.Iset1 = current, where current is a numeric value in A; values above 6.2A raise ValueError.
Type (get): float.
Allowed types (set): numeric.

Iout1 Read-only property for the measured output current on channel 1. The value is parsed from a string with units returned by the instrument (for example

"1.234A" or "500mA"). A convenience setter is provided that forwards to `Iset1`.
Get: `psu.Iout1` returns the measured current in A.
Set: `psu.Iout1 = I` is equivalent to `psu.Iset1 = I`.
Type (get): `float`.

`Vset1` Property for the programmed voltage setpoint on channel 1. The driver uses 1 mV resolution and verifies the setting by polling until the read-back matches within 0.001V, or a timeout is reached. The detailed voltage limit depends on the specific GPP model and is handled by the instrument.
Get: `psu.Vset1` returns the set voltage in V.
Set: `psu.Vset1 = voltage`, where `voltage` is a numeric value in V.
Type (get): `float`.
Allowed types (set): numeric.

`Vout1` Read-only property for the measured output voltage on channel 1. Internally the value is parsed from a string with units (for example "12.000V"). A convenience setter is provided that forwards to `Vset1`.
Get: `psu.Vout1` returns the measured voltage in V.
Set: `psu.Vout1 = V` is equivalent to `psu.Vset1 = V`.
Type (get): `float`.

`Pout1` Read-only property for the output power of channel 1, computed as `Vout1 * Iout1`.
Get: `psu.Pout1` returns the output power in W.
Set: not supported (derived quantity).
Type: `float`.

Channel 1 OVP/OCP protection

`OVPStatus1` Property for the over-voltage protection (OVP) status on channel 1.
Get: `psu.OVPStatus1` returns `'ON'` or `'OFF'` as reported by the instrument.
Set: `psu.OVPStatus1 = status`, where `status` is one of `0`, `1`, `'off'`, or `'on'` (case-insensitive). The driver writes the new state and polls until the instrument confirms it, or raises a `RuntimeError` if the change does not complete within the timeout.
Type (get): `str`.
Allowed types (set): `int`, `str`.

`OVPLevel1` Property for the OVP trigger level on channel 1.
Get: `psu.OVPLevel1` returns the OVP threshold in V.
Set: `psu.OVPLevel1 = voltage`, where `voltage` is a numeric value in V. The driver writes the new level and logs the actual value read back from the supply.
Type (get): `float`.
Allowed types (set): numeric.

`OCPStatus1` Property for the over-current protection (OCP) status on channel 1.
Get: `psu.OCPStatus1` returns `'ON'` or `'OFF'` as reported by the instrument.
Set: `psu.OCPStatus1 = status`, where `status` is one of `0`, `1`, `'off'`, or `'on'`

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

(case-insensitive). The driver writes the new state and polls until the instrument confirms it, or raises a `RuntimeError` on timeout.
Type (get): `str`.
Allowed types (set): `int`, `str`.

`OCPLevel1` Property for the OCP trigger level on channel 1.
Get: `psu.OCPLevel1` returns the OCP threshold in A.
Set: `psu.OCPLevel1 = current`, where `current` is a numeric value in A. The driver writes the new level and logs the value read back from the supply.
Type (get): `float`.
Allowed types (set): numeric.

Channel 1 activation

`activation1` Property for the output activation of channel 1. This corresponds to the per-channel output ON/OFF state.
Get: `psu.activation1` returns `'ON'` or `'OFF'`.
Set: `psu.activation1 = status`, where `status` is one of `0`, `1`, `'off'`, or `'on'` (case-insensitive). The driver writes the requested state and polls until the instrument confirms it, or raises a `RuntimeError` if the change does not complete within the timeout.
Type (get): `str`.
Allowed types (set): `int`, `str`.

Channels 2–4

Internally, the driver supports up to four channels and uses the device model to validate which channels are available:

- `GPP-1326`: channel 1 only,
- `GPP-2323`: channels 1–2,
- `GPP-3323`: channels 1–3,
- `GPP-4323`: channels 1–4.

The `printStatus()` method reports status for all valid channels. Programmatic access to channels 2–4 is implemented via internal helper methods with a `channel` argument. For typical astrocomb scripts, direct user-level control is provided explicitly for channel 1 through the properties listed above.

For additional implementation details, including low-level SCPI command formats, channel validation, and internal helper methods, please refer to the source file `InstekGppDCSupply.py`.

3.9 Keysight 33500 function generator (KeysightFG_33500)

3.9.1 Introduction

The KeysightFG_33500 driver controls a Keysight 33500 series function generator via SCPI. It provides high-level methods to select the output waveform (sine, square, ramp, DC), configure frequency, amplitude, offset, and phase for each channel, enable or disable the outputs, and configure trigger and sync behaviour. The class inherits from the generic Device base class.

3.9.2 Functions

Connection and identification

`connect()` Open the underlying VISA/USB connection if it is not already open.
Input: none.
Return: integer status (implementation dependent; typically nonzero on success).

`disconnect()` Close the connection if it is open.
Input: none.
Return: integer status (implementation dependent).

`IDN` Read-only property returning the instrument identification string (SCPI `*IDN?`).
Get: `fg.IDN` returns a comma-separated ID string (vendor, model, serial, firmware).
Set: not supported (read-only).
Type: `str`.

Waveform selection

For all methods below, `channel` is typically `1` or `2`, depending on the generator model.

`sin(channel)` Set the specified channel to a sine waveform.
Input: `channel` (`int` or `str`), output channel index.
Return: none.

`square(channel)` Set the specified channel to a square waveform.
Input: `channel` (`int` or `str`).
Return: none.

`ramp(channel)` Set the specified channel to a ramp waveform.
Input: `channel` (`int` or `str`).
Return: none.

`dc_voltage(channel)` Set the specified channel to DC output mode.
Input: `channel` (`int` or `str`).
Return: none.

`set_channel_func(channel, func)` Convenience method to select the waveform by name. Valid names are `"sine"`, `"sin"`, `"square"`, `"squ"`, `"ramp"`, and `"dc"`

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

(case-insensitive). Other values raise `ValueError`. After setting, the method prints the current function as reported by the instrument.
Input: `channel` (`int` or `str`); `func` (`str`), waveform name.
Return: none.

Channel output parameters

All amplitudes are in volts peak-to-peak (Vpp), offsets in volts (V), phases in degrees, and frequencies in hertz (Hz).

`set_channel_amplitude(channel, amp)` Set the output amplitude of a channel.
Input: `channel` (`int` or `str`); `amp` (numeric), amplitude in Vpp.
Return: none.

`set_channel_offset(channel, offset)` Set the DC offset of a channel.
Input: `channel` (`int` or `str`); `offset` (numeric), offset in V.
Return: none.

`set_channel_phase(channel, phase)` Set the phase of a channel.
Input: `channel` (`int` or `str`); `phase` (numeric), phase in degrees.
Return: none.

`set_channel_frequency(channel, freq)` Set the output frequency of a channel.
Input: `channel` (`int` or `str`); `freq` (numeric), frequency in Hz.
Return: none.

`set_channel_state(channel, state)` Enable or disable the output of a channel. The following values are accepted (case-insensitive): `"off"`, `"0"`, `"on"`, `"1"`. Any other value raises `ValueError`.
Input: `channel` (`int` or `str`); `state` (`int` or `str`), requested state.
Return: none.

Channel parameter readback

`get_channel_amplitude(channel)` Query the output amplitude for a channel.
Input: `channel` (`int` or `str`).
Return: `float`, amplitude in Vpp.

`get_channel_offset(channel)` Query the DC offset for a channel.
Input: `channel`.
Return: `float`, offset in V.

`get_channel_phase(channel)` Query the phase for a channel.
Input: `channel`.
Return: `float`, phase in degrees.

`get_channel_frequency(channel)` Query the output frequency for a channel.
Input: `channel`.
Return: `float`, frequency in Hz.

`get_channel_state(channel)` Query the output state for a channel. The raw response string from the instrument is returned.
Input: `channel`.
Return: `str`, output state (for example `"1"` or `"0"`).

`get_channel_function(channel)` Query the current waveform function for a channel.
Input: `channel`.
Return: `str`, function identifier such as `"SIN"`, `"SQU"`, `"RAMP"`, or `"DC"`.

Sync and global trigger configuration

`get_sync_state()` Query whether the sync output is enabled.
Input: none.
Return: `str`, typically `"1"` or `"0"` as returned by the instrument.

`set_sync_onoff(state)` Enable or disable the sync output. Accepted values are `"off"`, `"0"`, `"ON"`, `"on"`, `"1"` (case-insensitive). Other values raise `ValueError`. After writing, the method calls `get_sync_state()` to query the result.
Input: `state` (`int` or `str`).
Return: none.

Trigger configuration

`set_trigger_type(channel, state)` Set the trigger source type for a specific channel. Supported trigger types are:

- `'IMM'`: immediate,
- `'EXT'`: external,
- `'BUS'`: bus.

Any other value raises `ValueError`.
Input: `channel` (`int` or `str`); `state` (`str`), one of `'IMM'`, `'EXT'`, `'BUS'`.
Return: none.

`set_trigger_source(source)` Select which channel provides the trigger output. Internally this sets the SCPI trigger source to `"CH<source>"` and then prints the configured value.
Input: `source` (`int` or `str`), channel number used as trigger source.
Return: none.

`set_trigger_offupdown(state)` Configure the trigger gating or slope. The method accepts the following values (case-insensitive):

- `"off"` or `"0"`: turn trigger off,
- `"UP"`: trigger on positive edge,
- `"DOWN"`: trigger on negative edge.

Other values raise `ValueError`. After writing, the method prints both the trigger state and slope as reported by the instrument.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Input: state (str).
Return: none.

Channel parameter bundles

get_channel_parameters(channel) Query a bundle of parameters for a channel and return them as a dictionary. Internally this calls the individual getters. The dictionary keys are: "channel", "frequency", "Amplitude", "offset", "phase", "state", "function". All values are converted to strings.
Input: channel (int or str).
Return: dict mapping parameter names to string values.

set_channel_parameters(channel, freq, amp, offset, phase) Convenience method to set multiple output parameters at once. It programs frequency, amplitude, offset, and phase for the specified channel, then calls get_channel_parameters(channel) to query the resulting configuration.
Input: channel (int or str); freq (numeric, Hz), amp (numeric, Vpp), offset (numeric, V), phase (numeric, degrees).
Return: none.

For further implementation details, including low-level SCPI command formats and internal helper logic, please refer to the source file KeysightFG_33500.py.

3.10 ORION laser module (ORIONLaser)

3.10.1 Introduction

The ORIONLaser driver controls a compact ORION laser module over a custom serial protocol. It provides high-level access to identification (product ID, firmware version, serial number), board and thermistor temperatures, photodiode monitor voltage, laser diode current, and TEC setpoint. Both volatile (reset on power cycle) and non-volatile (stored in flash) settings are supported for the diode current and TEC temperature. The class inherits from the generic Device base class but uses a raw serial resource with a custom packet format.

3.10.2 Functions

Default operating point and status summary

default_DiodCur_TEC() Set the laser to a predefined safe default operating point using non-volatile settings. This writes a default TEC temperature (derived from a factory thermistor resistance) and a default laser diode current. Both values are written to non-volatile memory so that they persist across power cycles.
Input: none.
Return: none.

printStatus() Query the module and build a formatted multi-line status summary.

The summary includes:

- ◇ product ID, firmware version, serial number,
- ◇ status code bytes,
- ◇ photomonitor voltage,
- ◇ board temperature,
- ◇ thermistor temperature,
- ◇ current volatile and non-volatile diode current and TEC setpoints.

The summary is logged and also returned as a string.
Input: none.
Return: `str`, human-readable status summary.

Identification

`readFirmwareVersion()` Read the firmware version of the ORION module. The driver decodes a 16-bit version field into a dotted string `major.minor.patch`.
Input: none.
Return: `str`, firmware version (for example `"1.2.3"`).

`readSerialNumber()` Read the serial number of the module.
Input: none.
Return: `int`, serial number.

`readProductID()` Read the product ID string of the module.
Input: none.
Return: `str`, product identifier.

Status and monitors

`readStatus()` Read the raw status word array from the module. The response contains three status bytes which are printed for debugging and returned as a byte sequence. Interpretation of individual bits is left to the calling code.
Input: none.
Return: `bytes` or `bytearray` of length three containing the status bytes.

`readBoardTemp()` Read the internal board temperature. The driver converts an ADC code to a sensor voltage and then interpolates against a piecewise calibration table from −20 to 90 degrees Celsius. If the voltage is outside the calibrated range, the method logs a warning and returns `NaN`.
Input: none.
Return: `float`, board temperature in degrees Celsius, or `numpy.nan` if out of range.

`readThermisTemp()` Read the thermistor temperature used for laser TEC control. The driver converts an ADC code to voltage, then to thermistor resistance, and finally to temperature using the Steinhart–Hart equation with predefined coefficients.
Input: none.

Return: `float`, thermistor temperature in degrees Celsius.

`readPhotomonitorV()` Read the photomonitor voltage from the built-in photodiode monitor channel. The value is calculated from an ADC code and returned in volts. Because the monitor resistor is 1 kΩ, the same numeric value can also be interpreted as milliamps of monitor current.
Input: none.
Return: `float`, photomonitor voltage in volts.

Laser diode current

Laser diode current is always expressed in milliamps (mA) at the public API level.

`readLaserdiodeCur_mA(volatile=False)` Read the laser diode current setting from the module. The current is stored in units of 0.1 mA; the driver converts this to mA. When `volatile` is `True`, the current is read from the volatile (RAM) register that is reset on power cycle. When `volatile` is `False`, the current is read from the non-volatile (flash) register that persists across power cycles.
Input: `volatile` (`bool`), select volatile or non-volatile register (default `False`).
Return: `float`, laser diode current in mA.

`writeLaserdiodeCur_mA(mA, volatile=True)` Set the laser diode current. The driver converts the requested value in mA to an integer in units of 0.1 mA and writes it to the selected register. When `volatile` is `True`, only the volatile current is changed; when `False`, the value is stored in non-volatile memory. No explicit range checking is performed in the driver; safe limits must be enforced by the calling code.
Input: `mA` (`float` or numeric), target current in mA; `volatile` (`bool`), select volatile or non-volatile register (default `True`).
Return: low-level response object from the module (dictionary as returned by the internal `query` method).

TEC temperature setpoint

TEC setpoint is specified in degrees Celsius at the public API level and converted internally to thermistor resistance.

`readTECsetpoint(volatile=False)` Read the TEC temperature setpoint. The module stores the TEC setpoint as an equivalent thermistor resistance; the driver converts this resistance back to degrees Celsius using the same Steinhart–Hart coefficients as for `readThermisTemp()`. When `volatile` is `True`, the volatile register is read; otherwise the non-volatile register is used.
Input: `volatile` (`bool`), select volatile or non-volatile register (default `False`).
Return: `float`, TEC setpoint temperature in degrees Celsius.

`writeTECsetpoint(temp, volatile=True)` Set the TEC temperature setpoint. The driver converts the requested temperature in degrees Celsius to an equivalent thermistor resistance using the Steinhart–Hart equation, rounds to an integer number of ohms, and writes this value to the selected register. When `volatile`

is True, the change is volatile; when False, it is stored in non-volatile memory.
Input: temp (float), target temperature in degrees Celsius; volatile (bool), select volatile or non-volatile register (default True).
Return: low-level response object from the module (dictionary as returned by the internal query method).

Connection control

`connect()` Perform any required module-level initialisation and then call the base-class connect(). Internally this sends a specific command to enable communication with the ORION module before marking the device as connected.
Input: none.
Return: integer status from Device.connect() (implementation dependent; typically nonzero on success).

`disconnect()` Send a module-level disconnect command and then call the base-class disconnect(). This allows the ORION module to shut down its communication interface cleanly.
Input: none.
Return: integer status from Device.disconnect().

The driver also implements internal helper methods for packet construction, checksum validation, and Steinhart–Hart conversion between thermistor resistance and temperature. For detailed low-level behaviour, including the binary packet structure and status bit definitions, please refer to the source file ORIONLaser.py.

3.11 OZ Optics VOA (OZopticsVOA)

3.11.1 Introduction

The OZopticsVOA driver controls an OZ Optics variable optical attenuator (VOA) over a serial/VISA connection. It configures the communication parameters (baud rate, timeout, line termination) and provides a high-level property to read and set the attenuation in dB, as well as a status summary that shows the current attenuation and the internal device configuration string. The class inherits from the generic Device base class.

Internally the driver expects multi-line replies that end with the keyword Done. A safety limit _max_read_line (default 32) prevents infinite reads if the terminator is missing; this limit can be adjusted for unusually long replies.

3.11.2 Functions

Status summary

`printStatus()` Query the VOA and build a formatted multi-line status summary. The summary includes:

- current attenuation in dB (property atten_db),

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

- ◇ the device configuration string, showing model, firmware, serial number, maximum attenuation, calibration date, wavelength, and other parameters.

The summary is logged and also returned as a string.
Input: none.
Return: `str`, human-readable status summary.

Attenuation control

All attenuation values are expressed in decibels (dB).

`atten_db` Property for the VOA attenuation. The getter sends a query to the device and parses a reply of the form `"Atten:0.00(dB)"` to return a floating-point value. The setter writes a new attenuation value with 0.01 dB resolution and logs the change.
Get: `voa.atten_db` returns the current attenuation in dB.
Set: `voa.atten_db = a`, where `a` is a numeric value (int or float) specifying the desired attenuation in dB.
Type (get): `float`.
Allowed types (set): numeric.

For additional implementation details, including the low-level `read()`, `query()`, and `write()` helpers and the parsing of the configuration string, please refer to the source file `OZopticsVOA.py`.

3.12 Pendulum CNT-90 frequency counter (`PendulumCNT90`)

3.12.1 Introduction

The `PendulumCNT90` driver controls a Pendulum CNT-90 microwave frequency counter via GPIB/SCPI. It configures the communication timeout and read termination, provides a simple status summary (case temperature), and offers high-level methods to start/stop continuous measurements and to perform single-shot frequency measurements on channels A, B, or C with a configurable gate time. The class inherits from the generic `Device` base class.

3.12.2 Functions

Identification and status

`_IDN` Read-only property returning the instrument identification string (SCPI `*IDN?`).
Get: `cnt._IDN` returns a comma-separated ID string (vendor, model, serial, firmware).
Set: not supported (read-only).
Type: `str`.

`caseTemperature` Read-only property reporting the internal case temperature at the fan control sensor.

Get: `cnt.caseTemperature` returns the temperature in degrees Celsius.
Set: not supported (read-only).
Type: `float`.

`reset()` Interactively reset the frequency counter to factory settings. The method prompts on the console:
"Pendulum Freq Counter: Are you sure you want to RESET frequency counter? [Y/N]:"
and only sends the reset command if the user answers `Y` or `y`.
Input: none (requires interactive keyboard confirmation).
Return: none.

Continuous measurement control

`run()` Enable continuous frequency measurements on the counter display. Internally this turns on continuous initiation and enables the total gate function so that the front panel updates in real time. A short informational message is logged.
Input: none.
Return: none.

`stop()` Disable continuous measurements and hold the last result on the display. Internally this turns off continuous initiation and disables the total gate function. A short informational message is logged.
Input: none.
Return: none.

Single-shot frequency measurement

`measFreq(chan, measTime=1, measArray=1, log_info=False)` Measure the frequency on a specified channel and return a single result in hertz.

Channel selection:
The `chan` argument is case-insensitive and may be one of: `'a'`, `'b'`, `'c'`, `'1'`, `'2'`, `'3'`. These map to the hardware input channels:

- `'a'` or `'1'`: channel 1 (`(@1)`),
- `'b'` or `'2'`: channel 2 (`(@2)`),
- `'c'` or `'3'`: channel 3 (`(@3)`).

Any other value raises `ValueError`.

Gate time:
The `measTime` argument sets the acquisition aperture in seconds. Valid range is $[2 \times 10^{-8}, 1000]$ seconds (20 ns to 1000 s). Values outside this range raise `ValueError`. For long gate times (`measTime > 3` s) the method prints a countdown message once per second.

Inputs:

- `chan` (`str` or `int`): channel identifier (`'a'`, `'b'`, `'c'`, `'1'`, `'2'`, or `'3'`).
- `measTime` (`float`, optional): gate time in seconds; default `1`.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

- ◇ `measArray` (`int`, optional): number of measurements to return; currently unused in the implementation and kept for API compatibility (must be positive).
- ◇ `log_info` (`bool`, optional): if `True`, log the measured frequency via the driver's logger.

Return value:
On success, the method returns the measured frequency as `float` in hertz (Hz). If the instrument reports an error or the measurement does not complete, an error message is logged and the method returns `-1.0`.
Return: `float`, frequency in Hz or `-1.0` on failure.

For additional implementation details, including low-level SCPI command sequences, timeout handling, and internal logging, please refer to the source file `PendulumCNT90.py`.

3.13 Pritel optical amplifier (`PritelAmp`)

3.13.1 Introduction

The `PritelAmp` driver controls a Pritel optical fiber amplifier over a serial/VISA connection. It configures the communication parameters (baud rate, timeout, line termination) and provides high-level properties for pre-amplifier and power-amplifier pump currents, input and output optical power, automatic shut-down status, and pump activation. Optional current ramping is implemented in software to avoid large instantaneous changes to the amplifier pump currents. The class inherits from the generic `Device` base class.

3.13.2 Functions

Connection and status summary

`connect()` Open the underlying serial connection if it is not already open. The method sends a `READY?` probe, clears the interface, and marks the device as connected. On communication failure an error is logged and a negative status is returned.
Input: none.
Return: integer status (`-1` on failure, `0` if already connected, `1` or other positive value on success depending on the base class).

`printStatus()` Query the amplifier and build a formatted multi-line status summary. The summary includes:

- ◇ pump activation status (`ON` or `OFF`),
- ◇ input power and pre-amplifier current,
- ◇ output power and power-amplifier current,
- ◇ auto shut-down (ASD) status string.

The summary is logged and returned as a string.

Input: none.
Return: `str`, human-readable status summary.

Convenience power-up sequence

`easy_turnOn(pwr_mA='0.5A', pre_mA='600mA')` Convenience method to bring the amplifier to a typical operating point. It first sets the pre-amplifier current, waits one second, enables the pump activation, and then sets the power-amplifier current. The input arguments may be given as strings with units (for example `"0.5A"`, `"600mA"`). Internal safety limits on pre-amplifier and power-amplifier currents are enforced.
Input: `pwr_mA` (`str` or numeric), desired power-amplifier current; `pre_mA` (`str` or numeric), desired pre-amplifier current.
Return: none.

Pre-amplifier current

All pre-amplifier currents are expressed in milliamps (mA) at the property level. The driver enforces a maximum pre-amplifier current of 600mA.

`preAmp` Property for the pre-amplifier pump current. The getter queries the amplifier and parses a response of the form `"PreAmp Current = 000 mA"` into a numeric value in mA. The setter accepts either numeric values or strings with units; the special string `"max"` selects the internal maximum. Values above the limit raise `ValueError`. If software ramping is enabled (`self.ramp_pre_ma > 0`), the current is adjusted in steps of `ramp_pre_ma` until the target value is reached, with each step logged; if ramping is disabled (`ramp_pre_ma = 0`) the current is set in a single command.
Get: `amp.preAmp` returns the current in mA.
Set: `amp.preAmp = value`, where `value` is numeric or a unit string such as `"200mA"`, `"0.2A"`, or `"max"`.
Type (get): `float`.
Allowed types (set): numeric or `str`.

Power-amplifier current

Power-amplifier current is expressed in milliamps (mA) at the property level. The driver enforces a maximum value of 5800mA (5.8A).

`pwrAmp` Property for the power-amplifier pump current. The getter queries the amplifier and parses a response of the form `"PowerAmp Current = 0.00 A"` into a numeric value in mA. The setter accepts numeric values or strings with units; the special string `"max"` selects the internal maximum. If the amplifier pump is currently off, a warning is logged indicating that changing `pwrAmp` has no immediate effect. Values above the limit raise `ValueError`. If software ramping is enabled (`self.ramp_pwr_ma > 0`), the current is adjusted in steps of `ramp_pwr_ma` and rounded to the nearest 10 mA, with each step logged together with the current output power; if ramping is disabled, the current is set in a single command.
Get: `amp.pwrAmp` returns the current in mA.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Set: `amp.pwrAmp = value`, where `value` is numeric or a unit string such as `"3A"`, `"5000mA"`, or `"max"`.
Type (get): `float`.
Allowed types (set): numeric or `str`.

Input and output power monitors

At the public API level the power monitor properties are expressed in milliwatts (mW). Internally, responses with units `W` or `mW` are converted accordingly.

`inputPwr_mW` Read-only property for the amplifier input power. The getter sends the query `"FA INPUT?"`, expects a response of the form `"Input Power = 0 mW"`, and converts the value to mW.
Get: `amp.inputPwr_mW` returns the input power in mW.
Set: not supported (read-only).
Type: `float`.

`outputPwr_mW` Read-only property for the amplifier output power. The getter sends the query `"FA OUTPUT?"`, expects a response of the form `"Output Power = 0.00 W"`, and converts the value to mW.
Get: `amp.outputPwr_mW` returns the output power in mW.
Set: not supported (read-only).
Type: `float`.

Auto shut-down status

`ASD` Read-only property for the auto shut-down (ASD) status. The getter sends the query `"FA ASD?"` and returns the full response string, which typically indicates whether auto shut-down is enabled and whether the power-amplifier pump current is disabled.
Get: `amp.ASD` returns the ASD status string.
Set: not supported (read-only).
Type: `str`.

Pump activation

`activation` Property for the pump activation state of the amplifier. The getter sends `"FA PUMP?"` and returns the last word of the response (typically `"ON"` or `"OFF"`). The setter accepts `0`, `1`, `'off'`, or `'on'` (case-insensitive) and converts the request to the corresponding command `"FA OFF"` or `"FA ON"`. When turning the pump on, the driver prints a safety message reminding the user to verify the seed input power. The setter then loops until the device reports the requested activation state or a timeout is reached; progress is logged during this wait. If the state does not change within the configured activation timeout, a `RuntimeError` is raised.
Get: `amp.activation` returns `"ON"` or `"OFF"`.
Set: `amp.activation = status`, where `status` is one of `0`, `1`, `'off'`, or `'on'` (case-insensitive).

Type (get): `str`.
Allowed types (set): `int` or `str`.

For further implementation details, including low-level query/write helpers, parsing of unit strings, and private ramping methods, please refer to the source file `PritelAmp.py`.

3.14 Stanford Research System SIM900 mainframe (SRS_SIM900)

3.14.1 Introduction

The `SRS_SIM900` driver controls a Stanford Research System SIM900 mainframe that hosts up to eight plug-in modules (such as SIM960 PID controllers and SIM928 isolated voltage sources). It configures the serial communication parameters (baud rate, timeout, line termination) and provides convenience methods to clear the interface, switch the active slot, and route read/write/query operations to a specific module. The class inherits from the generic `Device` base class.

3.14.2 Functions

Status and housekeeping

`printStatus()` Print and return a simple SIM900 status header. This is mainly a placeholder for consistency with other device drivers and does not query detailed module information.
Input: none.
Return: `str`, formatted status header.

`clear()` Clear the SIM900 interface and reset mainframe routing. This calls the underlying VISA `clear()` method, resets the internal `active_module` to `None`, and leaves the mainframe selected (no module connection).
Input: none.
Return: none.

Slot routing helpers

All slot numbers are integers from `0` to `8`. Slot `0` refers to the mainframe itself (no module selected). Slots `1`–`8` select the corresponding SIM module position.

`switch_to(slot, clear=True)` Route subsequent communication to a specific slot. If `clear` is `True`, the method first calls `clear()` to reset the mainframe interface, then:

- if `slot == 0`, keep the mainframe active and do not connect to any module;
- otherwise send a `CONN <slot>, "<esc>"` command and update the internal `active_module`.

If `clear` is `False`, the method only sends a new `CONN` command when the requested `slot` differs from the current `active_module`.
Input: `slot` (`int`), slot index (`0`–`8`); `clear` (`bool`, optional), whether to clear

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

the mainframe first (default `True`).
Return: none.

`query_slot(slot, query, clear=True)` Send a SCPI query to a specific module slot and return the response. Internally this calls `switch_to(slot, clear=clear)` and then `inst.query(query)`. If a `VisaIOError` is raised, it waits one second and retries once.
Input: `slot` (`int`), module slot; `query` (`str`), SCPI command to send; `clear` (`bool`, optional), whether to clear the mainframe before switching (default `True`).
Return: `str`, raw reply from the module.

`write_slot(slot, text, clear=True)` Send a SCPI command (no response expected) to a specific module slot. Internally this calls `switch_to(slot, clear=clear)` and then `inst.write(text)`. If a `VisaIOError` is raised, it waits one second and retries once.
Input: `slot` (`int`), module slot; `text` (`str`), SCPI command to send; `clear` (`bool`, optional), whether to clear the mainframe before switching (default `True`).
Return: none.

`read_slot(slot, clear=True)` Read from a specific module slot without issuing a new command. Internally this calls `switch_to(slot, clear=clear)` and then `inst.read()`. If a `VisaIOError` is raised, it waits one second and retries once.
Input: `slot` (`int`), module slot; `clear` (`bool`, optional), whether to clear the mainframe before switching (default `True`).
Return: `str`, raw data read from the module.

For additional implementation details, including the escape string format and low-level error handling, please refer to the source file `SRS_SIM900.py`.

3.15 SRS PID controller SIM960 (`SRS_PIDcontrol_SIM960`)

3.15.1 Introduction

The `SRS_PIDcontrol_SIM960` driver controls a SIM960 PID controller installed in a SIM900 mainframe. It uses the mainframe routing helpers to send commands to a specified slot and exposes high-level properties for proportional, integral, and derivative gains, output offset, output mode (manual or PID), setpoint and setpoint ramping, manual output, output limits, and several monitor channels (setpoint input, measured input, amplified error, and output voltage). A manual sweep helper is also provided for characterising loop response.

3.15.2 Functions

Status and identification

`printStatus()` Query the SIM960 and build a formatted status summary. The summary includes:

- ◇ identification string (model and firmware),
- ◇ output mode and output limits,
- ◇ measured input, amplified error, and output voltage,
- ◇ manual output setting,
- ◇ internal setpoint, ramp status and rate, setpoint input mode,
- ◇ P, I, D gains and their on/off status,
- ◇ output offset and its on/off status.

The summary is logged via the mainframe logger and returned as a string.
Input: none.
Return: `str`, human-readable status summary.

`equipment_IDN` Read-only property returning the instrument identification string (`*IDN?`).
Get: `servo.equipment_IDN` returns a comma-separated ID string.
Set: not supported (read-only).
Type: `str`.

`serialNumber` Read-only property returning the serial number extracted from `equipment_IDN`.
Get: `servo.serialNumber` returns the serial number as a string.
Set: not supported (read-only).
Type: `str`.

P, I, D and offset configuration

All voltages are in volts (V). Integral gain is in inverse seconds (1/s); derivative gain is in seconds.

`prop_action`, `intg_action`, `derv_action`, `outoffset_action` Properties controlling whether the proportional, integral, derivative, and output-offset terms are active. Each property returns a human-readable status and can be set by specifying `'on'`, `'off'`, `1`, or `0` (case-insensitive).
Get: returns `'ON'` or `'OFF'`.
Set: accepts `int` or `str` representing on/off.
Type (get): `str`.

`prop_gain` Proportional gain in V/V. The driver enforces a symmetric range from -10^3 to $+10^3$. Values outside this range raise `ValueError`.
Get: `servo.prop_gain` returns the gain as `float`.
Set: `servo.prop_gain = g`, where `g` is numeric or a unit string such as `"1.0V/V"`. Precision is limited by the instrument resolution; excessive decimal places are rounded with a warning.
Type (get): `float`.

`intg_gain` Integral gain in 1/s. The allowed range is from 10^{-2} to 5×10^5 s^{-1}. Values outside this range raise `ValueError`.
Get: `servo.intg_gain` returns the gain as `float`.
Set: `servo.intg_gain = g`, where `g` is numeric or a compatible unit string such

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

as "100/s".
Type (get): float.

derv_gain Derivative gain in seconds. The allowed range is from 10^{-6} s to 10 s. Values outside this range raise ValueError.
Get: servo.derv_gain returns the gain as float.
Set: servo.derv_gain = g, where g is numeric or a compatible unit string such as "1ms".
Type (get): float.

outoffset Output offset in volts. The allowed range is −10 V to +10 V. Values outside this range raise ValueError.
Get: servo.outoffset returns the offset as float.
Set: servo.outoffset = v, where v is numeric or a unit string such as "10mV".
Type (get): float.

Output mode and setpoint source

output_mode Property selecting PID or manual output mode.
Get: servo.output_mode returns 'MAN' (manual) or 'PID' (closed-loop).
Set: servo.output_mode = status, where status is one of 0, 1, 'man', or 'pid' (case-insensitive). Invalid values raise ValueError.
Type (get): str.

setpoint_input_mode Property selecting internal or external setpoint.
Get: servo.setpoint_input_mode returns 'INT' or 'EXT'.
Set: servo.setpoint_input_mode = status, where status is one of 0, 1, 'int', or 'ext' (case-insensitive). Invalid values raise ValueError.
Type (get): str.

Setpoint and ramping

setpoint Internal setpoint voltage. The allowed range is −10 V to +10 V; values outside this range raise ValueError.
Get: servo.setpoint returns the setpoint in V.
Set: servo.setpoint = v, where v is numeric or a unit string such as "100mV".
Type (get): float.

spramp_action Property enabling or disabling internal setpoint ramping.
Get: returns 'ON' or 'OFF'.
Set: accepts 0, 1, 'off', or 'on' (case-insensitive).
Type (get): str.

spramprate Internal setpoint ramp rate in V/s. Allowed range is from 10^{-3} to 10^{4} V/s; values outside this range raise ValueError.
Get: servo.spramprate returns the rate in V/s.
Set: servo.spramprate = r, where r is numeric or a unit string such as "10mV/s".
Type (get): float.

spramp_status Read-only property reporting whether the internal ramp is running or paused.
Get: returns 'ON' or 'OFF'.
Set: not supported (read-only).
Type: str.

spramp_START(), spramp_STOP() Methods to start or stop internal setpoint ramping. They send the corresponding SIM960 commands and log a message.
Input: none.
Return: none.

Manual output and limits

manual_output Manual output voltage in manual mode. The allowed range is determined by internal limits; initial defaults are −10 V to +10 V but can be modified via set_manual_output_min/max(). Values outside the effective range raise ValueError. If manual_output_ramp is zero, the output is set in a single step; otherwise the driver ramps from the current value to the target in uniform steps of magnitude manual_output_ramp, rounded to 1 mV, logging each step.
Get: servo.manual_output returns the current manual output voltage.
Set: servo.manual_output = v, where v is numeric or a unit string such as "100mV".
Type (get): float.

set_manual_output_max(num), set_manual_output_min(num) Set software limits for manual output. These limits are checked when writing manual_output.
Input: num (float), new maximum or minimum allowed output in V.
Return: none.

manual_output_ramp Public attribute controlling the ramp step size (in volts) used by the manual_output setter. A value of 0.0 disables ramping; positive values enable stepwise ramping.
Type: float.

output_upperlim, output_lowerlim Output saturation limits in volts. The driver enforces an internal resolution of 10 mV.
Get: returns the limit as float in V.
Set: servo.output_upperlim = v or servo.output_lowerlim = v, where v is numeric or a unit string. Values are checked against the instrument's allowed range; invalid values raise ValueError.
Type (get): float.

Monitor channels

setpoint_input Read-only property for the setpoint input monitor (SMON).
Get: returns the voltage in V as float.
Set: not supported (read-only).

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

`measure_input` Read-only property for the measured input monitor (MMON).
Get: returns the voltage in V.

`amplified_error` Read-only property for the amplified error signal (EMON).
Get: returns the voltage in V.

Manual sweep helper

`manual_sweep(voltage_list, start_pause=1, interval_pause=0.1, plot=True)`
Perform a manual output sweep and record the measured input response. The method:

(1) clears the mainframe connection,

(2) forces `output_mode = 'MAN'`,

(3) for each value in `voltage_list`, sets the manual output and then queries the measured input monitor,

(4) optionally plots measured input versus manual output.

Errors in individual steps are caught and skipped; the method continues with remaining points.

Input:

- `voltage_list` (array-like), list of target output voltages;
- `start_pause` (`float`, optional), initial wait in seconds before the first measurement;
- `interval_pause` (`float`, optional), wait time in seconds between subsequent points;
- `plot` (`bool`, optional), whether to show a matplotlib plot (default `True`).

Return: `numpy.ndarray` of measured input voltages, same shape as `voltage_list`.

For further implementation details, including low-level SCPI command formats, unit decoding, and numeric range checks, please refer to the source file `SRS_SIM900.py`.

3.16 SRS isolated voltage source SIM928 (`SRS_VoltSorc_SIM928`)

3.16.1 Introduction

The `SRS_VoltSorc_SIM928` helper class provides a thin wrapper around a SIM928 isolated voltage source installed in a SIM900 mainframe. It exposes simple `write()`, `query()`, and `read()` methods that automatically route communication through the mainframe to the correct slot. High-level SCPI commands and response parsing are left to the user or to higher-level abstractions.

3.16.2 Functions

`write(cmd)` Send a SCPI command string to the SIM928 module in the configured slot. Internally this calls `mainframe.write_slot(slot, cmd)`.
Input: `cmd` (`str`), SCPI command to send.
Return: none.

`query(cmd)` Send a SCPI query string to the SIM928 module and return the response. Internally this calls `mainframe.query_slot(slot, cmd)`.
Input: `cmd` (`str`), SCPI query to send.
Return: `str`, raw reply from the module.

`read(cmd)` Read from the SIM928 module without issuing a new command. The `cmd` argument is unused and kept only for consistency with other interfaces; the method delegates to `mainframe.read_slot(slot)`.
Input: `cmd` (`str`), ignored.
Return: `str`, raw data read from the module.

For detailed behaviour of the SIM928 voltage source itself (voltage ranges, output modes, and protection features), as well as the exact SCPI command set, please refer to the SIM928 hardware manual and to the implementation in `SRS_SIM900.py`.

3.17 Tektronix TDS2024C oscilloscope (`TDS2024C`)

3.17.1 Introduction

The `TDS2024C` driver controls a Tektronix TDS2024C digital oscilloscope via VISA/USB. It configures timeouts and line terminations, provides high-level methods to acquire a waveform trace from a selected channel, save traces to disk in several formats, and perform a Lorentzian fit to extract an optical autocorrelation full width at half maximum (FWHM) from a single-shot trace. The class inherits from the generic `Device` base class and uses the base-class `write()` and `query()` methods for SCPI communication.

3.17.2 Functions

Waveform acquisition and analysis

`get_trace(trace=2, plot=True)` Acquire a waveform from the specified oscilloscope channel and decode it into physical X and Y arrays using the Tektronix waveform preamble. The method:

- selects the data source channel,
- configures ASCII encoding and point format,
- sets the start and stop indices (2500 points),
- queries the waveform preamble (`WFMP?`) and the data (`CURV?`),
- converts raw ADC values into voltages and time stamps.

The resulting waveform dictionary is stored in `self.wvfm` and can be reused by

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

other methods. Optionally, a quick plot of voltage versus time is shown using `matplotlib`.
Input: `trace` (`int`), channel index (`1-4`); `plot` (`bool`), whether to plot the trace (default `True`).
Return: `dict` `wvfm` containing waveform data and metadata, including:

- `'X'`: time axis, `numpy.ndarray` of length `NR_Pt`,
- `'Y'`: voltage axis, `numpy.ndarray`,
- preamble fields such as `'XINcr'`, `'XZEro'`, `'YMUlt'`, `'YZEro'`, `'YOFF'`, `'XUNit'`, `'YUNit'`, etc.

`save_trace(filename, trace=2, extensions=['.mat', '.txt'], plot=True, additional_dat`
Acquire a waveform from the specified channel and save it to disk in one or more formats. The method:

- validates the requested file extensions against the supported list `['.mat', '.txt', '.fits', '.csv']`,
- creates the target directory if it does not exist,
- renames any existing files with a timestamped backup suffix,
- calls `get_trace()` to read the waveform,
- attaches `additional_data` to the waveform dictionary under the key `'additional_data'`,
- saves the data in the requested formats:
  - `.mat`: MATLAB file with the full waveform dictionary,
  - `.txt`: two-column text file with X and Y,
  - `.fits`: FITS table of X and Y,
  - `.csv`: CSV file with X and Y columns.

Errors during saving are caught and logged; the method continues with remaining formats.
Input: `filename` (`str`), base path without extension; `trace` (`int`), channel index (`1-4`); `extensions` (list or str), one or more file extensions to save; `plot` (`bool`), whether to plot the trace when calling `get_trace()`; `additional_data`, arbitrary object to be stored under the `'additional_data'` key in the `.mat` file.
Return: none.

`get_autocorr_FWHM_single(data_x=None, data_y=None, filename="autocorr\\test", trace=2, fit_range=50, additional_plotdata=None)` Extract the optical comb FWHM from a single autocorrelation trace by fitting a Lorentzian model and converting the fitted width to an optical time scale. If `data_x` or `data_y` is `None`, the method first calls `save_trace()` to read a fresh trace from the specified channel and uses the stored `self.wvfm['X']` and `self.wvfm['Y']`. The algorithm:

- locates the peak of `data_y` and defines a symmetric fit window of length

`fit_range` around it,

- ◇ estimates an initial Lorentzian width from the half-maximum points on either side,
- ◇ fits a Lorentzian function to the data in the fit window using `scipy.optimize.curve_fit`,
- ◇ applies a fixed calibration factor between electrical and optical time to obtain the comb FWHM,
- ◇ optionally overlays `additional_plotdata` on a third panel in the saved figure.

Two subplots are always generated: a full-range fit and a zoomed view around the peak, both in optical time units (ps). The figure is shown and saved to `filename + "_{FWHM}ps.png"`.
Input: `data_x` and `data_y` (`numpy.ndarray` or `None`), optional pre-acquired time and voltage arrays; `filename` (`str`), base path for saving the figure (extension is forced to `.png`); `trace` (`int`), channel index if acquisition is needed; `fit_range` (`int`), number of points used in the Lorentzian fit window; `additional_plotdata`, optional tuple `(x, y)` for an extra panel.
Return: four values:

- ◇ comb FWHM in picoseconds (`float`),
- ◇ fitted Lorentzian evaluated on `data_x` (`numpy.ndarray`),
- ◇ `matplotlib.figure.Figure` object for the generated plot,
- ◇ raw FWHM estimate in picoseconds (`float`) computed directly from the half-maximum crossing points.

Vertical and horizontal scaling

All positions are in division units; scales are in seconds per division (time) or volts per division (voltage).

`set_vol_pos(channle, pos)` Set the vertical position of a given channel. Internally this sends `CH<n>:POS <pos>` and prints the resulting position queried back from the scope.
Input: `channle` (`int` or `str`), channel index; `pos` (numeric), vertical position in divisions.
Return: none (the new position is printed to standard output).

`set_vol_scale(channle, sca)` Set the vertical scale of a given channel. Internally this sends `CH<n>:SCA <sca>` and prints the resulting scale queried back from the scope.
Input: `channle` (`int` or `str`), channel index; `sca` (numeric), vertical scale in volts per division.
Return: none (the new scale is printed to standard output).

`set_time_pos(pos)` Set the horizontal (time) position of the trigger point on the display, in divisions. Internally this sends `HOR:POS <pos>` and prints the resulting position.
Input: `pos` (numeric), time position in divisions.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Return: none.

set_time_scale(sca) Set the horizontal time scale for the main timebase. Internally this sends HOR:SCA <sca> and prints the resulting scale queried back.
Input: sca (numeric), time scale in seconds per division.
Return: none.

Display and trigger control

trace_disp(chan, state) Enable or disable the on-screen display of a channel trace. If state is 1 or 'ON', the method sends SEL:CH<n> ON; if state is 0 or 'OFF', it sends SEL:CH<n> OFF. After the command, the method prints the current selection string returned by SEL?.
Input: chan (int or str), channel index; state (int or str), requested display state (0/1 or 'OFF'/'ON').
Return: none.

set_trigger_source(chan) Select the trigger source channel for the main edge trigger. Internally this sends TRIG:MAI:EDGE:SOU <chan> and prints the resulting source string queried back from the scope.
Input: chan (str), channel identifier as expected by the scope (for example "CH1", "CH2").
Return: none.

set_trigger_lev(lvl) Set the main trigger level in volts. Internally this sends TRIG:MAI:LEV <lvl> and prints the resulting level queried back from the scope.
Input: lvl (numeric), trigger threshold in volts.
Return: none.

For additional implementation details, including the interpretation of the waveform preamble fields and low-level communication helpers, please refer to the source file TDS2024C.py.

3.18 TE Technology TC-720 temperature controller (TEC_TC720)

3.18.1 Introduction

The TEC_TC720 driver controls a TE Technology TC-720 temperature controller over a direct serial connection (no VISA layer). It configures the serial port (baud rate, data bits, parity, stop bits, timeout) and implements the TC-720 binary protocol (messages with command, value, and checksum fields). The class provides high-level methods to read temperatures, configure operating modes, program ramp and soak sequences, start and stop those sequences, wait for thermal stability, and check for error conditions. It inherits from the generic Device base class.

3.18.2 Functions

Connection management

`connect()` Open the serial port and establish communication with the controller if not already connected. The serial parameters are: 230,400 baud, 8 data bits, no parity, one stop bit, 2 s timeout. On success the `connected` flag is set and an informational message is logged.
Input: none.
Return: integer status: `1` on success; `-1` on failure; `0` if the device was already connected.

`disconnect()` Close the serial port if it is open and clear the `connected` flag. An informational message is logged.
Input: none.
Return: integer status: `1` if a connection was closed; `0` if it was already disconnected.

Basic readback functions

All temperatures are in degrees Celsius; times are in seconds; output values are dimensionless integers in the range −511 to +511 corresponding to −100 % to +100 % output power.

`get_temp(log_info=False)` Read the current temperature from sensor 1. Optionally log the value via the driver logger.
Input: `log_info` (`bool`, optional), if `True` log the temperature.
Return: `float`, sensor 1 temperature in °C.

`get_temp2()` Read the current temperature from sensor 2.
Input: none.
Return: `float`, sensor 2 temperature in °C.

`get_mode()` Query the current operating mode of the controller.
Input: none.
Return: `int`, one of:

- `0`: Normal Set mode (single temperature/output).
- `1`: Ramp/Soak mode (programmed sequence).
- `2`: Proportional + Dead Band mode.

`get_control_type()` Query the control type used in Normal Set mode (mode 0).
Input: none.
Return: `int`, one of:

- `0`: PID, maintain a single temperature.
- `1`: Manual, maintain a fixed output power.
- `2`: Analog Out, driven by an external voltage.

`get_set_temp()` Read the current setpoint temperature configured for Normal Set

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

mode.
Input: none.
Return: `float`, setpoint temperature in °C.

`get_output()` Read the instantaneous output level (controller actual output).
Input: none.
Return: `int`, output level in the range [−511, 511].

`get_set_output()` Read the configured manual output setpoint for Normal Set mode when using manual control.
Input: none.
Return: `int`, output setpoint in the range [−511, 511].

Ramp/soak status and location readback

Ramp/soak programming uses eight locations (steps) numbered `1`–`8`. Each location has a soak temperature, ramp time, soak time, repeat count, and a "repeat location" (next step).

`get_ramp_soak_status()` Check whether a ramp/soak sequence is currently running and which phases are active. Internally it decodes status bits from the controller.
Input: none.
Return: either the string `"No sequence running"` or a list of one or more of:

- `"Sequence Running"`,
- `"Soak stage"`,
- `"Ramp stage"`.

`get_soak_temp(location)` Read the programmed soak (hold) temperature for a given location in Ramp/Soak mode.
Input: `location` (`int`), step index `1-8`. Invalid values raise `ValueError`.
Return: `float`, soak temperature in °C.

`get_ramp_time(location)` Read the ramp time for a given location. This is the time the controller spends ramping from the previous soak temperature to the new soak temperature.
Input: `location` (`int`), `1-8`.
Return: `int`, ramp time in seconds.

`get_soak_time(location)` Read the soak (hold) time for a given location.
Input: `location` (`int`), `1-8`.
Return: `int`, soak time in seconds.

`get_repeats(location)` Read how many times a given location is repeated before moving on.
Input: `location` (`int`), `1-8`.
Return: `int`, repeat count.

`get_repeat_location(location)` Read the next location executed after a given location completes its configured repeats.
Input: `location` (`int`), `1-8`.
Return: `int`, next location index.

Mode and control configuration

`set_mode(mode)` Set the overall operating mode of the controller.
Input: `mode` (`int`), must be one of:

- `0`: Normal Set,
- `1`: Ramp/Soak,
- `2`: Proportional + Dead Band.

Invalid values raise `ValueError`.
Return: none.

`set_control_type(control_type)` Select the control type used in Normal Set mode. This is only valid when `mode == 0`. The method checks the current mode using `check_mode(0)` and warns if the controller is not in the correct mode.
Input: `control_type` (`int`), one of:

- `0`: PID temperature control,
- `1`: Manual output control,
- `2`: Analog Out.

Invalid values raise `ValueError`.
Return: none.

Normal Set mode: single temperature or output

These functions are used when the controller operates in Normal Set mode (`mode = 0`).

`set_temp(temperature)` Set a single temperature and hold it using PID control. The controller must be in Normal Set mode with control type `0` (PID).
Input: `temperature` (`float` or `int`), target temperature in °C. Internally this is converted to a fixed-point representation with two decimal places.
Return: none.

`set_output(output)` Set a fixed output level in manual mode. The controller must be in Normal Set mode with control type `1` (Manual).
Input: `output` (`int`), integer in $[-511, 511]$ corresponding to $-100\,\%$ to $+100\,\%$ output power.
Return: none.

Ramp/soak programming

All ramp/soak setters require mode `1` (Ramp/Soak). Location indices must be integers in the range `1-8`; invalid values raise `ValueError`.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

set_soak_temp(location, temperature) Set the soak (hold) temperature for a specific location. Negative temperatures are supported.
Input:
- location (int), 1-8;
- temperature (float), target temperature in °C (two decimal places supported).

Return: none.

set_ramp_time(location, time) Set the ramp time for a specific location.
Input:
- location (int), 1-8;
- time (int), ramp time in seconds.

Return: none.

set_soak_time(location, time) Set the soak time at the soak temperature for a specific location.
Input:
- location (int), 1-8;
- time (int), soak time in seconds, in the range [1, 32768]. Invalid values raise ValueError.

Return: none.

set_repeats(location, repeats) Set how many times a given location is repeated before advancing to its next location.
Input:
- location (int), 1-8;
- repeats (int), repeat count.

Return: none.

set_repeat_location(location, repeat_loc) Specify which location should be executed next after the current one finishes its repeats.
Input:
- location (int), 1-8;
- repeat_loc (int), next location 1-8.

Return: none.

Start/stop of ramp/soak sequences

start_soak() Start executing the ramp/soak program defined in the eight locations. The controller must be in Ramp/Soak mode (mode = 1).
Input: none.
Return: none.

idle_soak() Stop the ramp/soak execution and leave the controller in an idle state

(still in mode `1`).
Input: none.
Return: none.

`set_idle()` Convenience helper to put the controller into a safe idle condition: the method switches to Normal Set mode (`mode = 0`), sets the manual output to zero, and selects manual control type.
Input: none.
Return: none.

Combined sequence helpers

`get_sequence(location='all')` Retrieve the full programmed ramp/soak table or a subset of locations. Internally it calls the per-location getters for soak temperature, ramp time, soak time, repeats, and repeat location.
Input: `location`, one of:

- `'all'`: read locations `1-8`;
- `int`: a single location index `1-8`;
- list of `int`: specific locations to read.

Return: `numpy.ndarray` of shape $(N + 1) \times 6$, where the first row contains column headers `['Loc', 'Temp', 'Ramp time', 'Soak time', 'Repeats', 'Repeat loc']` and each subsequent row contains data for one location.

`set_single_sequence(location, temp=20, ramp_time=60, soak_time=30000, repeats=1, go_to=None)` Configure all parameters for a single ramp/soak location. This is a convenience wrapper around `set_soak_temp()`, `set_ramp_time()`, `set_soak_time()`, `set_repeats()`, and `set_repeat_location()`. If `go_to` is `None`, the next location is chosen circularly (for example, from 8 back to 1).
Input:

- `location` (`int`), `1-8`;
- `temp` (`float`), target temperature in °C;
- `ramp_time` (`int`), ramp time in seconds;
- `soak_time` (`int`), soak time in seconds (default 30,000 s, recommended upper limit);
- `repeats` (`int`), repeat count;
- `go_to` (`int` or `None`), optional next location index; if `None`, the method uses the next location in a wrap-around list. Invalid values raise `ValueError`.

Return: none.

Waiting for temperature stability

`waitTemp(target_temp, error=1, array_size=5, sd=0.01, timeout=5, set_idle=True)` Block execution until the measured temperature stabilises near a target value, or until a timeout is reached. The method samples sensor 1 once per second, stores the last `array_size` readings in a buffer, and checks two conditions: (i) the

current temperature lies within ± `error` of `target_temp`; and (ii) the standard deviation of the buffer is below `sd`, indicating a plateau. If these conditions are met the method returns. If `timeout` minutes elapse, `check_error()` is called and, if `set_idle` is `True`, the controller is put into idle state.

Input:

- `target_temp` (`float`), desired temperature in °C;
- `error` (`float`), allowed temperature band around `target_temp`;
- `array_size` (`int`), buffer length for stability check;
- `sd` (`float`), maximum standard deviation in °C for stable plateau;
- `timeout` (`float`), timeout in minutes;
- `set_idle` (`bool`), whether to force the controller into idle state if timeout is reached.

Return: none on success; if stability is not reached within `timeout` and `set_idle` is `True`, an `Exception` is raised. If `set_idle` is `False`, the method only logs a warning and returns.

Error checking

`check_error(set_idle=True, raise_exception=True)` Query the controller error status bits and interpret them. If errors are present, the method can force the controller into idle state and optionally raise an exception.

Input:

- `set_idle` (`bool`), if `True` call `set_idle()` when an error is detected;
- `raise_exception` (`bool`), if `True` raise an `Exception` on error, otherwise only log a warning and return.

Return:

- if no errors: `[True, "No errors on temperature controller: ..."]`;
- if errors and `raise_exception=False`: `[False, "Error(s) on ..."]`;
- if errors and `raise_exception=True`: an `Exception` is raised.

Deprecated helper

`find_address(identifier=None)` (deprecated) Standalone helper function (not a class method) to locate a serial device by unplug/plug detection or by matching a string identifier against available ports. This function is marked as deprecated in the source and should not be used in new code.

Input: `identifier` (`str` or `None`), optional search key for `pyserial` port enumeration.

Return: a `pyserial` port object; the address is available as `port.device`.

For further protocol details (message format, checksums, and internal helpers used to build and parse messages), refer directly to the source file `TEC_TC720.py`.

3.19 Measurement Computing USB-2408 DAQ (USB2408)

3.19.1 Introduction

The USB2408 driver controls a Measurement Computing USB-2408 data acquisition (DAQ) device and is used here as an eight-channel thermocouple temperature monitor for the astrocomb system. Each board is identified by a board number (InstaCal "Board" index), and the driver provides a fixed mapping from channel indices to physical thermocouple positions for two pre-commissioned boards:

- Board 0: DAQ in the electronics rack (rack air baffle, waveshaper, Rb clock, Pritel, rack glycol lines, etc.).
- Board 1: DAQ on the optical table breadboard (RF oscillator, RF amplifier, phase modulators, filter cavity, glycol lines, Rb cell, etc.).

If a different board number is used, a generic device name and a placeholder list of thermocouple positions are assigned, and a warning is issued. The class inherits from the generic Device base class but uses the Measurement Computing mcculw library (ul) for all hardware access.

3.19.2 Functions

Status summary

printStatus() Build a formatted multi-line status summary of all eight channels, including channel index, measured temperature, and the corresponding entry from thermocouple_positions. The summary string is assembled in a local variable message; users may adapt the method to print or log this string as needed.
Input: none.
Return: None.

Connection and configuration

connect(set_TC=True) Connect to the USB-2408 board identified by self.addr (InstaCal board number). The method:

- queries the board name with ul.get_board_name(),
- checks that it contains the string "USB-2408",
- flashes the on-board LED for visual confirmation,
- optionally configures each channel as a J-type thermocouple in Celsius at a 60 Hz data rate by calling set_chan_TC() for channels 0–7,
- sets self.connected = True on success.

If no board is found at the given index, or if the board is not a USB-2408, an error message is printed and the method returns a negative status code.
Input: set_TC (bool, optional), if True (default) configure all channels for type-J thermocouple input on connection.
Return: int status code: 1 on successful connection, 0 if already connected, -1

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

on failure.

set_chan_TC(channel) Configure one channel as a thermocouple input. Internally the method:

- sets the channel type to thermocouple (AiChanType.TC),
- selects thermocouple type J (TcType.J),
- sets the temperature scale to Celsius (TempScale.CELSIUS),
- sets the data rate to 60 Hz.

A completion message is printed indicating which channel was configured.
Input: channel (int), channel index (0-7).
Return: None.

Temperature readback

All temperatures are returned in degrees Celsius.

get_temp(chan) Read the temperature from a single thermocouple channel. The method:

- calls ul.t_in() with Celsius scale and TInOptions.NOFILTER,
- prints a formatted line containing the channel index, the temperature (three decimal places), and the corresponding thermocouple position label from thermocouple_positions,
- returns the numeric temperature.

If an exception occurs during acquisition, the exception is printed and the method returns numpy.nan.
Input: chan (int), channel index (0-7).
Return: float, temperature in Celsius on success; numpy.nan on error.

get_temp_all() Read temperatures from all configured channels. Internally this calls get_temp(chan=ii) for each channel index ii = 0, 1, ..., 7, *collectingtheresultsintoaPyt*
Input:*none*.
Return:list*ofeight*float*values* (*temperaturesinCelsius*) , *orderedbychannelindex*.

For additional implementation details, including thermocouple position labels, error handling, and logging examples, please refer directly to the source file USB2408.py.

3.20 Finisar Waveshaper (Waveshaper)

3.20.1 Introduction

The Waveshaper driver controls a Finisar (II-VI) programmable optical waveshaper via the vendor wsapi library. It creates and opens a named Waveshaper instance, tracks the available frequency range (start and stop in THz), and maintains callable amplitude and phase profiles over that range. High-level methods are provided to

program second- and third-order dispersion, interpolate arbitrary phase masks, apply band-pass and inverse-attenuation filters based on optical spectrum analyser (OSA) data, write the composite profile to the hardware, and plot the current and previewed settings. The class inherits from the generic `Device` base class but does not use VISA.

3.20.2 Functions

Connection management and status

`connect()` Create a connection to the Waveshaper if one does not already exist. The method:

- opens the Waveshaper instance via `ws_open_waveshaper()`,
- queries and stores the start and stop frequencies (`startF`, `stopF`) in THz,
- builds the internal frequency grid `freq` with 1 GHz spacing and sets `numPixels`.

On success it calls the base-class `connect()`.
Input: none.
Return: integer status (error code from `ws_open_waveshaper()` on failure; otherwise the status returned by `Device.connect()`, or `0` if already connected).

`disconnect()` Close the connection to the Waveshaper if it is open. The method:

- calls `ws_close_waveshaper()` and `ws_delete_waveshaper()` for the current instance,
- then calls the base-class `disconnect()`.

If the Waveshaper instance is not valid or not connected, the method returns without action.
Input: none.
Return: integer status (error code from `ws_close_waveshaper()` on failure; otherwise the status returned by `Device.disconnect()`, or `0` if already disconnected).

`printStatus()` Build a short status summary and plot the current and previewed attenuation and phase profiles. The method:

- assembles a message containing `info_atten` and `info_phase`,
- calls `plotStatus()` to show four subplots of current and preview attenuation and phase versus frequency or wavelength,
- logs the status message via the base-class logger.

Input: none.
Return: none.

Dispersion programming

All dispersion settings are specified in terms of fiber-like parameters d_2 and d_3, expressed in ps/nm and ps/nm^2, respectively. Internally these are converted to propagation coefficients β_2 and β_3 and applied as spectral phase masks versus optical frequency.

`set2ndDisper(d2, center=1560, centerunit='nm', preview_plot=False)` Pro-

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

gram a pure second-order dispersion profile. The method:

- converts `center` to THz if `centerunit` is `'nm'`,
- computes β_2 from the specified d_2,
- sets `self.phase` to a function of frequency implementing a quadratic phase $\phi(\nu) = \beta_2 (2\pi(\nu - \nu_0))^2/2$,
- updates `info_phase` with a human-readable summary,
- optionally plots the phase versus frequency if `preview_plot` is `True`.

Input: `d2` (`float`), second-order dispersion in ps/nm; `center` (`float`), center wavelength or frequency; `centerunit` (`str`), `'nm'` or `'thz'` (case-insensitive); `preview_plot` (`bool`), whether to plot the phase profile.
Return: `float` β_2, the computed dispersion coefficient in SI units.

`give_2ndDisp_phase(x, d2, center=1560, centerunit='nm', preview_plot=False)`

Compute the second-order dispersion phase at a single frequency value without modifying the internal profile. The formula is identical to that used in `set2ndDisper()`, but the result is returned instead of stored. The `preview_plot` argument is currently unused.
Input: `x` (`float`), frequency in THz; `d2` (`float`), second-order dispersion in ps/nm; `center` (`float`), center wavelength or frequency; `centerunit` (`str`), `'nm'` or `'thz'`.
Return: `float`, spectral phase at frequency `x` in radians.

`set3rdDisper(d2, d3, center=1560, centerunit='nm', preview_plot=False)`

Program a combined second- and third-order dispersion profile. The method:

- converts `center` to THz if needed,
- computes β_2 and β_3 from the given d_2 and d_3,
- sets `self.phase` to a function implementing $\phi(\nu) = \beta_2 (2\pi(\nu - \nu_0))^2/2 + \beta_3 (2\pi(\nu - \nu_0))^3/6$,
- updates `info_phase` and logs a short message,
- optionally plots the phase profile if `preview_plot` is `True`.

Input: `d2` (`float`), ps/nm; `d3` (`float`), ps/nm^2; `center` (`float`); `centerunit` (`str`); `preview_plot` (`bool`).
Return: tuple `(beta2, beta3)` of `float` values for the computed dispersion coefficients.

Interpolated phase masks

`set_interp_phase(x_fre, y_phase, center=1560, centerunit='nm', preview_plot=False)`

Program an arbitrary phase mask by cubic interpolation of a user supplied phase array. The method:

- optionally converts `center` to THz (for logging only),
- constructs a cubic `scipy.interpolate.interp1d` object over the input frequency grid `x_fre`,
- assigns this interpolator to `self.phase`, so that the phase can be evaluated

at any frequency within the device range and extrapolated outside,

- ◇ updates `info_phase` and logs a short message,
- ◇ optionally plots the interpolated phase versus `self.freq` if `preview_plot` is `True`.

Input: `x_fre` (`array-like`), frequency axis in THz; `y_phase` (`array-like`), phase values in radians defined on `x_fre`; `center` (`float`); `centerunit` (`str`); `preview_plot` (`bool`).
Return: none.

Band-pass filtering and inverse attenuation

`setBandPass(center=192.175, span=4, unit='thz')` Configure a simple rectangular band-pass attenuation profile. The method:

- ◇ if `unit == 'nm'`, converts the requested centre and span in nm to an equivalent frequency band in THz;
- ◇ computes start and stop frequencies in THz,
- ◇ sets `self.atten` to a function that returns `0 dB` inside the band and `MAX_ATTEN` outside (full attenuation),
- ◇ updates `info_atten` and logs the new band in both THz and nm.

Input: `center` (`float`), band centre (frequency in THz or wavelength in nm); `span` (`float`), band width; `unit` (`str`), `'thz'` or `'nm'`.
Return: none.

`inverseAtten(osa_wl, osa_pw, max_atten_db=5, osa_pw_noise=-30, osa_wl_center=1560, osa_wl_FSR=16, bandpass_center=299792458/1560/1000, bandpass_span=None, bandpass_unit='thz', perform_peak_search=True, plot_peak_search=True)`
Construct an inverse attenuation profile from an OSA-measured comb spectrum. The method:

- ◇ flattens `osa_wl` (nm) and `osa_pw` (dBm),
- ◇ optionally performs a peak search using `scipy.signal.find_peaks()`, with minimum spacing derived from `osa_wl_FSR` and `osa_wl_center`, keeping only peaks above `osa_pw_noise`,
- ◇ converts the selected peak wavelengths to frequencies in THz,
- ◇ builds a 1D interpolator of peak comb power and shifts it so that the strongest peak experiences `max_atten_db` attenuation,
- ◇ if `bandpass_center` and `bandpass_span` are provided, constructs a band-pass window in THz (or nm),
- ◇ defines `self.atten` so that frequencies within the band receive the interpolated attenuation (clipped at `0 dB`), while frequencies outside the band receive `MAX_ATTEN`,
- ◇ updates `info_atten` with both the band limits and the maximum attenuation; logs a summary,
- ◇ optionally plots the OSA spectrum and the detected peaks if `plot_peak_search` is `True`.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

If no valid peaks are found, a ValueError is raised.
Input:

- osa_wl (array-like), wavelengths in nm;
- osa_pw (array-like), powers in dBm;
- max_atten_db (float), maximum attenuation applied at the strongest comb lines;
- osa_pw_noise (float), noise floor threshold in dBm;
- osa_wl_center (float), central wavelength in nm for FSR estimation;
- osa_wl_FSR (float), comb FSR in GHz;
- bandpass_center (float or None), band-pass centre (THz or nm depending on bandpass_unit);
- bandpass_span (float or None), band-pass span; if None, no additional band-pass filter is applied;
- bandpass_unit (str), 'thz' or 'nm';
- perform_peak_search (bool), whether to treat the input as a comb (peaks) or as a smooth envelope;
- plot_peak_search (bool), whether to plot the OSA spectrum and detected peaks.

Return: none.

Profile upload and visualisation

writeProfile(amp=None, phase=None, disable_beep=False) Upload the current attenuation and phase profile to the Waveshaper. The method:

- if amp is not None, checks that its length equals numPixels and replaces self.atten by a lookup over the input amplitude array;
- if phase is not None, checks that its length equals numPixels and replaces self.phase by a lookup over the input phase array;
- builds a text buffer containing, for each frequency in self.freq, a line with frequency, attenuation, phase, and a fixed mask flag (1);
- calls ws_load_profile() with this buffer;
- on success, copies the preview atten and phase functions into current_atten and current_phase and logs a success message;
- optionally plays two short beeps using winsound unless disable_beep is True.

Invalid amp or phase array lengths are logged as errors and cause an early return.
Input: amp (array-like or None), attenuation values in dB; phase (array-like or None), phase values in radians; disable_beep (bool), if True suppresses the completion beep.
Return: none.

plotStatus(unit='thz') Generate a four-panel figure summarising the current and preview profiles. The panels show:

- ◇ current attenuation (−attenuation) versus frequency or wavelength;
- ◇ preview attenuation versus frequency or wavelength;
- ◇ current phase versus frequency or wavelength (only where current attenuation is below `MAX_ATTEN`);
- ◇ preview phase versus frequency or wavelength (only where preview attenuation is below `MAX_ATTEN`).

Input: `unit` (`str`), `'thz'` or `'nm'` for the horizontal axis.
Return: `matplotlib.figure.Figure` and axes objects (implicitly via `plt.subplots()`); the figure is shown on screen.

Frequency–wavelength conversion

`nm2thz(nm)` Convenience wrapper for converting wavelength in nm to optical frequency in THz using $\nu = c/\lambda$.
Input: `nm` (`float` or array-like), wavelength in nanometres.
Return: `float` or `numpy.ndarray`, frequency in THz.

`thz2nm(thz)` Convenience wrapper for converting frequency in THz to wavelength in nm using $\lambda = c/\nu$.
Input: `thz` (`float` or array-like), frequency in THz.
Return: `float` or `numpy.ndarray`, wavelength in nanometres.

Private helper methods (for example `__nm2thz()` and `__thz2nm()`) and low-level error handling logic are not documented here. For full implementation details, including the vendor API calls and internal logging behaviour, please refer to the source file `Waveshaper.py`.

3.21 Example integrated control script

3.21.1 Overview

This section gives a practical example of how the astrocomb control software brings up and monitors the main subsystems. The script is organised in several logical blocks:

- ◇ acquire an initial comb spectrum with the OSA;
- ◇ bring up EDFAs and RF power supplies;
- ◇ enable the safety relay and verify the repetition frequency;
- ◇ program the waveshaper dispersion and band-pass;
- ◇ configure the minicomb intensity-lock servo;
- ◇ set up the Pritel amplifier, shutters, and VOAs;
- ◇ configure oscilloscope, function generators, ORION laser, and TEC;
- ◇ read rack and table temperatures via the USB-2408 DAQs.

All VISA/serial addresses and numerical values are *examples* copied from the com-

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

missioning notebook. They must be checked and updated before running on the real system.

3.21.2 Step 1: OSA startup comb spectrum

```
from KeckLFC.Hardware.Agilent_86142B import Agilent_86142B

osa = Agilent_86142B()
osa.connect()
osa.Run()                    # start continuous sweep

# Configure a 1553-1567 nm window with a safe reference level
osa.wlstart  = 1553          # nm
osa.wlstop   = 1567          # nm
osa.reflevel = -40           # dBm at top of display

# Acquire and save a trace as a startup reference
basedir  = r"C:\Users\KeckLFC\Desktop\hispec"
basename = r"\startup_comb_example"

osa.get_trace("a")
osa.save_trace("a", basedir + basename)
```

3.21.3 Step 2: EDFAs and RF power supplies

```
from KeckLFC.Hardware.AmonicsEDFA       import AmonicsEDFA
from KeckLFC.Hardware.InstekGPD_4303S   import InstekGPD_4303S
from KeckLFC.Hardware.InstekGppDCSupply import InstekGppDCSupply

# High-power Amonics EDFA (27 dBm, main booster)
amonic27 = AmonicsEDFA(addr="ASRL6::INSTR", name="Amonics EDFA 27 dBm")
amonic27.connect()
amonic27.printStatus()

amonic27._setChMode("apc")        # automatic power control mode
amonic27.accCh1Cur    = "450mW"   # target output power
amonic27.accCh1Status = 1        # enable channel 1
amonic27.activation   = 1        # enable main output

# Lower-power Amonics EDFA (23 dBm, preamp)
amonic23 = AmonicsEDFA(addr="ASRL12::INSTR", name="Amonics EDFA 23 dBm")
amonic23.connect()
amonic23.accCh1Cur    = "80mA"
amonic23.accCh1Status = 1
amonic23.activation   = 1

# RF oscillator supply (Instek GPD-4303S)
```

```
rfoscPS = InstekGPD_4303S(addr="ASRL13::INSTR",
                          name="RF oscillator PS (CH2 15 V, CH3 1 V)")
rfoscPS.connect()

rfoscPS.Vset2 = 15.0   # V for RF oscillator
rfoscPS.Iset2 = 3.0    # A current limit
rfoscPS.Vset3 = 0.8    # V bias for modulator
rfoscPS.Iset3 = 1.0    # A current limit
rfoscPS.activation = 1 # enable outputs

# RF amplifier supply (Instek GPP DC supply)
rfampPS = InstekGppDCSupply(addr="ASRL24::INSTR",
                            name="RF amplifier PS 30 V 4 A")
rfampPS.connect()
rfampPS.Vset1 = 30.0   # V for RF power amplifier
rfampPS.Iset1 = 4.2    # A current limit
rfampPS.activation1 = 1
```

3.21.4 Step 3: Safety relay and repetition-rate check

```
from KeckLFC.Hardware.Arduino_relay import Arduino_relay
from KeckLFC.Hardware.PendulumCNT90 import PendulumCNT90

# Safety relay (interlock)
arduino = Arduino_relay(addr="COM3")
arduino.connect()
arduino.printStatus()

arduino.reset_relay_latch()   # clear previous fault
arduino.pass_YJ()             # allow YJ path (still interlocked)
arduino.get_YJ_info()

# Pendulum CNT-90 frequency counter
counter = PendulumCNT90()
counter.connect()

freq = counter.measFreq("c")  # measure RF oscillator on input "c"
print("Measured RF frequency: {:.6f} GHz".format(freq/1e9))
print("Offset from 16 GHz: {:.1f} Hz".format(freq - 16e9))
```

3.21.5 Step 4: Waveshaper dispersion and band-pass

```
from KeckLFC.Hardware.Waveshaper import Waveshaper

wsp = Waveshaper()
wsp.connect()
```

```
# Program 2nd + 3rd order dispersion around 1559.8 nm (example values)
wsp.set3rdDisper(d2=2.14, d3=0.0, center=1559.8, centerunit="nm")

# Apply a 5 THz wide band-pass
wsp.setBandPass(span=5.0, unit="thz")

# Inspect and upload the profile
wsp.plotStatus(unit="nm")
wsp.writeProfile()
```

3.21.6 Step 5: Minicomb intensity-lock servo

```
import numpy as np
from KeckLFC.Hardware.SRS_SIM900 import SRS_SIM900, SRS_PIDcontrol_SIM960

srs = SRS_SIM900(addr="GPIB0::2::INSTR")
srs.connect()

servo_IM = SRS_PIDcontrol_SIM960(srs, 3,
                                 name="Minicomb Intensity Lock Servo")

# Safe output limits and initial PID parameters
servo_IM.output_upperlim = 10.0
servo_IM.output_lowerlim = -10.0
servo_IM.prop_gain = -2.0
servo_IM.intg_gain = 0.1
servo_IM.outoffset  = 0.2
servo_IM.set_setpoint(2.2)

# Start in manual mode and sweep the actuator for characterisation
servo_IM.output_mode = "man"
servo_IM.manual_output_ramp = 0.1

scan_start = -3.0
scan_stop  =  3.0
num_steps  = 100
voltages   = np.linspace(scan_start, scan_stop, num_steps)

servo_IM.manual_output = scan_start
response = servo_IM.manual_sweep(voltages)

# Park at a good value and switch to closed loop
servo_IM.manual_output = 1.4
servo_IM.output_mode   = "pid"
servo_IM.printStatus()
```

3.21.7 Step 6: Pritel amplifier, shutters, and VOAs

```
from KeckLFC.Hardware.PritelAmp   import PritelAmp
from KeckLFC.Hardware.hk_shutter  import hk_shutter
from KeckLFC.Hardware.OZopticsVOA import OZopticsVOA

# Pritel amplifier
ptamp = PritelAmp(addr="ASRL7::INSTR", name="Pritel Amp")
ptamp.connect()
ptamp.printStatus()

ptamp.preAmp = 600          # mA (seed EDFA)
ptamp.activation = 1

# Only increase pwrAmp after interlocks are checked
ptamp.pwrAmp = "0A"         # typical operation: 3.8-4.2 A

# Shutter
shutter = hk_shutter(addr="COM17")
shutter.connect()
shutter.set_status(0)       # default: block beam

# 1550 nm VOA
voa1550 = OZopticsVOA(addr="ASRL15::INSTR", name="1.55 um VOA")
voa1550.connect()
voa1550.atten_db = 0.0
```

3.21.8 Step 7: Scope, function generators, ORION laser, and TEC

```
import time
from KeckLFC.Hardware.TDS2024C         import TDS2024C
from KeckLFC.Hardware.KeysightFG_33500 import KeysightFG_33500
from KeckLFC.Hardware.ORIONLaser       import ORIONLaser
from KeckLFC.Hardware.TEC_TC720        import TEC_TC720

# Oscilloscope and function generators
osc = TDS2024C()
osc.connect()

fg_main = KeysightFG_33500(
    addr="USB0::0x0957::0x2807::MY59003824::INSTR",
    name="Keysight FG 33500 (main)"
)
fg_main.connect()

fg_pdh = KeysightFG_33500(
    addr="USB0::0x0957::0x2807::MY62003852::INSTR",
```

```
    name="Keysight FG 33500 (PDH)"
)
fg_pdh.connect()

fg_main.set_channel_parameters(channel=1, freq=10, amp=8, offset=0, phase=0)
fg_main.set_channel_func(1, "ramp")
fg_main.set_channel_state(1, 0)   # keep off until ready

osc_trace = osc.get_trace(trace=2)

# ORION laser
rio = ORIONLaser(addr="ASRL14::INSTR")
rio.connect()
rio.writeLaserdiodeCur_mA(150)
rio.writeTECsetpoint(19.181 + 4.1)
rio.writeLaserdiodeCur_mA(150, volatile=False)

# TEC controller (example PPLN doubler)
tec_ppln = TEC_TC720(addr="COM22", name="PPLN doubler TEC (TC720)")
tec_ppln.connect()

current_temp = tec_ppln.get_temp()
target_temp  = 20.1
step         = 0.5

if target_temp > current_temp:
    n_steps = int((target_temp - current_temp) / step)
    for i in range(1, n_steps + 1):
        tec_ppln.set_temp(current_temp + i * step)
        time.sleep(4.0)
        print("Intermediate temperature:", tec_ppln.get_temp())

tec_ppln.set_temp(target_temp)
```

3.21.9 Step 8: Rack and table temperature monitoring

```
from KeckLFC.Hardware.USB2408 import USB2408

daq_rack  = USB2408(addr=0)
daq_table = USB2408(addr=1)

daq_rack.connect()
daq_table.connect()

temps_rack  = daq_rack.get_temp_all()
temps_table = daq_table.get_temp_all()
```

```
print("Rack temperatures (C): ", temps_rack)
print("Table temperatures (C):", temps_table)
```

In a production control program, each of these steps should be wrapped in `try/finally` blocks with explicit calls to `disconnect()`, and additional interlock logic should be added to ensure that power-up and shutdown sequences are always safe.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

CHAPTER 4

LFC KTL keyword function, ICE server, and Dispatcher

4.1 Comb KTL service

4.1.1 Overview

The software that controls the laser frequency comb (LFC) is implemented in the `KeckLFC` class and runs on the Windows LFC laptop. This local control layer is connected to the observatory KTL service `comb`, which runs on the `IRASTROCOMB` server, via an ICE (Internet Communications Engine) connection.

In this directory:

- `server.py` is the ICE *server* script running on the LFC laptop.
- The KTL dispatcher `comb` on `IRASTROCOMB` is the ICE *client*.

Users never talk directly to `KeckLFC`. Instead they access LFC-related KTL keywords through the `comb` dispatcher:

- user commands such as `show -s comb KEYWORD` and `modify -s comb KEYWORD=value` go to the KTL dispatcher;
- the dispatcher acts as an ICE client and communicates with `server.py`;
- `server.py` calls the corresponding methods of `KeckLFC` to read or update keyword values.

To operate the comb entirely through KTL:

- the ICE server (`server.py`) must be running on the LFC laptop;
- the ICE connection between `server.py` and the `comb` dispatcher must be established and healthy.

As of August 16, 2024, both the dispatcher and the ICE server are still in an active development phase and support only a limited set of keywords. The mechanisms described here are therefore important both for day-to-day use and for future extension of the keyword set.

4.1.2 Establishing the ICE connection

In normal operation the ICE server and the dispatcher should be running continuously and remain connected. The connection state is exposed via the KTL keyword `ICESTA`. From any KTL-enabled shell this can be checked with:

```
show -s comb icesta
```

The `ICESTA` keyword reports:

- `Connected`: the ICE server and the dispatcher are connected and ready. It is safe to use `show` and `modify` to read and set LFC status.
- `Disconnected`: either
  - the ICE server is not running on the LFC laptop, or
  - the ICE server is running but the ICE connection has been lost (for example after restarting `server.py`).
- `Initializing`: the dispatcher is re-establishing the ICE connection.

If `ICESTA` shows `Disconnected`, proceed as follows:

(1) Check whether the ICE server is running on the LFC laptop. If it is not running, it must be started locally (see Section 4.1.3.2). This currently must be done by a qualified user (for example Jinhao or Yoo Jung, as of August 16, 2024). Nothing can be done from the KTL side alone.

(2) If `server.py` is running but the connection was lost, a KTL user can ask the dispatcher to reconnect by:

```
modify -s comb icesta=3
```

After a few seconds, run:

```
show -s comb icesta
```

again to confirm that the state returns to `Connected`.

A second keyword, `ICECLK`, is useful to verify that the connection is not only up but actually carrying traffic:

```
show -s comb iceclk
```

`ICECLK` is updated every $\sim$ 15 seconds by the ICE server. If the ICE connection is healthy, the reported time should be no more than 15 seconds old.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

If `ICESTA` remains `Disconnected` or `ICECLK` does not update even after retrying `icesta=3`, the issue may be with the dispatcher. In that case, restart the dispatcher as described in Section 4.1.3.3. If that still does not resolve the problem, the issue should be escalated to the LFC software maintainer (for example Yoo Jung).

4.1.3 Notes for developers

This section summarises the key files and workflows on the LFC Windows laptop and on the observatory servers for developers who need to modify keywords, dispatcher behaviour, or the ICE interface.

4.1.3.1 Important files on the LFC Windows laptop

The following files live on the LFC laptop in the `KeckLFC` repository:

- `combd.sin`: dispatcher code template for the `comb` service.
- `LFC.xml.sin`: full XML file in which KTL keywords are defined (names, types, and metadata). This is the XML file used by the ICE server.
- `server.py`: ICE server script that:
  - reads the keyword list from `LFC.xml.sin`;
  - instantiates `KeckLFC`;
  - exposes each keyword over ICE.
- `KTLIce.ice`, `config.server`, and (on the observatory side) `config.client`: ICE interface and configuration files.

4.1.3.2 Starting the ICE server

On the LFC laptop, the ICE server is started from the `KTL server` directory using the LFC conda environment. A typical command sequence is:

```
cd .\KeckLFC-main
cd .\KeckLFC
cd ".\KTL server"

& C:/Users/KeckLFC/.conda/envs/lfc-env/python.exe ^
  "c:/Users/KeckLFC/KeckLFC-main/KeckLFC/KTL server/server.py"
```

In practice the final prompt looks like:

```
PS C:\Users\KeckLFC\KeckLFC-main\KeckLFC\KTL server> \
  & C:/Users/KeckLFC/.conda/envs/lfc-env/python.exe \
  "c:/Users/KeckLFC/KeckLFC-main/KeckLFC/KTL server/server.py"
```

Alternatively, once the working directory is set, the command is simply:

```
python server.py
```

When `server.py` starts successfully, it reads `LFC.xml.sin`, creates a `KeckLFC` instance, and begins listening for ICE connections from the dispatcher.

4.1.3.3 Starting the dispatcher

On the observatory side, the dispatcher for the `comb` service runs on `IRASTROCOMB`. To start it:

```
ssh combbld@irastrocomb
astrocomb start comb
```

To restart an already running dispatcher (for example after updating XML or dispatcher code):

```
ssh combbld@irastrocomb
astrocomb restart comb
```

The dispatcher reads the same `LFC.xml.sin` file (copied and installed on the build server) and creates KTL keyword objects that are linked to the ICE server. It periodically polls read keywords and forwards writes to `KeckLFC` via ICE.

4.1.3.4 Making changes to keywords

KTL keywords are defined in the XML file `LFC.xml.sin`. When modifying keywords, two consistency rules are critical:

(1) For every keyword defined in `LFC.xml.sin`, a method with the same name must exist in `KeckLFC.py`. Even if the implementation is not ready, the method must be defined so that the dispatcher can call it safely.
(2) The `LFC.xml.sin` file used by the ICE server (on the LFC laptop) and the `LFC.xml.sin` file used by the KTL dispatcher (on the build and runtime servers) must be identical.

To deploy changes to `LFC.xml.sin`:

(1) Copy the updated XML file from the LFC laptop (or your local development machine) to the build server:

```
scp LFC.xml.sin \
  combbld@irastrocombbuild:/kroot/src/kss/astrocomb/comb/ktlxml/LFC.xml.sin
```

(2) Log in to the build server, install, and deploy:

```
ssh combbld@irastrocombbuild
cd /kroot/src/kss/astrocomb/comb/
make install
```

(3) Deploy to the runtime server:

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

```
kdeploy -a
```

(4) On `IRASTROCOMB`, restart the `comb` dispatcher:

```
ssh combbld@irastrocomb
astrocomb restart comb
```

After this procedure the updated keyword definitions are active both on the dispatcher and on the ICE server, assuming the new XML file has also been placed in the corresponding directory on the LFC laptop.

4.1.3.5 Making changes to dispatcher code

The KTL dispatcher logic is implemented in `combd.sin`. When editing dispatcher behaviour (for example, how quickly a keyword is polled or how it is presented), proceed as follows:

(1) Edit `combd.sin` in the LFC repository or in a local checkout.
(2) Copy the updated dispatcher file to the build server:

```
scp combd.sin \
  combbld@irastrocombbuild:/kroot/src/kss/astrocomb/comb/dispatcher/combd.sin
```

(3) On `irastrocombbuild`, run:

```
ssh combbld@irastrocombbuild
cd /kroot/src/kss/astrocomb/comb/
make install
kdeploy -a
```

(4) Finally, restart the dispatcher on `IRASTROCOMB`:

```
ssh combbld@irastrocomb
astrocomb restart comb
```

This ensures that the dispatcher and the ICE server stay in sync in terms of keyword lists and behaviour.

4.1.4 Details on keywords and type handling

4.1.4.1 Background KTL references

For more background on KTL and keyword XML definitions, see:

- https://spg.ucolick.org/DFW/keyword.html

- https://spg.ucolick.org/KTLXML/

In the dispatcher, KTL keywords are defined as `DFW.Keyword.Basic` objects, and specific types are subclasses such as `DFW.Keyword.Integer`, `DFW.Keyword.String`, and so on. For the comb system, the dispatcher defines convenience subclasses such as `IntegerIce`, `DoubleIce`, `EnumeratedIce`, `StringIce`, `BooleanIce`, and `DoubleArrayIce`. These classes inherit from the corresponding `DFW.Keyword.KEYWORDTYPE` classes and override methods like `read()`, `write()`, `postwrite()`, and `update()` to communicate with the ICE server instead of a local process.

All values passed between KTL and the dispatcher are strings. When a user modifies a keyword via `modify`, the dispatcher calls a `write()` method, which in turn invokes `modifiedkeyword()` in `server.py`. On the server side, `modifiedkeyword()` delegates to the `__setitem__` mechanism in `KeckLFC`:

- The incoming string is converted to the correct Python type by `convert_type()`.
- The corresponding keyword method in `KeckLFC` is called in its *write block* (with `value` not `None`).
- If the write block returns `0`, the new value is stored in `self.keywords`.

When accessing keyword values inside `KeckLFC`, the dictionary `self.keywords` contains the most recent values:

```
self.keywords["KEYWORDNAME"]
```

Array-type keywords are stored as Python lists. (Originally they were strings with space-separated elements; this has been updated so that the internal representation is now a list, while the dispatcher still sees a string.)

4.1.5 How the ICE connection works

The overall data flow between KTL, ICE, and `KeckLFC` is as follows.

4.1.5.1 Server-side behaviour

On the LFC laptop, `server.py` performs these steps:

(1) Read the keyword definitions from `LFC.xml.sin`.
(2) Create a `KeckLFC` instance, which:
    - parses the same XML file;
    - builds dictionaries of keyword names, types, and functions;
    - exposes methods whose names match the KTL keywords.
(3) Initialise ICE servants for each keyword and wait for the KTL dispatcher to connect.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

4.1.5.2 Dispatcher-side behaviour

On the observatory side, when the `comb` dispatcher starts:

(1) It reads `LFC.xml.sin` from the build tree on `irastrocombbuild`. (Therefore the XML file on the LFC laptop and the one on the build server must be identical.)
(2) In its internal `setupKeywords()` routine, it creates KTL keyword objects (such as `IntegerIce`, `DoubleIce`, etc.) corresponding to each XML entry.
(3) For each keyword, the dispatcher:

- ◇ calls `read()` periodically; this invokes a `receive()` method in the ICE interface class, which calls the corresponding `KeckLFC` function with `value=None` and stores the returned value;
- ◇ calls `write()` when the user modifies a value; this invokes `modifiedkeyword()` in `server.py`, which translates the value and calls the `KeckLFC` function with `value` not `None`.

A successful ICE connection therefore requires:

- ◇ the server (LFC laptop) and client (astrocomb server) to share an identical `LFC.xml.sin` file;
- ◇ a defined function in `KeckLFC` for every keyword in the XML file.

Even if the middle-layer implementation for a keyword is incomplete, `KeckLFC` must still provide a stub function. A minimal placeholder looks like:

```
def KEYWORD_NAME(self, value=None):
    return
```

This ensures that the dispatcher can call the keyword without raising an exception, even if it does nothing yet.

4.1.6 KTL keywords and the KeckLFC class

In `KeckLFC.py`, the `KeckLFC` class:

- ◇ reads the `LFC.xml.sin` file;
- ◇ stores keyword names, types, and current values in `self.keywords`;
- ◇ defines methods whose names match the keywords listed in the XML.

Each keyword method has the standard signature:

```
def KEYWORD(self, value=None):
    if value is None:
        # Called periodically (keyword read)
```

```
        # Insert logic to read hardware and return a value.
        # If KeckLFC should not modify this keyword (e.g. ICESTA),
        # simply do not return anything.
        return

    else:
        # Called when user modifies the keyword (keyword write)
        # Insert logic to act on the new value.
        # If successful, return 0 so the value is stored.
        return 0
```

Reads are driven by the dispatcher calling the method with `value=None`. Writes are driven by user `modify` commands, which cause the dispatcher to send a new value string; this is converted to the correct Python type and then passed into the keyword method with `value` not `None`.

This split read/write behaviour, combined with a shared XML file and the ICE transport layer, is what allows the LFC to be fully monitored and controlled using standard KTL commands (`show` and `modify`) while keeping all hardware-specific logic inside `KeckLFC`.

4.2 ICE-based control interface

4.2.1 Overview

The ICE layer provides the network-facing control interface for the astrocomb. It exposes a small set of methods (defined in the `KtlIce.ice` slice as the `CombIce.Lfc` interface) that allow the KTL dispatcher and other clients to read and modify LFC keywords over the network. Internally, these keyword operations are translated into attribute access on the main `KeckLFC` hardware control class (or on a `mockKeckLFC` object for testing).:contentReference[oaicite:0]index=0

The ICE server is configured with `config.server` and the KTL keyword definition file (`LFC.xml.sin` or a test variant). When running, it registers a single object with identity `"lfc"` on an ICE object adapter named `"Lfc"`.:contentReference[oaicite:1]index=1

A separate reference client script (adapted from the ZeroC “Hello” example) demonstrates how to connect to an ICE server, exercise different invocation modes (twoway, oneway, batch, datagram), toggle timeout and artificial delay, and shut down the server.:contentReference[oaicite:2]index=2

4.2.2 LFC ICE server (`server.py`)

Server class and construction

The main implementation of the `CombIce.Lfc` interface is the `LfcI` class defined in `server.py`.:contentReference[oaicite:3]index=3

`class LfcI(CombIce.Lfc)` Implementation of the ICE interface used by the KTL dispatcher to interact with the LFC. On construction, the class:

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

- ◇ parses the KTL description file (`LFC.xml.sin` or `testLFC.xml.sin`) to obtain the list of keyword names and types via `parse_xml()`,
- ◇ stores the list of keyword names in `self.keyword_names`,
- ◇ instantiates either a real `KeckLFC` controller or a `mockKeckLFC` object (selected by the `use_mock` flag) and stores it as `self.mkl`,
- ◇ prints a startup banner (`"ICE server starts ..."`).

The `KeckLFC`/`mockKeckLFC` instance provides a mapping from keyword names to live values and performs all hardware access.:contentReference[oaicite:4]index=4
Input: none (constructor).
Return: a new `LfcI` instance.

Keyword access methods

These methods implement the core ICE interface used by the KTL dispatcher to send and receive keyword values.:contentReference[oaicite:5]index=5

`modifiedkeyword(name, value, current)` Notify the LFC that a keyword value has changed. This method is called by the dispatcher whenever a keyword is modified in the KTL layer. Internally it forwards the change to the `KeckLFC` instance by assigning `self.mkl[name] = value`, and prints a log message indicating the new value.
Input: `name` (`str`), keyword name; `value` (any type that the underlying controller accepts), new keyword value; `current` (ICE `Current` object, provided by the framework and not normally used directly).
Return: none.

`receive(name, current)` Retrieve the current value of a keyword. This method is used by the dispatcher to obtain the value stored inside the `KeckLFC` object. Internally it reads `self.mkl[name]` and returns its string representation.
Input: `name` (`str`), keyword name; `current` (ICE `Current` object).
Return: `str`, the keyword value converted to a string.

`keylist(current)` Return the list of available keyword names. This allows the KTL dispatcher or other clients to discover which keywords are exposed by the LFC ICE interface.
Input: `current` (ICE `Current` object).
Return: list of `str`, keyword names parsed from the `*.xml.sin` configuration.

`cleanup(current)` Perform server-side cleanup when requested by the dispatcher. The current implementation prints a message (`"cleaning up"`) and leaves commented-out placeholders for stopping clocks, disconnecting hardware (for example the Arduino), and optionally shutting down the ICE communicator. This method is a hook for future resource management; it does not stop the server by default.
Input: `current` (ICE `Current` object).
Return: none.

`shutdown(current)` Shut down the ICE server. This method is not normally called

by the dispatcher, but can be used for an orderly shutdown of the LFC ICE process. It obtains the communicator from `current.adapter.getCommunicator()` and calls `shutdown()` on it, which causes the main event loop to exit.
Input: `current` (ICE `Current` object).
Return: none.

For more details on the internal keyword mapping and the `KeckLFC` implementation, see the main hardware-control module (`KeckLFC`) and the XML configuration files referenced by `parse_xml()`.:contentReference[oaicite:6]index=6

Server main loop and adapter setup

The end of `server.py` contains the top-level code that initialises the ICE communicator, installs the signal handler, and starts the server event loop.:contentReference[oaicite:7]index=7

`with Ice.initialize(sys.argv, 'config.server') as communicator:` Initialise ICE with the command-line arguments and the configuration file `config.server`. All ICE-related arguments are removed from `sys.argv`. If any non-ICE arguments remain, the program prints an error and exits. The communicator object manages network connections and the lifetime of the adapter.

`signal.signal(signal.SIGINT, lambda signum, frame: communicator.shutdown())` Register a signal handler so that pressing `Ctrl+C` triggers `communicator.shutdown()`. This provides a clean way to stop the server from the command line.

`adapter = communicator.createObjectAdapter("Lfc")` Create an ICE object adapter named `"Lfc"`. The adapter listens on the endpoints specified in `config.server` and dispatches incoming requests to registered servants.

`adapter.add(LfcI(), Ice.stringToIdentity("lfc"))` Create an `LfcI` instance and register it with the adapter under the identity `"lfc"`. Clients that know this identity can obtain a proxy to the object via standard ICE mechanisms.

`adapter.activate()` Activate the adapter, enabling it to receive requests.

`communicator.waitForShutdown()` Enter the main event loop and block until `communicator.shut` is called (either by the signal handler or by the `shutdown()` method of `LfcI`). When this call returns, the `with` block exits and the communicator is destroyed automatically.

4.2.3 Reference ICE client (`client.py`)

Client overview

The `client.py` script is a reference interactive client adapted from the ZeroC "Hello" demo. It is not specific to the LFC interface, but demonstrates the main features of an ICE client: initialising a communicator, creating a proxy from configuration, using different invocation modes, setting timeouts and artificial delays, and shutting down the server.:contentReference[oaicite:8]index=8

The script:

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

- ◇ loads the slice definition `Hello.ice` and imports the generated `Demo` module,
- ◇ obtains a proxy to `Demo.Hello` from the `Hello.Proxy` property in `config.client`,
- ◇ builds variants of this proxy for twoway, oneway, batch oneway, datagram, and batch datagram modes,
- ◇ enters an interactive loop where single-character commands trigger different types of remote calls.:contentReference[oaicite:9]index=9

Interactive commands

Within the `run()` loop, the following commands are handled::contentReference[oaicite:10]index=10

`t` Call `sayHello(delay)` on the twoway proxy. The client waits for the server to respond (synchronous RPC).

`o` Call `sayHello(delay)` as a oneway invocation. The request is sent and the client does not wait for a reply.

`O` Queue a batch oneway `sayHello(delay)` request. The queued batch can later be sent with the `f` command.

`d, D` Send `sayHello(delay)` as datagram or batch datagram invocations (if secure mode is not enabled). Datagrams are connectionless and unreliable.

`f` Flush all pending batch oneway and batch datagram requests by calling `ice_flushBatchRequests` on the corresponding proxies.

`T` Toggle an invocation timeout. When enabled, a timeout of 2000 ms is applied to subsequent calls by decorating the proxies with `ice_invocationTimeout(2000)`. When disabled, the timeout is set back to `-1` (no timeout). The script prints the new timeout state.

`P` Toggle an artificial server delay by switching a `delay` variable between 0 and 2500 ms. The value of `delay` is passed to `sayHello(delay)` so that the server can simulate slow responses.

`S` Switch secure mode on or off. When secure mode is toggled, all proxies are re-decorated with `ice_secure(secure)`. Secure datagrams are not supported, so the corresponding commands print a warning in secure mode.

`s` Call `shutdown()` on the server side (for the demo). This instructs the server to shut down its communicator.

`x` Exit the client loop and terminate the program.

`?` Print the help menu listing all commands (via the `menu()` function).

The client is initialised with `with Ice.initialize(sys.argv, "config.client")` as communicator: and checks that no extra non-ICE arguments are present on the command line. This pattern can be reused when writing a dedicated client for the

LFC ICE server (using the `CombIce.Lfc` interface instead of the demo `Demo.Hello` interface).:contentReference[oaicite:11]index=11

4.3 KeckLFC keyword controller (code realization level)

4.3.1 Environment and configuration

The `KeckLFC` class is the central keyword controller for the laser frequency comb (LFC). It sits between the KTL/ICE keyword layer and the individual hardware drivers in `Hardware.*`. All KTL keywords defined in `LFC.xml.sin` are mapped to methods of `KeckLFC`; each method is responsible for reading from or writing to the appropriate instrument.

Two global flags near the top of `KeckLFC.py` control the operating mode:

- `icetest_mode`: when set to `True`, the constructor parses `testLFC.xml.sin` instead of `LFC.xml.sin` (ICE / KTL test mode).
- `test_mode`: when set to `True`, most hardware methods return immediately without touching real instruments, so the logic can be exercised safely.

KTL keyword definitions (names, types, and enumerations) are stored in `LFC.xml.sin` and are read at runtime by `parse_xml()`. To add or modify keywords, edit this XML file and ensure that a corresponding method with the same name exists in `KeckLFC`.

The ICE server that exposes these keywords is configured via `config.server` (endpoints, adapter name, properties). On the client or dispatcher side, the ICE configuration is given in `config.client` or in the KTL dispatcher configuration.

Instrument VISA or serial addresses and human-readable names are set via arguments to `KeckLFC__init__` and inside private connection helpers such as `__LFC_RIO_connect()`, `__LFC_EDFA27_connect()`, and `__LFC_USB2408_0_connect()`. When the rack wiring changes, these addresses should be updated in `KeckLFC.py`.

4.3.2 Core infrastructure

parse_xml Parse the KTL keyword XML description.
Function: read `LFC.xml.sin` (or `testLFC.xml.sin`), iterate over `<keyword>` elements, and collect the `<name>` and `<type>` fields. Used by the `KeckLFC` constructor.
Input: `xmlfile` (string), path to the XML file.
Return: two lists of strings `(keyword_names, keyword_types)` of equal length.

KTLarray Convert a Python list to a KTL-style array string.
Function: join list elements into one string with space separators (used for `"double array"` types).
Input: `values` (list or `None`).
Return: empty string if `values` is `None`; otherwise one space-separated string.

KeckLFC___init___ Constructor for the central keyword controller.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Function:

- ◇ choose the XML file (`LFC.xml.sin` or `testLFC.xml.sin`) based on `icetest_mode`;
- ◇ build internal dictionaries `self.keywords` and `self.types` from `parse_xml()`;
- ◇ build `self.funcs` mapping keyword names to methods of `KeckLFC`;
- ◇ create helper attributes (for example `self.arduino`, `self.ws`, `self.clock`) and start the internal clock thread.

Input: optional address and name strings for EDFAs, Pritel amplifier, RF power supplies, rubidium clock, and waveshaper.
Return: a configured `KeckLFC` instance.

KeckLFC___getitem___ Read a keyword value.
Function: look up the handler in `self.funcs[key]` and call it with `value=None`. If the handler returns a non-`None` value, store it in `self.keywords[key]`. For array types, convert lists into KTL strings with `KTLarray()`.
Input: `key` (string), keyword name.
Return: Python value for the keyword (number, string, boolean, or array string), or `None` on error.

KeckLFC___setitem___ Write a keyword value.
Function: convert the incoming KTL string `val` to a Python value using `convert_type(self.typ` then call `self.funcs[key](value=val_python)`. If the handler returns `0`, treat the write as successful and update `self.keywords[key]`.
Input: `key` (string), keyword name; `val` (string), new KTL value.
Return: none.

convert_type Convert a KTL string to the correct Python type.
Function: map type names (`"integer"`, `"double"`, `"enumerated"`, `"string"`, `"boolean"`, `"double array"`) to the appropriate conversion (`int()`, `float()`, list of `float`, etc.). Boolean strings such as `"True"` or `"1"` are mapped to `True`.
Input: `typ` (string), KTL type name; `val` (string), raw KTL value.
Return: Python value (int, float, string, bool, or list).

test_clock Background clock thread.
Function: while `stop()` returns `False`, update `mkl.keywords["ICECLK"]` once per second with the current time string (HH:MM:SS). When `stop()` returns `True`, clear `mkl.clock` and exit.
Input: `stop` (callable returning bool), `mkl` (`KeckLFC` instance).
Return: none.

___sleep Convenience wrapper around `time.sleep()`.
Function: pause execution for a given number of seconds and return the same value; used as a settling delay between hardware commands.
Input: `value` (float, seconds).
Return: the same float `value`.

___sendemail Email notification helper.
Function: build a multipart email (plain text plus optional attachments) and

send it via the Gmail SMTP server. Used by protection keywords (for example temperature and RF monitors) when they trigger `LFC_CLOSE_ALL`.
Input: `mail_content` (string) and optional subject, recipient list, attachment list, and log path.
Return: none.

A large set of private helpers of the form `__LFC_XXX_connect()` (for example `__LFC_RIO_connect()`, `__LFC_EDFA27_connect()`, `__LFC_USB2408_0_connect()`) each import a specific hardware driver and return a ready-to-use device object. They usually take no arguments and do not call `connect()`; the public keyword handlers are responsible for connecting, commanding, and disconnecting each instrument.

4.3.3 Representative keyword functions

Each keyword function follows the same pattern:

- ◇ if called with `value=None`, it performs a *read* and returns the current status or measurement;
- ◇ if called with `value` not `None`, it performs a *write*, sends commands to the hardware, and usually returns `0` on success.

Below we summarise representative functions by subsystem.

4.3.3.1 Temperature monitoring and DAQs

LFC_TEMP_TEST1 Rack-side temperature array (USB-2408 at address 0).
Function: on read, connect to `USB2408(addr=0)`, read all thermocouple channels, cache them, and return a list of temperatures. On write, the input is ignored and no hardware is changed.
Input: `value=None` for read; any value for write (ignored).
Return: list of floats (degrees Celsius) on read; integer `0` on write.

LFC_TEMP_TEST2 Table-side temperature array (USB-2408 at address 1).
Function: same pattern as `LFC_TEMP_TEST1` but using `USB2408(addr=1)` for components on the optical table.
Input: `value=None` for read; any value for write (ignored).
Return: list of floats on read; integer `0` on write.

LFC_T_GLY_RACK_IN Rack glycol inlet temperature.
Function: on read, return a single element of the cached array from `LFC_TEMP_TEST1` corresponding to the glycol inlet sensor. On write, ignore the input.
Input: `value=None` for read.
Return: single float on read; integer `0` on write.

LFC_T_GLY_RACK_OUT Rack glycol outlet temperature.
Function: same structure as `LFC_T_GLY_RACK_IN` but for the outlet sensor.
Input: `value=None` for read.
Return: single float on read; integer `0` on write.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

LFC_T_EOCB_IN EOCB inlet temperature.
Function: on read, return the appropriate element from the cached temperature arrays; on write, ignore the input.
Input: `value=None` for read.
Return: float on read; integer `0` on write.

LFC_T_EOCB_OUT EOCB outlet temperature.
Function: same pattern as `LFC_T_EOCB_IN`.
Input: `value=None` for read.
Return: float on read; integer `0` on write.

LFC_T_RACK_TOP Rack top temperature.
Function: on read, return top-of-rack temperature from the cached DAQ array.
Input: `value=None` for read.
Return: float on read; integer `0` on write.

LFC_T_RACK_MID Rack mid temperature.
Function: same as above for the middle sensor.
Input: `value=None` for read.
Return: float on read; integer `0` on write.

LFC_T_RACK_BOT Rack bottom temperature.
Function: same as above for the bottom sensor.
Input: `value=None` for read.
Return: float on read; integer `0` on write.

LFC_TEMP_MONITOR Temperature safety monitor.
Function: on read, inspect the cached temperature arrays and compare against fixed thresholds. If any channel exceeds its limit, this keyword may call `LFC_CLOSE_ALL(1)` and send an email.
Input: `value=None` (read-only).
Return: integer status flag (for example `1` for OK, `0` for fault).

4.3.3.2 RF power supplies and RF monitoring

LFC_RFOSCI_V RF oscillator supply voltage.
Function: on read, connect to the RF oscillator supply and return the measured output voltage (for example channel 2, `Vout2`). On write, set the voltage setpoint (for example `Vset2`) either to a default value or to the user-provided value.
Input: `value=None` to read; `value='default'` for a standard setting; numeric `value` (volts) to write.
Return: float voltage on read or after write.

LFC_RFOSCI_I RF oscillator supply current.
Function: same pattern as `LFC_RFOSCI_V`, but for current (for example `Iout2` and `Iset2`).
Input: `value=None`, `'default'`, or numeric amps.
Return: float current in amperes.

LFC_RFOSCI_ONOFF RF oscillator on/off state.
Function: on read, return the activation state of the RF oscillator supply; on write, map boolean-like input to `0` or `1` and set the `activation` flag accordingly.
Input: `value=None` to read; boolean or string ("ON"/"OFF") to write.
Return: integer state on read; integer `0` on write.

LFC_RFAMP_V RF amplifier supply voltage.
Function: on read, return the measured voltage of the RF amplifier supply (channel 1); on write, set the voltage setpoint to a default or user-specified value.
Input: `value=None` or numeric volts.
Return: float voltage on read; `0` on write.

LFC_RFAMP_I RF amplifier supply current.
Function: same pattern as `LFC_RFAMP_V` but for current (for example `Iout1` and `Iset1`).
Input: `value=None` or numeric amps.
Return: float current on read; `0` on write.

LFC_RFAMP_ONOFF RF amplifier on/off state.
Function: on read, return the enable state of the RF amplifier supply; on write, map boolean-like input to `0` or `1` and set `activation1`.
Input: `value=None` to read; boolean to write.
Return: integer state on read; `0` on write.

LFC_RFAMP_DEFAULT Apply default RF amplifier settings.
Function: when called with `value=1`, set a standard operating point (for example 30 V and 4.2 A).
Input: `value=1` to apply defaults.
Return: integer `0`.

LFC_RFOSCI_MONITOR RF oscillator safety monitor.
Function: on read, check whether the RF oscillator supply is running and, if so, compare voltage and current against expected thresholds. If limits are exceeded, this keyword may call `LFC_CLOSE_ALL` and send an email.
Input: `value=None` (read-only).
Return: integer status flag.

LFC_RFAMP_MONITOR RF amplifier safety monitor.
Function: same concept as `LFC_RFOSCI_MONITOR` but for the RF amplifier supply.
Input: `value=None` (read-only).
Return: integer status flag.

LFC_PENDULEM_FREQ_MONITOR Repetition-frequency monitor using the Pendulum counter.
Function: on read, measure the RF frequency and check that it remains near its nominal value (for example 16 GHz). If the deviation is too large, the keyword can initiate a shutdown.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Input: `value=None` (read-only).
Return: integer status flag.

4.3.3.3 EDFA chain and Pritel amplifier

LFC_EDFA27_P Amonics 27 dBm EDFA output power.
Function: on read, return the measured channel 1 output power (for example in mW). On write, set the automatic power control level or current, or apply a default setting when `value='default'`.
Input: `value=None` to read; `'default'` or numeric value to write.
Return: float output power on read; float or integer on write.

LFC_EDFA27_ONOFF 27 dBm EDFA enable state.
Function: on read, return whether channel 1 and the main output are enabled; on write, map boolean-like input to on/off and program `accCh1Status` and `activation`.
Input: `value=None` to read; boolean to write.
Return: integer state on read; `0` on write.

LFC_EDFA23_P Amonics 23 dBm EDFA output power.
Function: same pattern as `LFC_EDFA27_P` but for the lower power pre-amplifier.
Input: `value=None`, `'default'`, or numeric power/current.
Return: float on read; `0` or float on write.

LFC_EDFA23_ONOFF 23 dBm EDFA enable state.
Function: same structure as `LFC_EDFA27_ONOFF` for the preamp.
Input: `value=None` to read; boolean to write.
Return: integer on read; `0` on write.

LFC_EDFA23_P_DEFAULT Apply default 23 dBm EDFA power.
Function: when called with `value=1`, set a standard current or power (for example around 80 mA).
Input: `value=1`.
Return: integer `0`.

LFC_PTAMP_PRE_P Pritel preamplifier current setpoint.
Function: on read, return the current setting (mA); on write, set the preamp current to the requested value.
Input: `value=None` to read; numeric current (mA) to write.
Return: numeric or string on read; `0` on write.

LFC_PTAMP_I Pritel main amplifier current.
Function: on write, set `pwrAmp` to the requested current (up to the hard limit of about 4.2 A) and return the updated current. Reading is typically for debugging.
Input: numeric current value.
Return: string or numeric current after write; `0` on success.

LFC_PTAMP_I_DEFAULT Apply default Pritel main current.

Function: when called with `value=1`, set a standard operating current (for example around 3.9 A).
Input: `value=1`.
Return: integer `0`.

LFC_PTAMP_ONOFF Pritel amplifier enable state.
Function: on read, return the activation state mapped from internal ON/OFF strings; on write, map boolean-like input to `0` or `1` and set `activation`.
Input: `value=None` to read; boolean to write.
Return: integer on read; `0` on write.

LFC_CHECK_FULLCOMB Check for a full-comb configuration.
Function: on read, infer whether the system is in a "full comb" state based on Pritel status and related keywords.
Input: `value=None`.
Return: integer flag.

4.3.3.4 RIO seed laser, waveshaper, and intensity lock

LFC_RIO_T RIO laser TEC temperature setpoint.
Function: on read, return the current TEC setpoint; on write, apply a new setpoint on the RIO controller.
Input: `value=None` to read; numeric temperature (degrees Celsius) to write.
Return: float on read; `0` on write.

LFC_RIO_I RIO laser diode current.
Function: on read, return the diode current (mA); on write, set the current to the requested value and return the updated reading.
Input: `value=None` to read; numeric current (mA) to write.
Return: float current on read; float on write.

LFC_WSP_PHASE Waveshaper dispersion configuration.
Function: on write, interpret the input as a dispersion parameter and call waveshaper methods (for example `set3rdDisper()` and `setBandPass()`) and then upload the profile. Read is not normally used.
Input: numeric dispersion value; `value=None` for no-op.
Return: `0` on write; `None` on read.

LFC_IM_BIAS Minicomb intensity servo bias.
Function: on read, return the SIM960 output voltage used to bias the intensity modulator; on write, set the manual output voltage.
Input: `value=None` to read; numeric voltage to write.
Return: float voltage on read; `0` on write.

LFC_IM_RF_ATT RF attenuation control for the intensity modulator.
Function: on read, return the RF supply output voltage that controls the RF attenuation; on write, set the corresponding setpoint to a default or user-provided value.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Input: `value=None` to read; `'default'` or numeric voltage to write.
Return: float voltage on read or after write.

LFC_IM_LOCK_MODE Intensity-lock servo mode (manual or PID).
Function: on read, map the SIM960 `output_mode` ("man"/"pid") to `0` or `1`; on write, map integer input back to the mode string and set the servo.
Input: `value=None` to read; integer `0` (manual) or `1` (PID) to write.
Return: integer mode on read; `0` on write.

4.3.3.5 Shutters, VOAs, and global safety

LFC_PTAMP_LATCH Status of the amplifier safety relay latch.
Function: on read, connect to the Arduino safety relay if needed, call `get_relay_status()`, and map the returned message to an integer code indicating whether it is safe to enable the amplifier. On write with `value=1`, reset the latch using `reset_relay_latch()`.
Input: `value=None` to read; `value=1` to reset.
Return: integer status on read; `0` on write.

LFC_YJ_SHUTTER YJ-path shutter control.
Function: on read, query `get_YJ_info()` and map its message to `0` (blocked) or `1` (passing); on write, call `shut_YJ()` or `pass_YJ()` depending on boolean input.
Input: `value=None` for read; boolean to write.
Return: integer state on read; `0` on write.

LFC_HK_SHUTTER HK beam shutter control.
Function: on read, return the hardware shutter status (`get_status()`); on write, set the shutter state using `set_status()`.
Input: `value=None` to read; boolean or "open"/"close" to write.
Return: integer state on read; `0` on write.

LFC_VOA1550_ATTEN Optical attenuator for the 1550 nm path.
Function: on read, return the current attenuation in dB (rounded); on write, set `atten_db` to either a default or user-provided value.
Input: `value=None` to read; numeric dB value or `'default'` to write.
Return: float attenuation on read; `0` on write.

LFC_VOA1310_ATTEN Optical attenuator for the 1310 nm path.
Function: same pattern as `LFC_VOA1550_ATTEN` but for the 1310 nm VOAs.
Input: `value=None` to read; numeric or `'default'` to write.
Return: float on read; `0` on write.

LFC_VOA2000_ATTEN Optical attenuator for the 2000 nm path.
Function: same pattern as `LFC_VOA1550_ATTEN` but for the 2000 nm VOAs.
Input: `value=None` to read; numeric or `'default'` to write.
Return: float on read; `0` on write.

LFC_CLOSE_ALL Global emergency shutdown.

Function: when called with `value=1`, send a warning email and shut down all high-power devices (EDFAs, RF supplies, clarity, Pritel amplifier, and others) in a safe order. When called with `value=None`, do nothing.
Input: `value=None` for no action; `value=1` to trigger shutdown.
Return: none (implicit).

All remaining keyword methods in `KeckLFC.py` follow the same pattern: reading with `value=None` returns the current state; writing with `value` not `None` applies a change to the hardware and returns a simple status code. For exact implementation details, see the source of `KeckLFC.py` together with the individual device modules in the `Hardware` package.

4.4 KTL functions and keywords (GUI level and basic level concept)

This section summarises the KTL interface used by the comb control software. For each function or keyword we list the GUI layout (if any), input and output values, data type, passcode requirement, a short description, and whether the keyword is polled periodically.

4.4.1 GUI-level KTL functions

`LFC_SET_STANDBY`
Button / layout: Single button labelled `LFC_SET_STANDBY`.
Input: None.
Output: Optional status value (number, string, or none), used internally for logging.
Type: Mixed (no input; output is implementation dependent).
Passcode: No.
Description: High-level command that drives the comb into the *STANDBY* state by calling a predefined sequence of low-level keywords (power supplies, EDFAs, shutters, etc.). The operator normally uses only this button, not the individual device keywords.
Periodic: No (one-shot on button press).

`LFC_SET_FULL_COMB`
Button / layout: Single button labelled `LFC_SET_FULL_COMB`.
Input: None.
Output: Optional status value (number, string, or none).
Type: Mixed.
Passcode: No.
Description: High-level command that starts the comb and brings the system into *FULL_COMB* mode. Internally it turns on the lasers, drives the RF chain, enables EDFAs, sets reference states, and engages locking loops according to the defined recipe.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Periodic: No.

`LFC_SET_OFF`

Button / layout: Single button labelled `LFC_SET_OFF`.
Input: None.
Output: Optional status value (number, string, or none).
Type: Mixed.
Passcode: Yes (GUI passcode is required).
Description: High-level command that shuts down the comb in a safe, predefined order. It turns off RF amplifiers, EDFAs, the Pritel output, shutters, etc., in a sequence that avoids damage and leaves the system in a safe state.
Periodic: No.

`LFC_CHECK_STATUS`

Button / layout: No dedicated button. Results are displayed as a list of device names with green / red indicator lights in the GUI status panel.
Input: None.
Output: Integer equal to the product of a set of prime numbers.
Type: Integer.
Passcode: No.
Description: Encodes the health of several subsystems as a product of prime factors. Each subsystem corresponds to a prime number. When all devices are healthy the return value is the product of all primes; if one device fails its prime factor is omitted, and the missing factor can be found by checking divisibility of the product. The GUI uses this to drive the green / red status indicators for individual devices.
Periodic: Yes, approximately every 30 minutes.

`LFC_HK_SHUTTER`

Button / layout: Two buttons labelled `HK PASS` and `HK SHUT`. No text input box.
Input: Integer 0 or 1 (set by button press).
Output: Integer 0 or 1 (current shutter state).
Type: Integer / boolean.
Passcode: No.
Description: HK-band shutter control. Pressing `HK PASS` calls the keyword with value 1 (open shutter, pass HK comb). Pressing `HK SHUT` calls the keyword with value 0 (close shutter, block HK comb). The keyword can also be read to display the current state.
Periodic: No.

`LFC_YJ_SHUTTER`

Button / layout: Two buttons labelled `YJ PASS` and `YJ SHUT`. No text input box.
Input: Integer 0 or 1.
Output: Integer 0 or 1 (current shutter state).
Type: Integer / boolean.

Passcode: No.
Description: YJ-band shutter control. Pressing `YJ PASS` calls the keyword with value 1 (open shutter, pass YJ comb). Pressing `YJ SHUT` calls the keyword with value 0 (close shutter, block YJ comb).
Periodic: No.

`LFC_PTAMP_I`

Button / layout: Label "set amplifier current" with one numeric input box and two small buttons `Get` and `Set`.
Input: Floating-point current setpoint between about 3.7 and 4.2 A (entered by the user) when using `Set`; no input when using `Get`.
Output: When `Get` is pressed, returns the current set value. When `Set` is pressed, typically returns a confirmation code (or the updated value).
Type: Double.
Passcode: Yes (lockout; changing the current requires GUI passcode).
Description: Controls the Pritel high-power EDFA output current. This is one of the most critical safety parameters; normally operated around 3.8–4.2 A, with an absolute limit at 4.2 A for routine operation.
Periodic: No fixed polling; read on demand via the `Get` button.

`LFC_2BY2_SWITCH`

Button / layout: Two buttons labelled `HK channel` and `YJ channel`. No text input box.
Input: Integer 1 or 2 (set internally on button press).
Output: Current switch state (1 or 2).
Type: Enumerated integer.
Passcode: No.
Description: Selects which flattened comb (YJ or HK) is routed to the spectrograph. The `YJ channel` button calls the keyword with value 1; the `HK channel` button calls it with value 2.
Periodic: No.

`LFC_GET_TEMP`

Button / layout: No dedicated button; used by the status panel to show a list of temperatures.
Input: None.
Output: List of floating-point temperatures, for example `[27, 31, 32, 37, 16, 18]`. Each index corresponds to a fixed sensor position in the comb system.
Type: Array of doubles.
Passcode: No.
Description: Convenience function that aggregates several DAQ channels into a compact temperature summary for the GUI. The mapping between array index and physical location is documented in the hardware section.
Periodic: Yes, typically every 30 minutes in the background.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

4.4.2 Low-level KTL keywords (maybe no direct GUI button, depend on demand)

The following keywords are normally called by high-level sequences (`LFC_SET_*`) or by diagnostic tools rather than by the operator directly.

`LFC_RFOSCI_ONOFF`
Function: Turn the RF oscillator DC power supply on or off.
Input: Integer 0 (off) or 1 (on).
Output: Current on/off state.
Type: Boolean (integer).
Passcode: Yes (lockout keyword).
Periodic: Not polled; used by high-level sequences.

`LFC_RFAMP_ONOFF`
Function: Turn the RF amplifier DC power supply on or off.
Input: Integer 0 (off) or 1 (on).
Output: Current on/off state.
Type: Boolean (integer).
Passcode: Yes.
Periodic: No.

`LFC_PTAMP_LATCH`
Function: Read or reset the Arduino latch that interlocks the Pritel amplifier.
Input: `None` to read latch status; 1 to reset the latch.
Output: Latch state (boolean / integer).
Type: Boolean.
Passcode: Yes.
Periodic: No (called when recovering from a fault).

`LFC_VOA1550_ATTEN`
Function: Set or read the attenuation of the 1550 nm VOA in the pump path.
Input: Floating-point attenuation in dB (0–60).
Output: Current attenuation in dB.
Type: Float.
Passcode: No.
Periodic: No.

`LFC_VOA1310_ATTEN`
Function: Set or read the attenuation of the 1310 nm VOA for the YJ comb path.
Input: Float (0–60 dB).
Output: Current attenuation.
Type: Float.
Passcode: No.
Periodic: No.

`LFC_VOA2000_ATTEN`
Function: Set or read the attenuation of the 2000 nm VOA for the HK comb path.

Input: Float (0–60 dB).
Output: Current attenuation.
Type: Float.
Passcode: No.
Periodic: No.

`LFC_IM_LOCK_MODE`
Function: Switch the mini-comb intensity-lock loop between manual and PID modes.
Input: 0 for manual mode, 1 for PID mode.
Output: Current mode (0 or 1).
Type: Boolean.
Passcode: Yes.
Periodic: No.

`LFC_EDFA27_INPUT_POWER_MONITOR`
Function: Monitor the input power of the 27 dBm Amonics EDFA.
Input: None.
Output: Status flag (0 or 1).
Type: Boolean.
Passcode: No.
Periodic: Yes, typically every 120 s.

`LFC_EDFA23_INPUT_POWER_MONITOR`
Function: Monitor the input power of the 23 dBm Amonics EDFA.
Input: None.
Output: Status flag (0 or 1).
Type: Boolean.
Passcode: No.
Periodic: Yes, about 120 s.

`LFC_CLARITY_ONOFF`
Function: Control or read the Clarity reference laser state.
Input: `None` to read status (0: down, 1: just on, 2: locking, 3: locked); 1 to turn on; 0 to turn off.
Output: Status code 0–3.
Type: Integer.
Passcode: Yes.
Periodic: No.

`LFC_MINICOMB_AUTO_SETUP`
Function: Automatic mini-comb setup, including IM bias lock.
Input: 1 to trigger auto-setup; `None` to query status.
Output: Implementation-dependent status code; may trigger email on failure.
Type: Integer.
Passcode: Yes.
Periodic: No (one-shot).

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

`LFC_PENDULEM_FREQ_MONITOR`

Function: RF repetition-frequency monitor using the Pendulum counter.
Input: None (or simple enable flag).
Output: Frequency reading and/or status flag.
Type: Float / boolean (TBD).
Passcode: No.
Periodic: TBD (used for diagnostics).

`LFC_TEMP_MONITOR`

Function: Autonomous temperature safety monitor.
Input: None.
Output: Boolean status flag.
Type: Boolean.
Passcode: No.
Description: If any monitored temperature exceeds its threshold, this keyword triggers `LFC_CLOSE_ALL` and `LFC_EMAIL` to safely shut down the system and send an alarm.
Periodic: Yes (internal watchdog).

`LFC_RFOSCI_MONITOR`

Function: RF oscillator health monitor.
Input: None.
Output: 0 for normal, 1 for fault.
Type: Boolean.
Passcode: No.
Periodic: Yes, about every 20 s.

`LFC_RFAMP_MONITOR`

Function: RF amplifier health monitor.
Input: None.
Output: 0 for normal, 1 for fault.
Type: Boolean.
Passcode: No.
Periodic: Yes, about every 20 s.

`LFC_CLOSE_ALL`

Function: Global emergency shutdown of all high-power devices.
Input: 1 to execute shutdown; `None` to query status.
Output: Status / confirmation.
Type: Boolean.
Passcode: Yes.
Periodic: No.

`LFC_EMAIL`

Function: Internal helper keyword for sending alarm or status emails.
Input: Implementation-dependent message payload.
Output: None.

Type: Internal.
Passcode: Yes.
Periodic: No.

`LFC_TEMP_TEST1`
Function: Read all temperatures from USB-DAQ board 1 (optical table sensors).
Input: None.
Output: Float array of temperatures.
Type: Float array.
Passcode: No.
Periodic: Optional, typically 20 s when enabled.

`LFC_TEMP_TEST2`
Function: Read all temperatures from USB-DAQ board 0 (rack sensors).
Input: None.
Output: Float array.
Type: Float array.
Passcode: No.
Periodic: Optional, typically 20 s.

`LFC_T_GLY_RACK_IN`
Function: Rack glycol inlet temperature.
Input: None.
Output: Temperature (float).
Type: Float.
Passcode: No.
Periodic: About 20 s.

`LFC_T_GLY_RACK_OUT`
Function: Rack glycol outlet temperature.
Input: None.
Output: Float.
Type: Float.
Passcode: No.
Periodic: About 20 s.

`LFC_T_EOCB_IN`
Function: EOCB inlet temperature.
Input: None.
Output: Float.
Type: Float.
Passcode: No.
Periodic: About 20 s.

`LFC_T_EOCB_OUT`
Function: EOCB outlet temperature.
Input: None.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Output: Float.
Type: Float.
Passcode: No.
Periodic: About 20 s.

`LFC_T_RACK_TOP`
Function: Temperature at the top of the electronics rack.
Input: None.
Output: Float.
Type: Float.
Passcode: No.
Periodic: About 20 s.

`LFC_T_RACK_MID`
Function: Temperature at the middle of the rack.
Input: None.
Output: Float.
Type: Float.
Passcode: No.
Periodic: About 20 s.

`LFC_T_RACK_BOT`
Function: Temperature at the bottom of the rack.
Input: None.
Output: Float.
Type: Float.
Passcode: No.
Periodic: About 20 s.

`LFC_RIO_T`
Function: RIO pump laser temperature setpoint.
Input: Float in the approximate range 10–25 C.
Output: Current temperature setpoint.
Type: Float.
Passcode: Yes (lockout).
Periodic: No.

`LFC_RIO_I`
Function: RIO pump laser current.
Input: Float in the range 145–155 mA.
Output: Current setpoint.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_EDFA27_P`
Function: Output power setpoint and monitor for the 27 dBm Amonics EDFA.
Input: Float (mW) or the string `'default'` to select APC mode at 450 mW.

Output: Monitored output power.
Type: Float.
Passcode: Yes.
Periodic: No (read as needed).

`LFC_EDFA27_P_DEFAULT`
Function: Helper keyword for EDFA27 default mode.
Input: `None` to read power; non-`None` value to set APC mode at 450 mW.
Output: Power reading or confirmation.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_EDFA27_ONOFF`
Function: Emission on/off state of the 27 dBm EDFA.
Input: 0 or 1.
Output: Current emission state.
Type: Boolean.
Passcode: Yes.
Periodic: No.

`LFC_EDFA27_AUTO_ON`
Function: Automatic turn-on sequence for EDFA27.
Input: `None` to read power; non-`None` (e.g. 1) to set APC mode and turn on.
Output: Power reading / status.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_EDFA13_P`
Function: Output power setpoint for the small (20 mW) EDFA (channel 2).
Input: Float (mW).
Output: Monitored power.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_EDFA13_ONOFF`
Function: Emission on/off state of the 20 mW EDFA.
Input: 0 or 1.
Output: Current state.
Type: Boolean.
Passcode: Yes.
Periodic: No.

`LFC_EDFA23_P`
Function: Output power setpoint for the 23 dBm Amonics EDFA.
Input: Float (mW) or the string `'default'` to select ACC mode at 80 mA.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Output: Monitored output power.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_EDFA23_P_DEFAULT`
Function: Helper keyword for EDFA23 default mode.
Input: `None` to read current power; non-`None` to set ACC mode at 80 mA.
Output: Power / status.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_EDFA23_ONOFF`
Function: Emission on/off state of the 23 dBm EDFA.
Input: 0 or 1.
Output: Current state.
Type: Boolean.
Passcode: Yes.
Periodic: No.

`LFC_EDFA23_AUTO_ON`
Function: Automatic turn-on sequence for EDFA23.
Input: `None` to read emission status; non-`None` to set ACC mode (80 mW equivalent) and turn on.
Output: Status / power.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_RFAMP_I`
Function: RF amplifier current setpoint.
Input: Float (A), typical operating value around 4.4 A.
Output: Current setpoint.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_RFAMP_V`
Function: RF amplifier voltage setpoint.
Input: Float, normally fixed at 30 V (no ramping allowed).
Output: Voltage setpoint.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_RFAMP_DEFAULT`
Function: Read or set default RF amplifier voltage and current.

Input: `None` to read; non-`None` to set $V = 30$ V, $I = 5$ A.
Output: V/I pair or confirmation.
Type: Internal.
Passcode: Yes.
Periodic: No.

`LFC_RFOSCI_I`
Function: RF oscillator supply current.
Input: Float (A), typical 0.41 A (up to ∼0.62 A while warming).
Output: Current setpoint.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_RFOSCI_V`
Function: RF oscillator supply voltage.
Input: Float, normally 15 V.
Output: Voltage setpoint.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_RFOSCI_DEFAULT`
Function: Read or set default RF oscillator supply values.
Input: `None` to read; non-`None` to set $V = 15$ V, $I = 3$ A.
Output: V/I pair or confirmation.
Type: Internal.
Passcode: Yes.
Periodic: No.

`LFC_IM_BIAS`
Function: IM bias offset voltage for the mini-comb amplitude lock.
Input: Float in the range −3 V to +3 V.
Output: Current bias voltage.
Type: Float.
Passcode: Yes.
Periodic: No (normally controlled by the servo).

`LFC_IM_RF_ATT`
Function: RF voltage-controlled attenuation setting for the mini-comb drive.
Input: Float in the range 0–10 V (default around 0.72 V).
Output: Current attenuation voltage.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_WSP_PHASE`
Function: Waveshaper dispersion compensation (d_2).

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Input: Float giving the d_2 parameter in ps/nm.
Output: Current d_2 setting.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_WSP_ATTEN`
Function: Full attenuation profile of the waveshaper.
Input: Two-by-N float array (wavelength and attenuation).
Output: Current attenuation profile.
Type: Float array.
Passcode: Yes.
Periodic: No.

`LFC_PTAMP_PRE_P`
Function: Pritel pre-amplifier current / output.
Input: Float (mA), default around 600 mA.
Output: Monitored current.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_PTAMP_PRE_P_DEFAULT`
Function: Read or set default preamp current.
Input: `None` to read; non-`None` to set to 600 mA.
Output: Default current or confirmation.
Type: Internal.
Passcode: Yes.
Periodic: No.

`LFC_PTAMP_OUT`
Function: Pritel main amplifier output power (read-only).
Input: None.
Output: Output power in W (0–4 W).
Type: Float.
Passcode: Yes.
Periodic: Optional, about 120 s when enabled.

`LFC_PTAMP_I`
Function: Pritel main amplifier current setpoint.
Input: Float in the range 0–4.2 A (typical 3.8–4.2 A).
Output: Current setpoint.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_PTAMP_I_DEFAULT`
Function: Default Pritel main amplifier current.

Input: `None` to read; non-`None` to set to 4.1 A.
Output: Default current or confirmation.
Type: Internal.
Passcode: Yes.
Periodic: No.

`LFC_PTAMP_ONOFF`
Function: Pritel high-power EDFA emission on / standby.
Input: 0 or 1.
Output: Current state.
Type: Boolean.
Passcode: Yes.
Periodic: No.

`LFC_WGD_T`
Function: Waveguide TEC control temperature.
Input: Float (approximately 22.5–23.5 C).
Output: Setpoint.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_PPLN_T`
Function: PPLN TEC control temperature.
Input: Float (typical operating point around 37 C).
Output: Setpoint.
Type: Float.
Passcode: Yes.
Periodic: No.

`LFC_HK_ATT`
Function: HK-band comb attenuation via VOA2000.
Input: Float attenuation in dB (0.01–60).
Output: Current attenuation.
Type: Float.
Passcode: No.
Periodic: No.

`LFC_FLT_ATT`
Function: Neutral-density filter slider position in the spectral flattener (future use).
Input: Integer position index (TBD).
Output: Current position.
Type: Integer.
Passcode: No.
Periodic: No.

`LFC_PMP_ATT`

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Function: Pump attenuation via the 1550 nm VOA (duplicate view of `LFC_VOA1550_ATTEN`).
Input: Float in dB (0.01–60).
Output: Current attenuation.
Type: Float.
Passcode: No.
Periodic: No.

`LFC_YJ_ONOFF`
Function: Open or close the shutter of the flattened YJ comb output (alias for `LFC_YJ_SHUTTER`).
Input: Boolean 0 or 1.
Output: Current state.
Type: Boolean.
Passcode: No.
Periodic: No.

`LFC_HK_ONOFF`
Function: Open or close the shutter after the HK flattener (alias for `LFC_HK_SHUTTER`).
Input: Boolean 0 or 1.
Output: Current state.
Type: Boolean.
Passcode: No.
Periodic: No.

`LFC_YJ_ATT`
Function: YJ-band comb attenuation (VOA1310) for the KPF comb input.
Input: Float in dB (0.01–60).
Output: Current attenuation.
Type: Float.
Passcode: No.
Periodic: No.

`LFC_YJ_HK`
Function: Select whether YJ or HK comb is routed to the spectrograph (high-level alias for `LFC_2BY2_SWITCH`).
Input: String or integer: 1 for YJ, 2 for HK.
Output: Current selection.
Type: String / enumerated.
Passcode: No.
Periodic: No.

4.5 LFC `.sin` files and keyword definition

In addition to the Python code in `KeckLFC.py`, the comb control software relies on two text files with the `.sin` extension:

- ◇ **LFC.xml.sin**: XML keyword definition file used by both the ICE server and the KTL dispatcher.
- ◇ **combd.sin**: dispatcher source file that defines the KTL-side keyword objects and their behaviour.

Together these files specify the full interface between the KTL layer and the KeckLFC Python class. In practice, LFC.xml.sin is the authoritative list of keywords and their properties, while combd.sin provides the KTL implementation (polling, caching, and ICE calls).

4.5.1 LFC.xml.sin: XML keyword list

LFC.xml.sin is an XML file that defines each LFC keyword in a machine-readable way. Both the ICE server (server.py + KeckLFC.py) and the KTL dispatcher (combd.sin) read this file at startup to build a consistent keyword list.

Conceptually, each keyword in LFC.xml.sin plays the role of a function:

- ◇ the *keyword name* is the function name (for example LFC_PTAMP_I);
- ◇ the *keyword value* is the function argument when the keyword is written, and the return value when the keyword is read;
- ◇ additional XML attributes specify the type, units, limits, default value, and polling interval of the keyword.

For each keyword, LFC.xml.sin stores a set of parameters. The exact XML syntax is defined by the KTL keyword schema, but for the LFC system the following logical fields are used.

Name

The unique keyword identifier, for example LFC_PTAMP_I, LFC_RFOSCI_ONOFF, LFC_WGD_T, LFC_YJ_HK. In the Python code the corresponding function in KeckLFC must have the same name:

```
def LFC_PTAMP_I(self, value=None):
    ...
```

Type

The data type of the keyword value as seen by KTL. Typical types in LFC.xml.sin are:

- ◇ Integer: integer-valued keywords such as on/off flags or enumerated modes (for example LFC_RFOSCI_ONOFF);
- ◇ Double: floating-point values, such as currents, voltages, temperatures, or attenuation in dB (for example LFC_PTAMP_I, LFC_PPLN_T, LFC_PMP_ATT);
- ◇ Boolean: logical flags (for example LFC_TEMP_MONITOR, LFC_EDFA27_ONOFF);
- ◇ String: textual values or small enumerations (for example LFC_YJ_HK);
- ◇ DoubleArray: array-valued keywords such as the temperature vector returned by LFC_TEMP_TEST1 or the waveshaper attenuation profile LFC_WSP_ATTEN.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

The type in LFC.xml.sin must be consistent with the Python conversion performed by convert_type() in KeckLFC.py.

Read / write mode

This parameter declares whether the keyword is readable, writable, or both. In the LFC file it corresponds to the "R/W" column in the design spreadsheet:

- **R**: read-only keywords such as LFC_PTAMP_OUT, LFC_EDFA27_INPUT_POWER_MONITOR, LFC_T_RACK_TOP;
- **W**: write-only command keywords such as LFC_CLOSE_ALL, LFC_MINICOMB_AUTO_SETUP;
- **R/W**: full read–write keywords such as LFC_PTAMP_I, LFC_RFAMP_I, LFC_PPLN_T, LFC_YJ_SHUTTER.

The dispatcher uses this information to decide whether a keyword can be modified from the KTL side.

Units

Physical units of the keyword value, where applicable. Examples include:

- A for currents (LFC_PTAMP_I, LFC_RFAMP_I, LFC_RFOSCI_I);
- V for voltages (LFC_RFAMP_V, LFC_RFOSCI_V);
- mW for EDFA output power (LFC_EDFA27_P, LFC_EDFA23_P);
- dB for attenuations (LFC_PMP_ATT, LFC_HK_ATT, LFC_YJ_ATT);
- degrees Celsius for temperatures (LFC_PPLN_T, LFC_WGD_T, LFC_T_RACK_MID).

The units field is for documentation and client-side display only; the actual value is always passed as a string.

Limits (min / max)

Optional lower and upper bounds for the keyword. These are used for sanity checks and for defining safe operating ranges. Examples:

- LFC_PTAMP_I: minimum 0 A, maximum 4.2 A;
- LFC_RIO_I: minimum 145 mA, maximum 155 mA;
- LFC_PMP_ATT: minimum 0.01 dB, maximum 60 dB;
- LFC_WGD_T: typical range around 22.5–23.5 °C.

On the KTL side these limits may be used to reject out-of-range writes.

Default value

Recommended initial value for the keyword at dispatcher startup. For example:

- LFC_PTAMP_I_DEFAULT: initial Pritel current (for example 4.1 A);
- LFC_PTAMP_PRE_P_DEFAULT: preamp current default (600 mA);
- LFC_PMP_ATT: default 60 dB pump attenuation.

The dispatcher and/or the ICE server can use these defaults when initialising hardware.

Polling period

The nominal interval (in seconds) at which the dispatcher should poll the keyword

and update the cached value. This is reflected in the design table as "time period (unit: s)". Examples:

- 120 s for relatively slow parameters such as EDFA powers (`LFC_EDFA27_P`, `LFC_PTAMP_OUT`) and RF supply settings;
- 20 s for temperature and RF status keywords (`LFC_TEMP_TEST1`, `LFC_T_RACK_TOP`, `LFC_RFOSCI_MONITOR`);
- longer intervals (for example 30 min) for aggregated status checks such as `LFC_CHECK_STATUS` and `LFC_GET_TEMP`.

Lockout / passcode flag

Some keywords are marked as "lockout" in the design table. These require an additional passcode on the GUI side before write access is allowed. Typical lockout keywords are those that control high-power hardware or safety-critical parameters, for example:

- `LFC_PTAMP_I`, `LFC_PTAMP_ONOFF`;
- `LFC_RFAMP_I`, `LFC_RFAMP_V`, `LFC_RFAMP_ONOFF`;
- `LFC_RIO_T`, `LFC_RIO_I`;
- `LFC_IM_BIAS` and `LFC_IM_RF_ATT`.

The XML file encodes this flag so that both dispatcher and GUI can enforce the lockout behaviour.

Description text

Free-form text that explains the purpose of the keyword. This corresponds to the "Description" and "Additional Notes" columns in the design spreadsheet. For example:

- `LFC_PTAMP_I`: "High-power EDFA output current set; default 3.8 A; limit 4.2 A";
- `LFC_TEMP_MONITOR`: "Automatic temperature safety monitor; if any channel exceeds threshold, trigger global shutdown and email".

The description is not interpreted by the software but is essential for documentation and for tools that auto-generate help text.

In summary, `LFC.xml.sin` is the single source of truth for all keyword names, types, limits, defaults, and polling periods. Any change to the KTL interface must be reflected here and then deployed to both the Windows LFC laptop and the observatory servers as described in the developer notes.

4.5.2 `combd.sin`: dispatcher source

The dispatcher file `combd.sin` implements the KTL-side behaviour of the keywords defined in `LFC.xml.sin`. It runs on the `irastrocomb` server and plays the role of an ICE client.

Conceptually, `combd.sin` performs the following steps:

(1) At startup, it reads `LFC.xml.sin` and creates a KTL keyword object for each

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

entry (for example a subclass of `DFW.Keyword.Integer`, `DFW.Keyword.Double`, `DFW.Keyword.String`, etc.).

(2) For each keyword it implements the `read()`, `write()`, `postwrite()`, and `update()` methods so that:

- ◇ `read()` calls the ICE server, which in turn calls the corresponding method in `KeckLFC` with `value=None` and returns the result;
- ◇ `write()` sends the new value to the ICE server, which calls `KeckLFC.KEYWORD(value)`; if the write returns 0, the dispatcher updates its cached value;
- ◇ `update()` is used by the polling thread to refresh values at the intervals specified by the "time period" parameter in `LFC.xml.sin`.

(3) The dispatcher exposes the keywords to end users via the KTL `show` and `modify` commands with service name `comb`.

Because both the dispatcher and the ICE server read the same `LFC.xml.sin` file, they share a consistent keyword list. When a new keyword is added, the required steps are:

(1) Add the keyword to `LFC.xml.sin` with correct type, limits, default value, polling period, and description.

(2) Implement the corresponding method in `KeckLFC.py`:

```
def NEW_KEYWORD(self, value=None):
    if value is None:
        # periodic read path
        ...
        return current_value
    else:
        # write path from KTL
        ...
        return 0   # on success
```

(3) Update and install `combd.sin` on `irastrocombbuild`, run `make install` and `kdeploy`, then restart the `comb` dispatcher on `irastrocomb`.

After these steps, the new keyword becomes available to the GUI via the `comb` KTL service and can be integrated into the LFC STATUS and CONTROL interface.

CHAPTER 5
Instrument Description

This chapter summarizes the optical, mechanical, and electrical layout of the NIR LFC.

5.1 Facility

The NIR Astrocomb (or NIR LFC) assembly is located in the basement of the Keck II telescope and occupies an optical bench and one electronics rack. The interfaces (optical, mechanical, electrical, thermal, and communications) between the NIR LFC and the facility are detailed in the NIR LFC Interface Control Document (ICD). The astrocomb assembly is composed of an instrument rack (24"W x 38" D x 81"H), two 18" x 24" glycol-cooled optical breadboards that are fixed to a 3' wide optical bench and housed inside a 24" x 36" x 12" optical enclosure, and a self-contained spectral flattener unit that rests on the optical bench adjacent to the breadboard enclosure. Adjacent to the electronics rack on the side opposite the optical bench is a ceiling-mounted facility interconnect panel that provides clean power, glycol coolant, facility air, and ethernet. Underneath the interface panel on the floor are two uninterupted power source (UPS) units, 15 Amp each, to provide smooth power handling in the event of a facility power outage. The electronics rack has an interconnect panel on the side for optical, power, and data connections with the optical breadboard assemblies on the optical bench. The side of the instrument rack abuts the 3' end of the optical bench, and a 15-inch-wide interconnect panel on the side of the rack serves to connect components on the optical bench with the instruments inside the rack via optical ports (FC/APC), RF ports (SMA connectors), USB, ethernet, and electrical ports. Electrical power connections run from the top of the instrument cabinet across a ceiling-mounted cable tray to the interface panel. Glycol cooling lines run from a manifold on the back of the interface panel down the side of the optical bench, along the bottom, and into the instrument rack, as well as up the opposite side of the optical bench to the breadboards and a component cooling block on one of the breadboards. (Figure 5.2) The two optical breadboards are housed inside of an optical enclosure with a top-hinged lid to provide access to the boards. (see Figure 5.3). Adjacent to the optical enclosure on the optical bench is a spectral flattener assembly. The output from the spectral flattener is an optical fiber that carries the comb light to the observatory dome.

(TBD - the facility GPS 1 pps signal via a BNC connector) (see Figure xx).

The instrument rack houses the large electronics, namely three erbium doped fiber amplifier, the rubidium clock used for the comb frequency stabilization, the dispersion

compensation unit, power supplies to run the oscillator and RF amplifiers, servo controller units, frequency counter, computer for local system control, an optical spectrum analyzer, and web-power supplies for remote power control of each instrument.

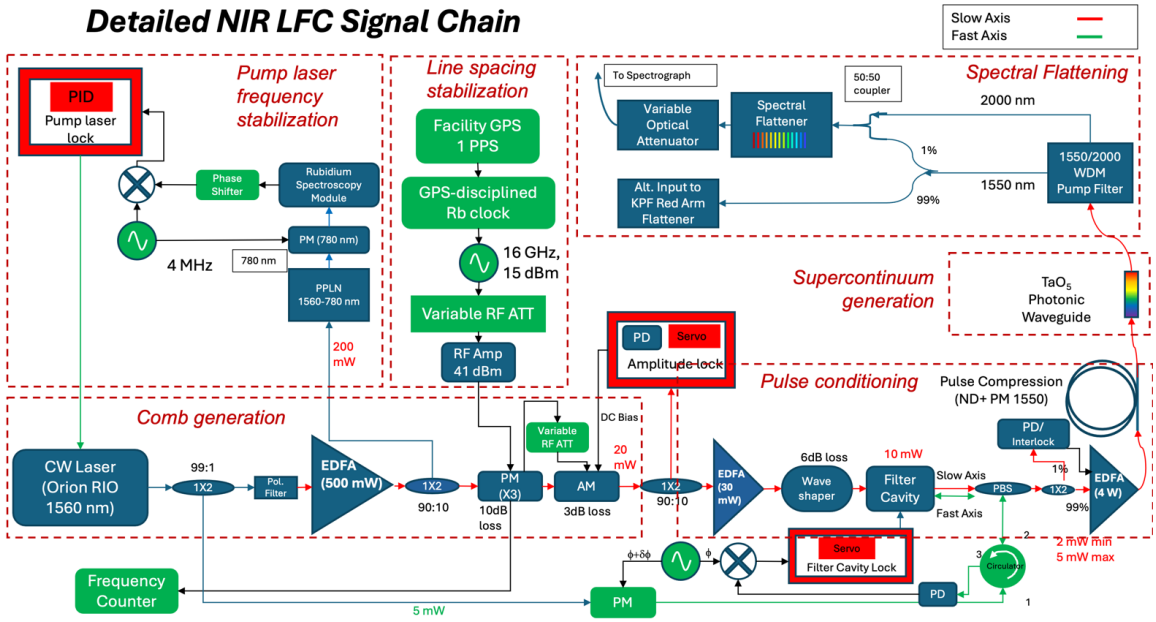


Figure 5.1. Detailed NIR LFC signal chain

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

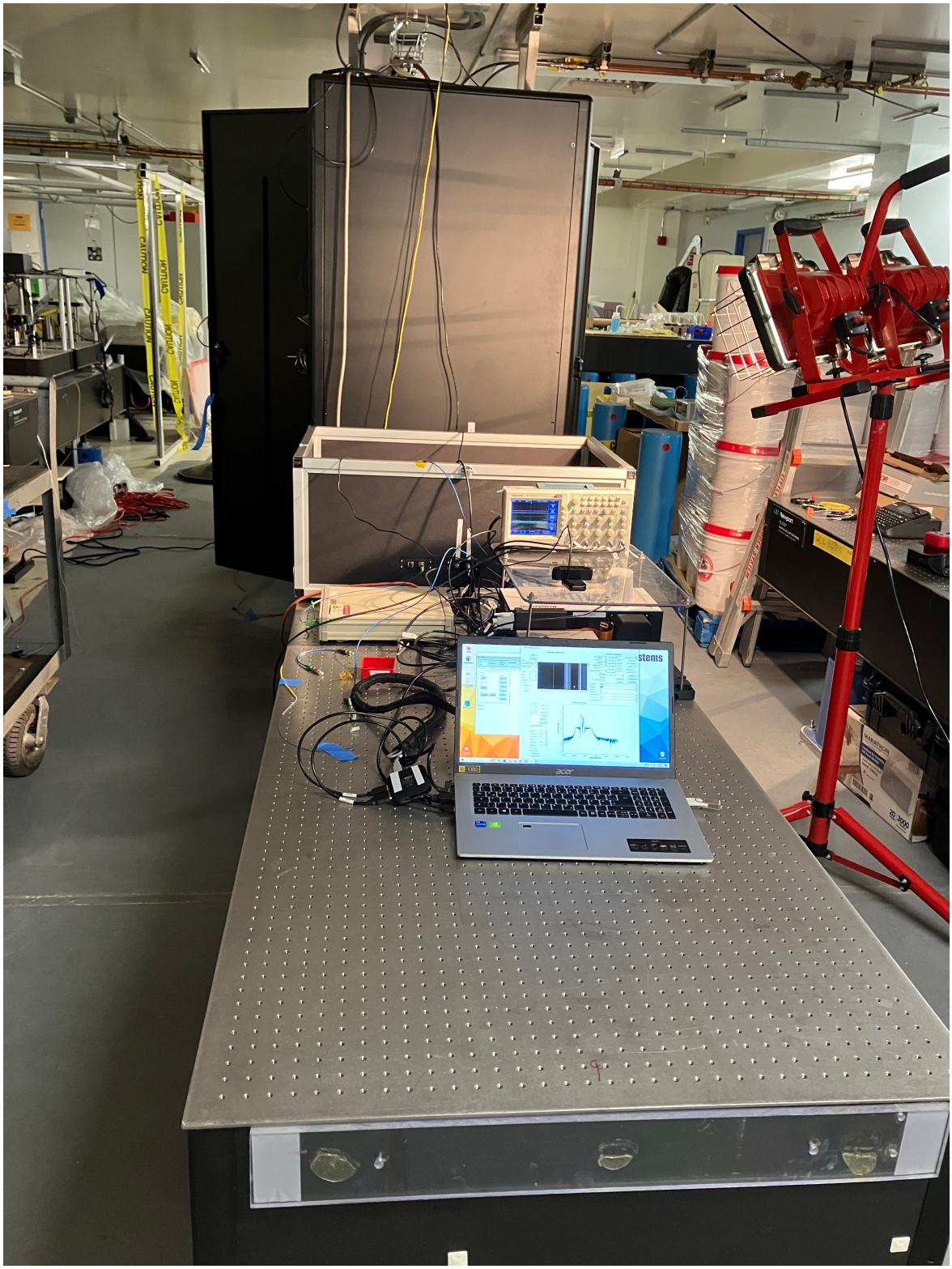


Figure 5.2. Overall view of LFC setup

5.2 Mapping of numbered components in the assembly diagram

Figure 5.3 shows the overall hardware layout of the astrocomb system. The numbers overlaid on the photograph correspond to the physical items listed in this section. For each item we summarise the model, remote-control interface, location, power, role, and a manual/specification link when available.

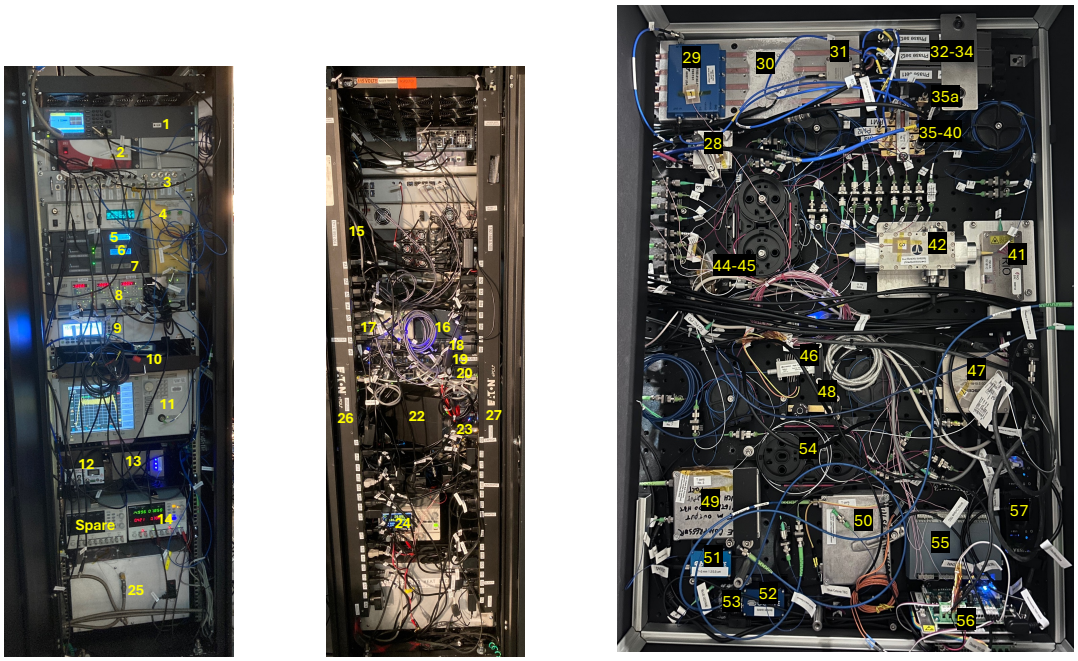


Figure 5.3. Device photo with numbered components; see text for detailed description of each item.

Items 1–10: front-end RF and minicomb diagnostics

1 – Signal generator **Model / description:** Keysight 33500B function / arbitrary waveform generator (two units in the final setup; only one is visible in Fig. 5.3).
Remote control: Yes, via LAN/USB.
Location: Electronics rack.
Power: Up to 3 A from port A5.
Role: Provides ramps and modulation tones for the rubidium lock and other test signals.
Manual / spec: https://www.keysight.com/us/en/assets/7018-03562/product-fact-sh 5991-0757.pdf

2 – Waveshaper **Model / description:** II-VI Finisar / Coherent Waveshaper WS-01000A-C-R-1-AA-01.
Remote control: Yes, via Ethernet and vendor API.
Location: Electronics rack.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Power: 110/240 VAC, 50/60 Hz, 50 VA max, via ePDU port A9 (5 A fuse).
Role: Programmable phase and amplitude shaping for the EO minicomb to generate pulses and tailor the spectrum.
Manual / spec: https://www.coherent.com/content/dam/coherent/site/en/resources/datasheet/networking/waveshaper-family-br.pdf

3 – Small component chassis **Model / description:** Custom chassis containing Arduino interlock controller, VOAs, shutters, and associated drivers.
Remote control: Yes, via Arduino and dedicated drivers.
Location: Electronics rack.
Power: Supplied from local low-voltage rails and interface panel.
Role: Implements the safety interlock chain for the Pritel EDFA and controls VOAs/shutters in the optical paths.
Manual / spec: Internal documentation and separate spreadsheet.

4 – High-power EDFA **Model / description:** Pritel LNHP-PMFA-37-IO-NMA.
Remote control: Yes, via vendor interface and KTL keywords (`LFC_PTAMP_I`, `LFC_PTAMP_ONOFF`, etc.).
Location: Electronics rack.
Power: 85–265 VAC, 47–63 Hz, < 125 W, fused at 2.5 A (slow-blow), via ePDU port A8 (2 A).
Role: Main high-power EDFA that boosts the minicomb power for supercontinuum generation. A spare unit is stored at Caltech (Watson lab).
Manual / spec: https://www.pritel.com/pdfs/LNHP-PMFA-NMA_R10.pdf

5 – 23 dB EDFA **Model / description:** Amonics AEDFA-PM-23-R-FA ("23 dB EDFA" after modulators).
Remote control: Yes, via serial (`AmonicsEDFA` driver).
Location: Electronics rack.
Power: Typical draw ∼1.5 A from ePDU port A11.
Role: Pre-amplifier to boost the EO minicomb after modulation.
Manual / spec: https://www.amonics.com/product/20

6 – 27 dB EDFA **Model / description:** Amonics AEDFA-PM-27-R-FA ("27 dB EDFA" before modulators).
Remote control: Yes, via serial.
Location: Electronics rack.
Power: Typical draw ∼2.5 A from ePDU port A6.
Role: Pump amplifier to generate enough optical power for both pump-line referencing and EO comb generation.
Manual / spec: https://www.amonics.com/product/20

7 – 13 dB EDFA (spare) **Model / description:** Amonics AEDFA-PM-13-R-FA.
Remote control: Yes, if deployed.
Location: Electronics rack, on port A7 (spare).
Power: As specified in the Amonics datasheet.
Role: Spare low-gain EDFA reserved for future upgrades.

Manual / spec: https://www.amonics.com/product/20

8 – SRS SIM900 mainframe and SIM960 PID controllers **Model / description:** Stanford Research Systems SIM900 mainframe with embedded SIM960 PID controllers and SIM928 modules.
Remote control: Yes, via GPIB (`SRS_SIM900` driver).
Location: Electronics rack.
Power: Up to 70 W total; internal rails +5 V, −5 V, ±15 V, +24 V; supplied via port A1.
Role: Provides PID control for the minicomb amplitude lock and rubidium lock loops.
Manual / spec: SIM900: https://www.thinksrs.com/products/sim900.html; SIM960: https://www.thinksrs.com/downloads/pdfs/manuals/SIM960m.pdf

9 – Frequency counter **Model / description:** Pendulum CNT-90XL microwave frequency counter.
Remote control: Yes, via LAN/GPIB (`PendulumCNT90` driver).
Location: Electronics rack.
Power: 90–265 V_{rms}, 45–440 Hz, < 40 W, via ePDU port 4B.
Role: Reports the comb repetition rate by counting the 16 GHz signal; this unit is owned by JPL.
Manual / spec: https://pendulum-instruments.com/products/microwave-counters/cnt-90xl/

10 – LFC laptop **Model / description:** Dedicated Windows laptop for Keck LFC control.
Remote control: Yes, via Ethernet (remote desktop, ICE server, KTL).
Location: Electronics rack.
Power: 19.5 V, 3.34 A (65 W) via port 17B.
Role: Hosts the `KeckLFC` Python code, ICE server, and GUIs.
Manual / spec: Laptop vendor documentation (model dependent).

Items 11–27: optical diagnostics and rack infrastructure

11 – Optical spectrum analyser (OSA) **Model / description:** Keysight / Agilent HP 86142B OSA.
Remote control: Yes, via GPIB (`Agilent_86142B` driver).
Location: Electronics rack.
Power: ∼150 VA, 100–240 VAC, via port 13A (1.25 A).
Role: Monitors EO minicomb and broadened comb spectra.
Manual / spec: https://wp.optics.arizona.edu/milster/wp-content/uploads/sites/48/2016/06/8614XBSeries.pdf

12 – Filter cavity piezo controller **Model / description:** Piezo controller for the filter cavity (model to be defined if the cavity is used).
Remote control: Local control only in current setup.
Location: Electronics rack.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Power: From a dedicated low-voltage supply.
Role: Would drive the cavity piezo when the filter cavity is enabled; currently not in use.
Manual / spec: To be added once the controller model is fixed.

13 – Rubidium spectroscopy power supply **Model / description:** Vescent Photonics D2-005.
Remote control: Local panel.
Location: Electronics rack.
Power: From port 18A.
Role: DC supply for the D2-210 rubidium spectroscopy module.
Manual / spec: https://vescent.com/products/laser-system-power-supply-d2-005

14 – DC power supply 1 **Model / description:** GW Instek GPD-4303S multi-output supply.
Remote control: Yes (`InstekGPD_4303S` driver).
Location: Electronics rack.
Power: AC input via port 19A; outputs 0–30 V / 3 A and 0–5/10 V.
Role: Supplies the RF oscillator and IM-bias voltages.
Manual / spec: https://www.gwinstek.com/en-global/products/downloadSeriesDownNew/7851/1474

15 – Chimney fan **Model / description:** Rack exhaust fan.
Remote control: None.
Location: Rack side or top.
Power: From facility interface panel.
Role: Improves rack airflow.
Manual / spec: Facility hardware documentation.

16 – Ethernet router **Model / description:** TP-Link Ethernet router.
Remote control: Yes, via web interface.
Location: Electronics rack.
Power: From port A14.
Role: Internal routing for LFC laptop, flattener laptop, and Ethernet instruments.
Manual / spec: TP-Link router documentation (model dependent).

17 – Rubidium clock **Model / description:** SRS FS725 rubidium frequency standard.
Remote control: Yes, via serial / GPIB.
Location: Electronics rack.
Power: 90–132 VAC or 175–264 VAC, 47–63 Hz, 50 W, via 5B.
Role: Provides 10 MHz reference for RF chain and frequency counter.
Manual / spec: https://www.thinksrs.com/downloads/pdfs/manuals/FS725m.pdf

18 – USB hub A (15-port) **Model / description:** 15-port powered USB hub.
Remote control: Not applicable (transparent USB hub).

Location: Electronics rack.
Power: 12 V, 7.5 A from 15A.
Role: USB aggregation for rack instruments.
Manual / spec: Vendor USB-hub datasheet.

19 – USB hub B (15-port) **Model / description:** Second 15-port powered USB hub.
Remote control: Not applicable.
Location: Electronics rack.
Power: 12 V, 7.5 A from 23B.
Role: Additional USB capacity for rack instruments.
Manual / spec: Vendor USB-hub datasheet.

20 – Ethernet switch **Model / description:** Unmanaged Ethernet switch.
Remote control: None (plug-and-play).
Location: Electronics rack.
Power: From port 21B.
Role: Local network switching between router and endpoints.
Manual / spec: Switch vendor documentation.

21 – DAQ 2 **Model / description:** Measurement Computing USB-2408 data acquisition module.
Remote control: Yes, via USB.
Location: Electronics rack.
Power: USB powered.
Role: Rack-side thermocouple and analog inputs.
Manual / spec: https://microdaq.com/mesurement-computing-usb-2408-2ao-data-acqu
php

22 – Filter cavity TEC **Model / description:** Wavelength Electronics LFI-3751.
Remote control: Local or analog control.
Location: Electronics rack.
Power: Up to 5 A, 40 W from 10A.
Role: Controls filter cavity temperature (when the cavity is used).
Manual / spec: Wavelength Electronics LFI-3751 manual.

23 – FPGA (Red Pitaya) **Model / description:** Red Pitaya STEMlab 125-14 (three units in the system).
Remote control: Yes, via Ethernet and SCPI.
Location: Electronics rack.
Power: 5 V, 2 A each; supplied from 24A, 24B, and 20B.
Role: Historically used to generate 10 MHz reference and lock signals; being phased out in favour of the Keysight generator.
Manual / spec: https://redpitaya.com/product/stemlab-125-14/

24 – DC power supply 2 **Model / description:** GW Instek GPP-1326 for RF amplifier.
Remote control: Yes (`InstekGppDCSupply` driver).

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Location: Electronics rack.
Power: AC 100/120/220/230 V, 50/60 Hz, via ports A2 and A4.
Role: Provides 30 V / 6 A for RF amplifier; one supply was replaced by an Acopian unit after a failure.
Manual / spec: https://www.gwinstek.com/en-global/products/downloadSeriesDownNew/14230/1737

25 – Heat exchanger **Model / description:** Liquid-cooled heat exchanger for RF and amplifier chassis.
Remote control: None.
Location: Electronics rack, cooling panel.
Power: Fed from port 22A.
Role: Removes heat from RF amplifier and EDFA mounting plate.
Manual / spec: Cooling-system vendor manual.

26–27 – Web power supplies (WPS) **Model / description:** Eaton EMA114-10 web-enabled power controllers (left and right units).
Remote control: Yes, via Ethernet web interface.
Location: Facility interface panel.
Power: Feed-through AC distribution.
Role: Provide remotely switchable AC outlets for the rack and optical bench.
Manual / spec: https://www.eaton.com/us/en-us/skuPage.EMA114-10.pdf

Items 28–40: RF chain and EO modulators

28 – RF oscillator **Model / description:** Nexyn 16 GHz RF oscillator.
Remote control: Local control; frequency monitored via counter.
Location: Optical bench.
Power: Supplied from DC power supply 1.
Role: Primary 16 GHz drive tone for the EO comb.
Manual / spec: Nexyn oscillator datasheet (internal link).

29 – RF amplifier **Model / description:** CTT / Kratos GaN power amplifier.
Remote control: None; controlled through its DC supply and RF input.
Location: Optical bench, on cooled panel.
Power: 30 V, up to ∼4–5 A from DC power supply 2.
Role: Amplifies the 16 GHz signal for the phase and intensity modulators.
Manual / spec: https://www.cttinc.com/-/media/ctt/pdf/ctt-2022-catalog-gan-power-pdf

30 – RF amplifier (spare) **Model / description:** Spare CTT / Kratos GaN power amplifier.
Remote control: None.
Location: Optical bench / spares location 21A.
Power: As per main RF amplifier.
Role: Replacement for the main RF amplifier in case of failure.
Manual / spec: https://www.cttinc.com/-/media/ctt/pdf/ctt-2022-catalog-gan-power-pdf

31 – RF splitter **Model / description:** Broadband RF power splitter.
Remote control: None.
Location: Optical bench, in RF chain.
Power: Passive device.
Role: Distributes RF power to multiple EO modulators.
Manual / spec: Manufacturer splitter datasheet.

32–34 – RF phase shifters **Model / description:** Adjustable RF phase shifters at 16 GHz.
Remote control: Manual adjustment.
Location: Optical bench, in RF chain.
Power: Passive devices.
Role: Optimise RF phase alignment across multiple modulators.
Manual / spec: Phase-shifter manufacturer datasheets.

35–37 – EO phase modulators **Model / description:** High-power EOSpace phase modulators, including models such as PM-5SES-20-PFA-PFA-UV-SOP1W-UL and PM-5SES-20-PFA-PFA-UV-UL.
Remote control: Driven by RF chain; no digital interface.
Location: Optical bench.
Power: Optical input from EDFA chain; RF input from amplifier.
Role: Generate phase-modulated sidebands that form the EO minicomb; one modulator has a small programmable RF attenuator (35a).
Manual / spec: https://www.eospace.com/phase-modulator

38 – Intensity modulator **Model / description:** EOSpace AX-0MSS-20-PFA-PFA-LV-UL 20 GHz $LiNbO_3$ intensity modulator.
Remote control: RF drive plus DC bias via IM-bias loop.
Location: Optical bench.
Power: Optical and RF as above.
Role: Provides intensity modulation for flattening the comb and for IM-bias locking.
Manual / spec: https://www.eospace.com/10-20-gb/s-modulators

39–40 – additional RF / EO components **Model / description:** Remaining EO components and RF hardware in the 35–40 group (cabling, terminations, etc.).
Remote control: None.
Location: Optical bench.
Power: Passive components.
Role: Complete the EO comb modulation chain.
Manual / spec: See internal RF-chain documentation.

Items 41–56: pump laser, nonlinear stages, and sensing

41 – Pump laser **Model / description:** Rio / Luna Orion RIO0175-5-01-4-AW2.
Remote control: Yes (`ORIONLaser` driver).
Location: Optical bench / enclosure.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Power: 5 V DC, 3.2 A (16 W) from port 9A.
Role: Main 1550 nm pump laser, typically locked to the Rb spectroscopy module.
Manual / spec: https://rio-lasers.com/Product-Briefs/Rio_1550_nm_Orion_Product_Brief_170126.pdf

42 – Filter cavity TEC (SLS) **Model / description:** SLS TEC controller for filter cavity (spare).
Remote control: Local or analog control.
Location: Optical bench (B3).
Power: From local DC supply.
Role: Optional temperature control; currently unused.
Manual / spec: SLS TEC controller documentation.

43 – Circulator **Model / description:** Fiber circulator.
Remote control: None.
Location: Optical bench.
Power: Passive device.
Role: Routes light through filter cavity and Rb module as required.
Manual / spec: Circulator manufacturer datasheet.

44 – AM photodetectors **Model / description:** Three amplitude photodiodes (two on breadboard, one in chassis).
Remote control: None; analog outputs only.
Location: Optical bench and chassis.
Power: 12 V DC, 0.25 A each (ports 18B, 19B, 17A).
Role: Provide amplitude monitor signals for IM lock and other diagnostics.
Manual / spec: Photodetector manufacturer datasheets.

46 – Pump frequency doubler **Model / description:** Covesion WGHP-S-1560-40 SHG waveguide module (fiber-coupled).
Remote control: None; controlled via optical and thermal inputs.
Location: Optical bench.
Power: Optical pump from RIO laser; temperature via TEC.
Role: Doubles 1560 nm light to 780 nm for rubidium spectroscopy.
Manual / spec: https://covesion.com/solutions/fiber-coupled-waveguide-components/

47 – Rubidium spectroscopy module **Model / description:** Vescent D2-210 Rb spectroscopy module.
Remote control: Local controls; output signals to scope and servo.
Location: Optical bench.
Power: Supplied from Vescent D2-005 power unit.
Role: Provides absorption and PDH signals for the rubidium lock.
Manual / spec: https://vescent.com/products/spectroscopy-module-d2-210

48 – Rubidium lock phase modulator **Model / description:** EOSpace phase modulator for PDH on the Rb line.
Remote control: Driven by RF from function generator.

Location: Optical bench.
Power: Optical plus RF inputs.
Role: Applies RF dither for PDH locking of the Rb transition.
Manual / spec: EOSpace phase-modulator datasheet.

49–50 – Pulse compression stages **Model / description:** Home-built OFS HNLF compression chassis and spare stage.
Remote control: None.
Location: Optical bench.
Power: Passive optical modules.
Role: Nonlinear compression and broadening before the waveguide.
Manual / spec: Internal design notes and OFS fiber datasheet.

51–53 – SCG waveguides **Model / description:** Octave Photonics Ta_2O_5 supercontinuum waveguides (one active, two spares).
Remote control: None.
Location: Optical bench.
Power: Passive devices; pumped by Pritel amplifier.
Role: Generate broadband astrocomb spectrum via supercontinuum generation.
Manual / spec: Octave Photonics waveguide specification sheets.

54 – WDM **Model / description:** 1550/2000 nm WDM pump filter.
Remote control: None.
Location: Optical bench.
Power: Passive device.
Role: Separates pump and long-wavelength comb light.
Manual / spec: WDM manufacturer datasheet.

55 – DAQ 1 **Model / description:** Measurement Computing USB-2408 (breadboard DAQ).
Remote control: Yes, via USB.
Location: Optical bench.
Power: USB powered.
Role: Acquires thermocouple and analog signals on the optical table.
Manual / spec: https://microdaq.com/mesurement-computing-usb-2408-2ao-data-acqu
php

56 – TEC controllers **Model / description:** Two TETech TC-720 OEM controllers (for waveguide and doubler).
Remote control: Yes, via serial or analog interface (`TEC_TC720` driver).
Location: Optical bench.
Power: 12 V, up to 10 A from supply on port 16A and associated wiring.
Role: Control the temperatures of the PPLN/doubler and the SCG waveguide.
Manual / spec: https://tetech.com/files/temperature_controller/TC-720_MANUAL.pdf

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Items 57–70: auxiliary components and candidate hardware

57 – USB hub 2 (7-port) **Model / description:** Powered USB hub for breadboard components.
Remote control: Not applicable.
Location: Optical bench.
Power: From local DC adapter.
Role: Aggregates USB devices on the optical table.
Manual / spec: USB-hub vendor datasheet.

58 – SLM **Model / description:** Santec spatial light modulator (within Menlo flattener unit).
Remote control: Yes, via flattener control laptop.
Location: Optical bench / flattener enclosure.
Power: From port 16B.
Role: Part of spectral flattener, shaping amplitude across spectral channels.
Manual / spec: Santec SLM documentation.

59 – Spectrometer **Model / description:** Arcoptix FTNIR-L1-025-2TE NIR spectrometer.
Remote control: Yes, via USB.
Location: Optical bench / flattener enclosure.
Power: From port 13B.
Role: Monitors the flattener output spectrum.
Manual / spec: https://arcoptix.com/nir-spectrometer.htm

60 – Shutter **Model / description:** Thorlabs shutter.
Remote control: Yes, via Thorlabs shutter controller.
Location: Optical bench.
Power: From controller (item 64).
Role: Mechanical beam block for flattener or diagnostic beam paths.
Manual / spec: Thorlabs shutter datasheet.

61 – Filter slider **Model / description:** Thorlabs motorised filter slider.
Remote control: Yes, via controller interface.
Location: Optical bench.
Power: From port 22B.
Role: Inserts/removes ND filters in the flattener path.
Manual / spec: Thorlabs filter-slider manual.

62 – Oscilloscope **Model / description:** HP oscilloscope (model as labelled).
Remote control: Possible via GPIB/USB (not normally used).
Location: Optical bench.
Power: From port 12A.
Role: Observes Rb spectroscopy and PDH signals. The scope occasionally freezes and may require an ePDU power cycle.
Manual / spec: HP oscilloscope user manual (model dependent).

63 – Line-referenced pump laser **Model / description:** Clarity 1560-HP NLL Wavelength References laser.
Remote control: Yes, via vendor interface.
Location: Optical bench.
Power: As specified by Clarity manual.
Role: Alternative pump laser with lower frequency stability (on the order of 30 cm/s); used when not operating the RIO lock.
Manual / spec: Clarity / Wavelength References NLL user manual.

64 – Shutter controller **Model / description:** Thorlabs shutter controller.
Remote control: Local or remote via digital interface.
Location: Optical bench / electronics shelf.
Power: From port 11A.
Role: Drives the Thorlabs shutter (item 60).
Manual / spec: Thorlabs shutter-controller manual.

65 – Flattener control laptop **Model / description:** Laptop dedicated to Menlo / flattener control.
Remote control: Yes, via Ethernet.
Location: Near flattener rack.
Power: From port 9B.
Role: Runs Menlo software, SLM control, and spectrometer interfaces.
Manual / spec: Laptop vendor documentation.

66 – USB hub 1 (4-port) **Model / description:** 4-port USB hub for flattener subsystem.
Remote control: Not applicable.
Location: Interface panel / flattener region.
Power: From local adapter, plugged into interface panel.
Role: Connects SLM, spectrometer, and related devices.
Manual / spec: USB-hub vendor datasheet.

67 – Webcam **Model / description:** Standard USB webcam.
Remote control: Accessible via host PC.
Location: Mounted to view optical table.
Power: USB powered.
Role: Provides visual monitoring of the optical setup.
Manual / spec: Webcam user manual (model dependent).

68 – Autocorrelator **Model / description:** Femtochrome FR-103XL autocorrelator.
Remote control: Local panel and analog outputs.
Location: Brought in as needed; powered from 14A.
Power: As specified in FR-103XL manual.
Role: Measures pulse duration when needed; on loan from JPL or Caltech.
Manual / spec: Femtochrome FR-103XL user manual.

69 – Rack side-wall muffin fans **Model / description:** Pair of side-wall cooling

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

fans.
Remote control: On/off via ePDU.
Location: Rack side wall.
Power: From port 12B.
Role: Enhance rack-side airflow and thermal stability.
Manual / spec: Fan manufacturer datasheet.

70 – Laptop monitor on desktop **Model / description:** External monitor for LFC / flattener laptops.
Remote control: None.
Location: On desktop near rack.
Power: From port 14B.
Role: Improves usability of GUIs and remote sessions.
Manual / spec: Monitor user manual (model dependent).

Candidate future hardware – SIM983 scaling voltage amplifier **Model / description:** SRS SIM983 ±10 V scaling voltage amplifier (not yet installed).
Remote control: Would be addressed via SIM900 mainframe.
Location: Planned for SIM rack slot.
Power: From SIM900 internal rails.
Role: Candidate for future use to provide higher-voltage control signals within the SIM rack.
Manual / spec: https://www.thinksrs.com/downloads/pdfs/catalog/SIM983c.pdf

This section provides a one-to-one mapping between the numbered components in Fig. 5.3, the hardware inventory, and the relevant manual/specification links, so that new users can quickly locate documentation and understand the role of each device in the astrocomb system.

5.2.1 Rack interface connection introduction

Table 5.1. Rack interface: optical fiber connections.

Row	Port 1	Port 2	Port 3	Port 4
A	Broad comb output to chassis/VOA	Labeled "comb output to VOA", actually going to benchtop OSA	Unused	Unused
B	Unused	Unused	Unused	From 23 dBm EDFA to FC
C	Minicomb in to OSA	FC in to OSA monitor	Doubler in to OSA monitor	From FC to Pritel relay
D	Pritel to HNLF pulse compression stage	Minicomb to waveshaper	27 dBm EDFA to phase modulators	RIO to 27 dBm EDFA

Table 5.2. Rack interface: coaxial cable connections.

Row	Port 1	Port 2	Port 3	Port 4
A	FC PD to mixer	Function generator to FC phase modulator	IM DC bias to SRS PID	Minicomb photodiode to SRS PID
B	Function generator to Rb phase modulator	SRS to RIO laser feedback	Spare	FC piezo controller to FC
C	Rb cell to PDH	Temporary rubidium PDH (?)	Temporary rubidium PDH (?)	10 MHz FS725 to 16 GHz RF oscillator
D	Oscilloscope channel 4	Oscilloscope channel 3	DC to IM RF attenuator	Unused

Table 5.3. Rack interface: RJ45 and USB connections.

Row	Port 1	Port 2
A	–	RJ 24
B	Breadboard USB hub (7 ports)	Oscilloscope (USB)

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Table 5.4. Rack interface: power connectors.

Row	Port 1	Port 2
A	Bad	Osc
B	RF Amp	Feedthrough for flattener power

Table 5.5. SMA connections.

SMA connectors	
Port	Device
1	Frequency counter in
2	Spare
3	Spare
4	Spare

5.2.2 Rack router IP information

Table 5.6. Rack network devices and assigned IP addresses.

Rack	Device / Name	MAC address	Assigned IP address
1	laptop-lfc	F8-E4-3B-7C-96-EA	192.168.0.159
2	rp-f0602c	00-26-32-F0-60-2C	192.168.0.104
3	rp-f072a9	00-26-32-F0-72-A9	192.168.0.119
4	rp-f072ec	00-26-32-F0-72-EC	192.168.0.103
5	TP-Link router (TL-R600VPN)	–	–

Rack router: TP-Link TL-R600VPN. **User guide:** https://static.tp-link.com/2021/202102/20210223/1910012490_TL-R600VPN_UG.pdf.

5.3 Detailed Instrument Design

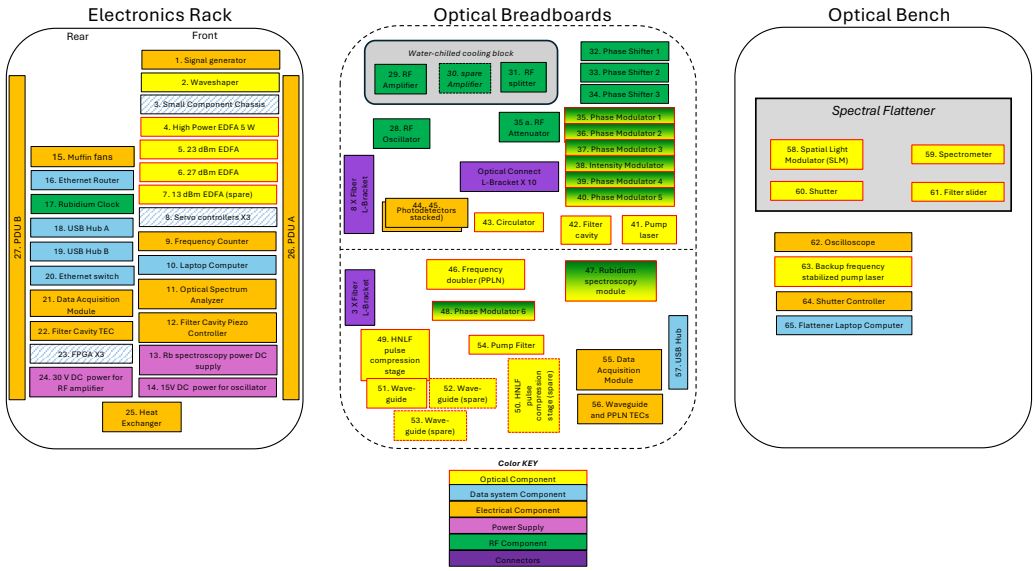


Figure 5.4

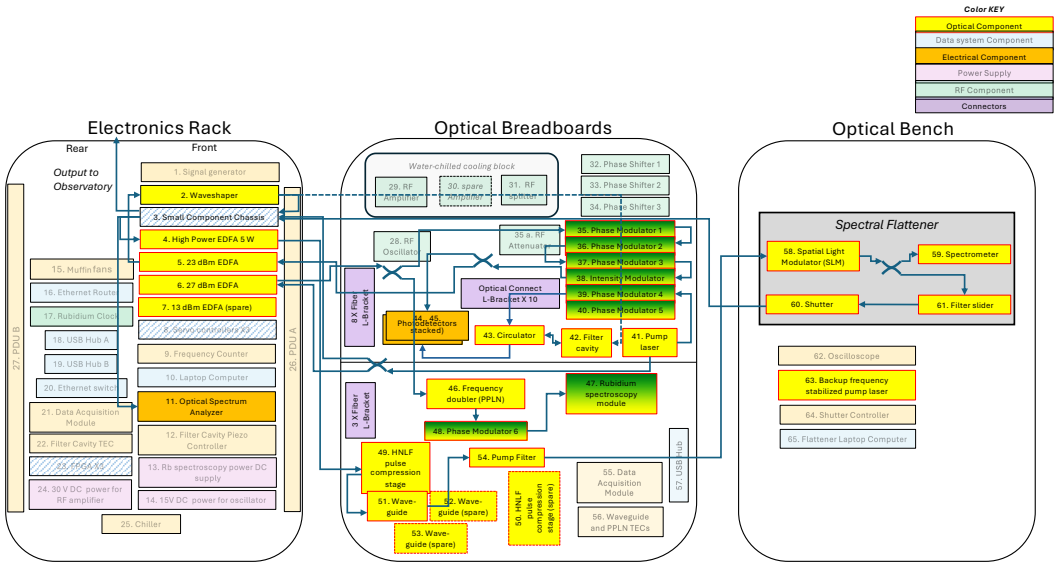


Figure 5.5

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

5.3.1 Comb Generator

5.3.1.1 Minicomb Generation

The HSF EO astrocomb signal chain is depicted in Figure 5.6 and Figure 5.7. It uses three cascaded low V_{pi} phase modulators and an intensity modulator driven by a low-phase-noise, 16 GHz oscillator locked to a GPS-referenced rubidium clock to modulate an amplified, frequency stabilized 1560 nm pump laser. RF phase shifters are used between the modulators to assure an optimized (broad, flat) minicomb approximately 11 nm wide (see Figure 5.8). A variable RF attenuator is used to tune the RF power that drives the intensity modulator. To attain enough RF power to overdrive the modulators, the RF oscillator signal is sent to a high power (41 dBm) RF amplifier. The output from the RF amplifier is split into three channels, each supplying one phase modulator. Rejected RF power from one of the phase modulators is sent to drive the intensity modulator, and supplies the signal to a frequency counter to monitor the comb repetition rate. Residual power from the other two phase modulators is dissipated in 50-ohm termination ports. A small portion of the minicomb signal is tapped off and sent to a photodetector to create the control signal for a PDH control loop for minicomb amplitude stabilization. The signal is sent to a servo controller (SRS SIM 960) that feeds back a bias voltage to the intensity modulator. The main minicomb signal is amplified in an erbium-doped fiber amplifier (EDFA) to increase the pulse energy, sent through a dispersion compensation unit (a Finisar Waveshaper), and then amplified again. The light is then split in a polarizing beam splitter (PBS) into two fibers corresponding to the fast- and slow-polarization axes. The fast axis light may be used in the control loop for the FP filter cavity as described in a subsequent section. The slow axis light continues to a high power EDFA (Pritel) where it is amplified to approximately 4W of optical power. In order to maintain constant comb intensity profile, the pulse energy of the comb light injected into the HNLF pulse compression stage and supercontinuum generating waveguide must be held constant. This is accomplished by utilizing an amplitude feedback control loop on the minicomb; a 90:10 optical tap directs 10% of the minicomb light to a photodetector that supplies a voltage proportional to the incident minicomb light intensity. This signal is sent to a servo controller that adjusts the bias voltage (-10V to +10V) applied to the intensity modulator to hold the minicomb amplitude constant.

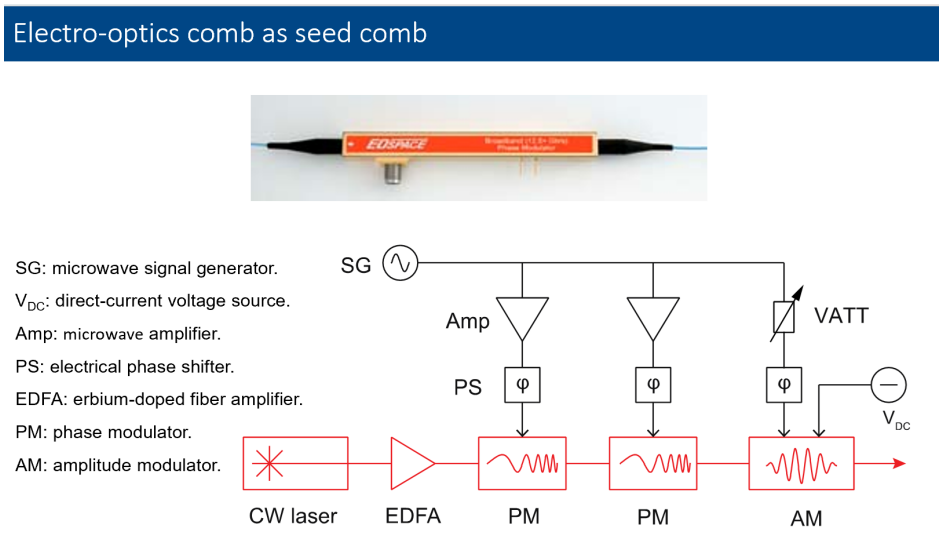


Figure 5.6. Minicomb (EO comb) setup

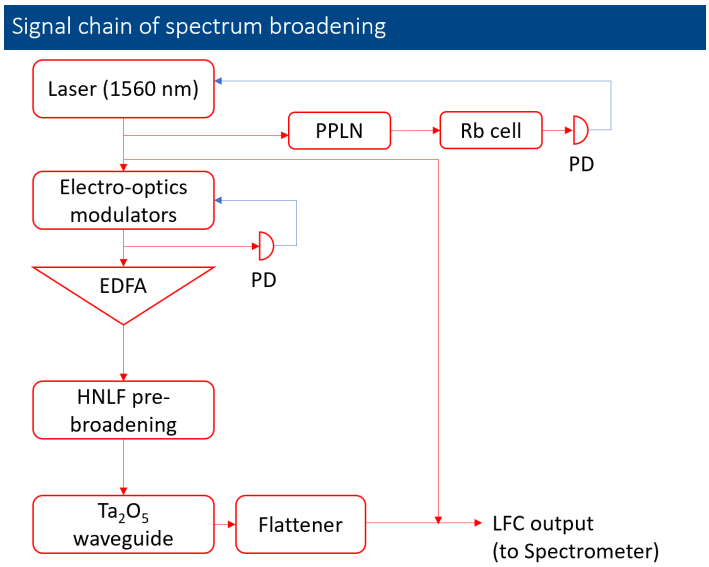


Figure 5.7. Minicomb (EO comb) signal chain

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

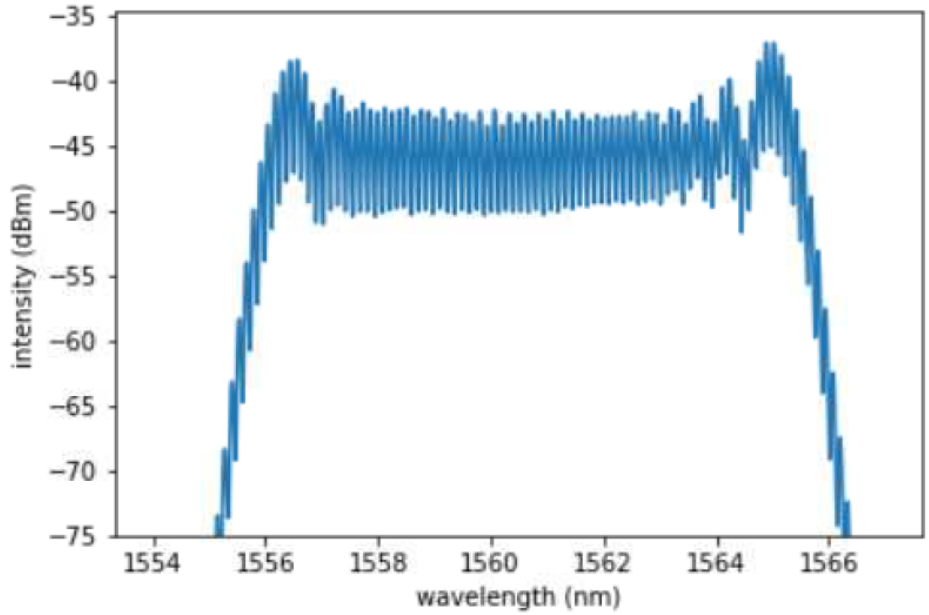


Figure 5.8. Minicomb (EO comb) spectrum

5.3.1.2 Supercontinuum Generation

The pulse is then temporally compressed in a stage of normal-dispersion (-2.3 ps/nm/km) highly non-linear fiber (HNLF) spliced to a length of PM 1550 fiber to further increase peak pulse energy. These spliced fiber segments, the pulse compression stage, is housed in a small aluminum chassis on the breadboard. The compressed pulse is then sent into a specially engineered, temperature-controlled (via a TEC) tantala waveguide (Octave Photonics) where, through nonlinear photonic processes, the minicomb spectrum is broadened to well over an octave of bandwidth.

The full spectrum shown below (Fig. 5.9, Fig. 5.10, Fig. 5.11) are from test with Octave team on summit, May 17, 2025.

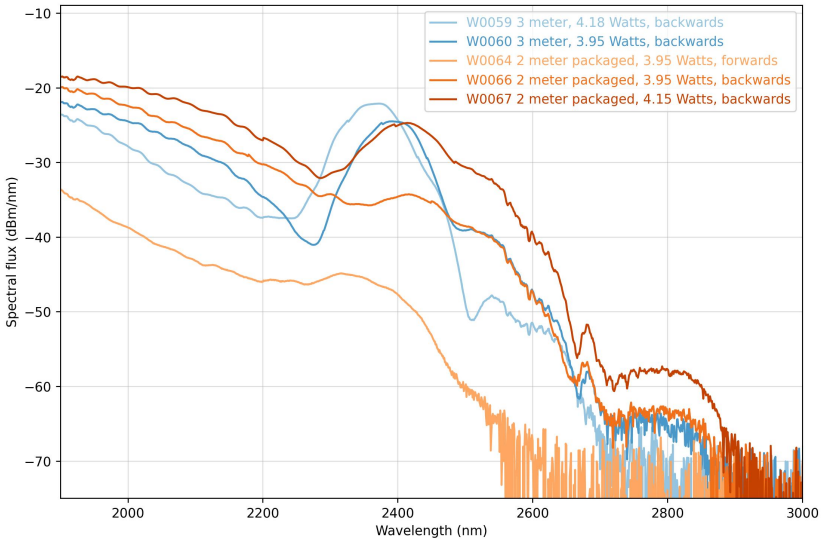


Figure 5.9. spectrum direct from waveguide

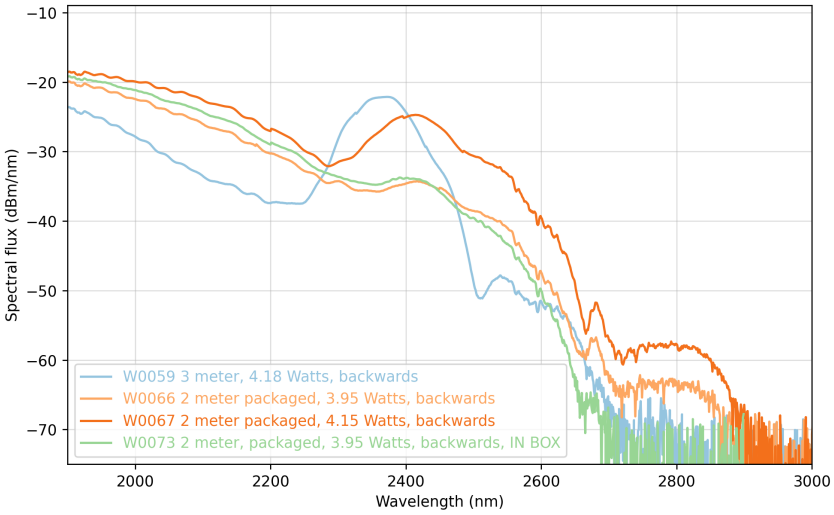


Figure 5.10. spectrum before menlo flattener

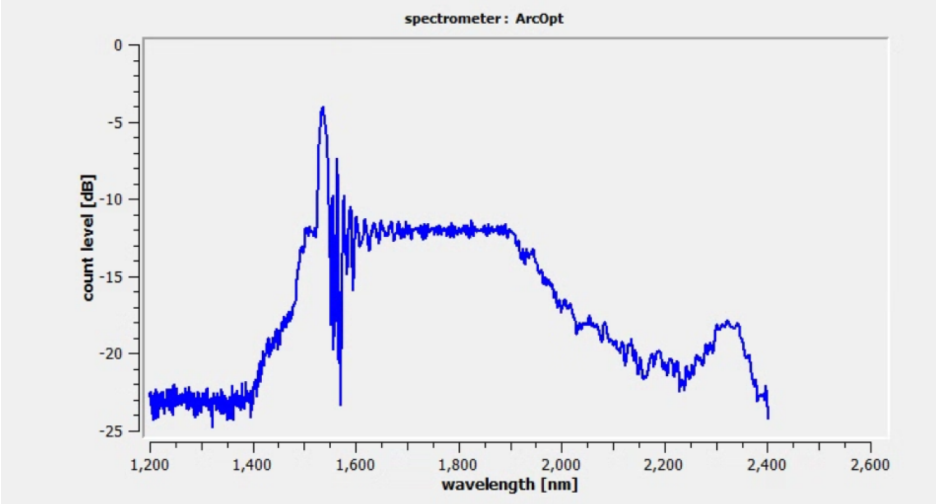


Figure 5.11. Full spectrum shown on menlo flattener

5.3.2 Frequency Stabilization of the Pump Laser

The EO astrocomb has two main sources of frequency instability that must be controlled. The repetition rate, or line spacing, is controlled by a 16 GHZ phase locked loop dielectric resonance oscillator (PLL-DRO) stabilized at 10 MHz by a GPS-disciplined (1 PPS) rubidium clock (SRS FS725). The pump laser may be stabilized to one of three references – a HCN stabilized laser at 1559 nm (the Clarity laser by Wavelength References), a rubidium spectroscopy module by Vescent Photonics, or a GPS-referenced, low repetition rate fiber laser frequency comb. Each of these options is described below.

5.3.2.1 Clarity Laser

Currently, the NIR LFC pump laser (Fig. 5.12) is the Clarity laser with stability corresponding to 30 cm/s radial velocity precision at 100 s.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Figure 5.12. Clarity laser

5.3.2.2 Rubidium-locked Laser

The laser frequency can be locked to a rubidium transition at 780 nm. This is accomplished by tapping a portion of the amplified 1560 nm pump light near the beginning of the signal chain, and frequency doubling it via second harmonic generation in a PM fiber coupled doubling crystal (Covesion). The resulting 780 nm light is sent to a rubidium spectroscopy module (Vescent Photonics) that outputs an RF signal maximized when the doubled laser frequency matches the absorption peak. A control loop adjusts the current to the pump laser to hold this frequency fixed. The technique works well if the system has been operating continuously, but not from a cold start. This is because the pump laser, a RIO-Orion module with 1 kHz line width and nominal center wavelength of 1560.492 nm $\pm$ 0.015 nm, drifts significantly over time when not locked to a reference.

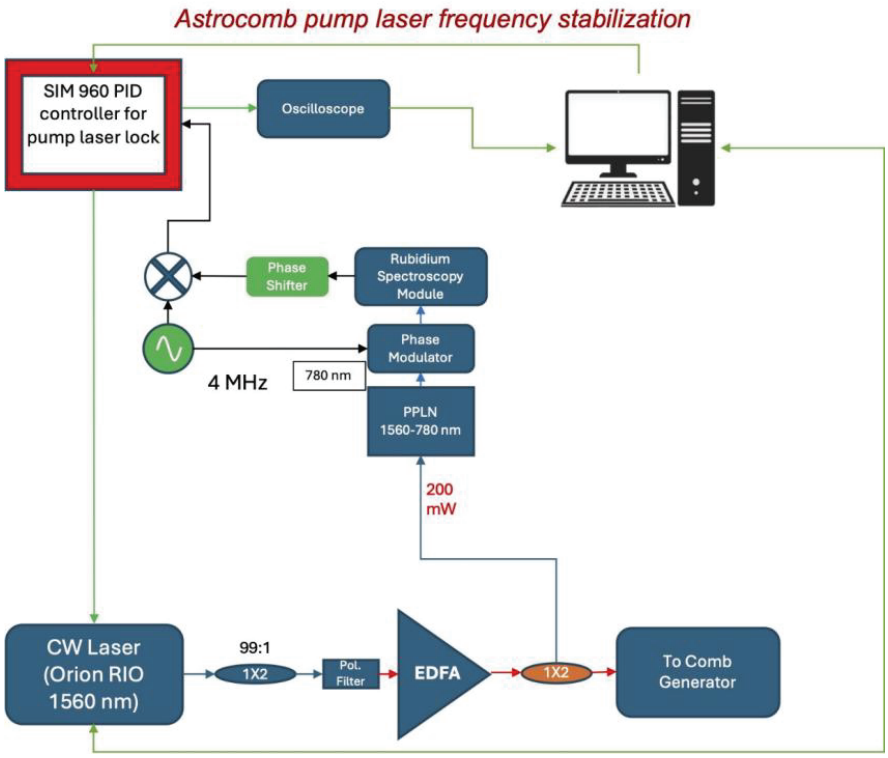


Figure 5.13. rb locking assembly

Instrumentation Figure 5.13 shows the pump laser locking assembly. A RIO Orion laser is amplified with an EDFA and the output split between the comb input and the frequency locking arm. In the locking arm, the 1560 nm light is doubled in a PPLN, sent through a phase modulator modulated at 4 MHz with a signal generator, and then into the Vescent D2-210-RB-FC. The RF output from the spectroscopy module is phase shifted and mixed down, then sent to a SIM 960 PID controller, which is used for PDH locking. The trace on the oscilloscope is digitized and when the laser frequency is in a good slope region (around the absorption feature) for the lock, it is stored as a template to correlate for later auto-locking. There are two ways to scan the laser frequency. The first is tuning the laser current, which is like "coarse" tuning over the full 30 pm range; if we scan using the laser current too quickly and over too wide a range, there is concern of mode hopping, and there is not enough resolution on the oscilloscope trace to clearly identify the desired absorption feature. The second method uses a voltage bias applied to the Orion's modulation port. This is a "fine tuning" method, and better for a narrow sweep range around the desired Rb absorption feature. When we tune the laser frequency, we must also tune the temperature of the Covesion doubler using a TEC to maximize transmission. This process slows data acquisition and lengthens the time needed to identify the desired locking region. In general, if environmental conditions in the facility are consistent, we can unusually scan the same region using the voltage modulation and find the peak which we can then use for PDH locking. Figures 5.14 a) and b) show the scans of the rubidium cell as the pump laser modulation port bias voltage is swept. The locking algorithm is as follows: First, we use convolution to compare a), the blue trace (present scan) and b) the orange trace (stored data). This generates a value that indicates where they

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

most closely match. The target locking region (fine absorption line) is highlighted by the red rectangle on the blue trace. The green curve below in Figure 5.15 represents this matching value, while the red point marks the position where the two traces align best. Importantly, the orange trace only represents the shape of the reference data and does not indicate the optimal alignment position—only the red point shows the location of the best match.

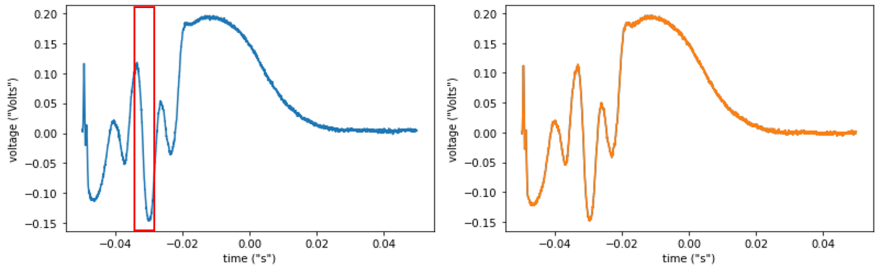


Figure 5.14. a) (left) Rubidium cell present scan data, and b) (right) stored scan data.

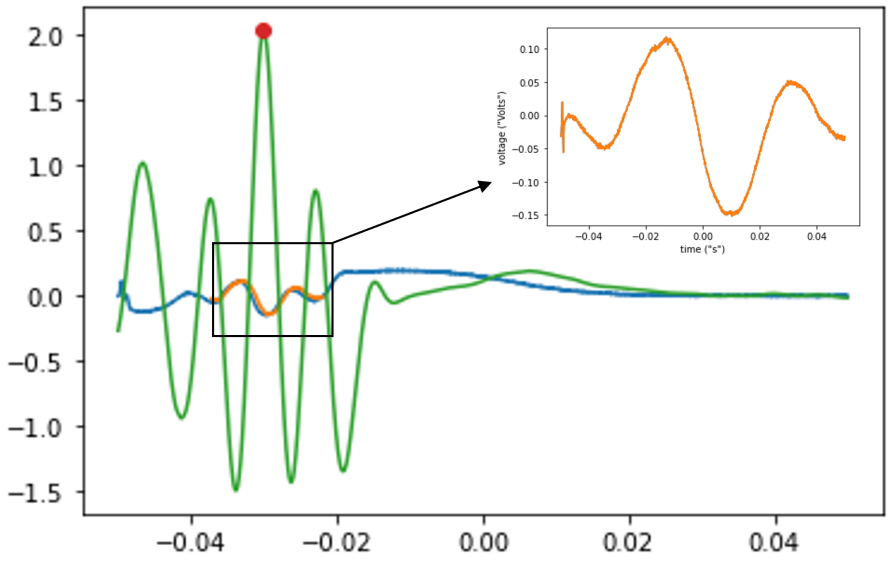


Figure 5.15. Convolution of present and stored rubidium cell absorption traces.

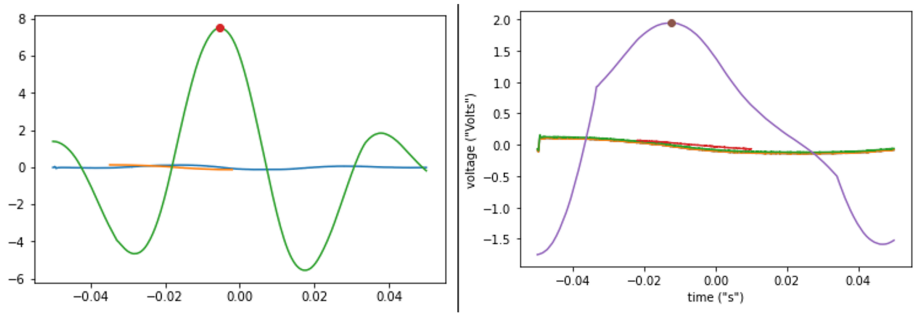


Figure 5.16. Narrow-range sweep. The red dot marks the maximum of the convolution, the point where the stored trace (orange) overlaps best with the current trace (blue).

By iterating this process (over smaller and smaller voltage sweep regions Fig. 5.16) we gradually refine the locking range (the amplitude of the voltage sweep) until we achieve a single monotonic slope of the fine absorption line in Rb. Once this condition is met, we can directly lock onto the center of the slope, completing the locking process. The total Rb absorption line see Fig. 5.17

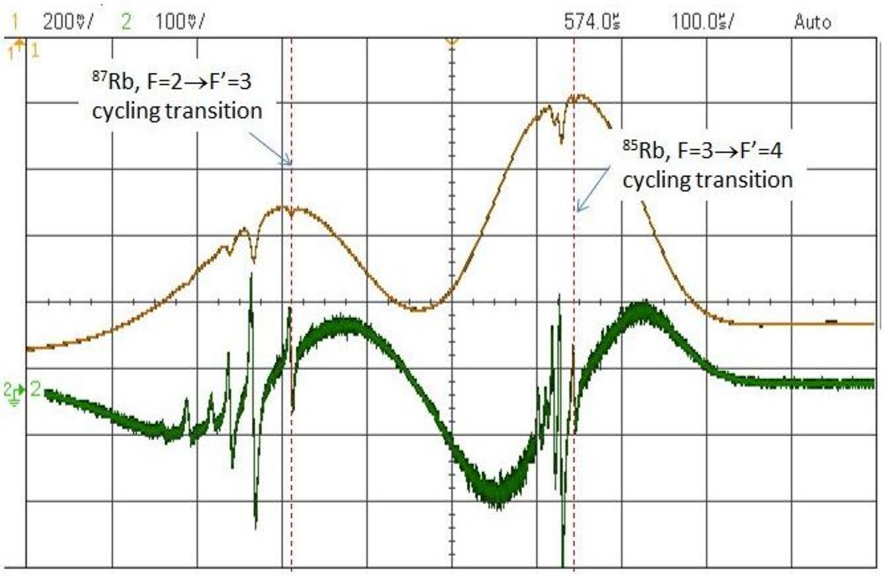


Figure 5.17. Rb module line

5.3.2.3 Fiber Frequency Comb Referenced Pump Laser

Long term stability measurements of the Astrocomb may be accomplished by heterodyning a portion of the comb output signal with that from another (more) stable reference; available for this purpose is the red arm output of the Keck Planet Finder (KPF) Astrocomb supplied by Menlo Systems. The overlap region between approximately 1100nm and 1400 nm (TBC) is optimal for this test.

5.3.3 Electronics Rack Small component chassis

The following items are mounted in the small-component chassis and nearby rack space. Power port numbers refer to the labelled ePDU outlets; USB port numbers refer to the numbered USB hubs in the rack when applicable.

Interlock controller

Model / description: Arduino-based interlock controller.
Power: From ePDU outlet 1B.
USB: Connected to a USB hub (port number depends on cabling).
Function: Monitors input optical power to the Pritel EDFA and triggers an interlock shutdown if the power is outside the allowed range.
Notes: Forms part of the safety chain for the high-power EDFA.

VOA-1550

Model / description: OzOptics pump-line VOA (1550 nm).

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Power: 5 VDC, 3 A max from ePDU outlet 7B.
USB: Controlled through the VOA electronics (no dedicated rack USB port number assigned).
Function: Sets attenuation for the pump-line path of the comb.
Notes: See the VOA control section for software interface details.

VOA-1310
Model / description: OzOptics YJ-channel VOA (1310 nm).
Power: 5 VDC, 3 A max from ePDU outlet 7B.
USB: As for VOA-1550; shared VOA control electronics.
Function: Adjusts attenuation for the YJ comb output.
Notes: Electrically identical to VOA-1550 and VOA-2000.

VOA-2000
Model / description: OzOptics HK-channel VOA (around 2000 nm).
Power: 5 VDC, 3 A max from ePDU outlet 7B.
USB: Same VOA control interface as other channels.
Function: Controls attenuation for the HK-band comb output.

2×2 optical switch
Model / description: Agiltron FFSW-222C00323 MEMS latching 2×2 optical switch.
Power: Powered through its dedicated driver electronics (see "2×2 switch driver" below).
USB: Controlled via the Agiltron USB driver.
Manual / spec:https://agiltron.com/dlc/specs/MEMS%20Latching%20Series%20Switch.pdf
Driver software:https://agiltron.com/product/mems-1x1-dual-1x11x2-quad-1x11x2-ful
Function: Routes the comb between different fiber outputs (for example, selecting YJ or HK paths).

RF amplifier
Model / description: High-power RF amplifier for the EO comb drive.
Power: From ePDU outlet 2B (DC supply described in the RF chain section).
USB: None (controlled via analog/RF connections only).
Function: Amplifies the 16 GHz RF drive for the phase and intensity modulators.
Notes: Thermal management is provided by the cooling panel and heatsink; see RF-chain safety notes.

RF mixer
Model / description: Passive RF mixer in the RF chain.
Power: Passive component, no dedicated power port.
USB: None.
Function: Frequency translation / mixing as required by the RF electronics.
Notes: Mechanical location is documented in the RF layout drawings.

RF filter

Model / description: Passive RF filter.
Power: Passive component, no power port.
USB: None.
Function: Shapes the RF spectrum and suppresses unwanted harmonics.

Pritel interlock cables
Model / description: Dedicated cable set between the Arduino interlock controller and the Pritel EDFA interlock ports.
Power: Passive cabling, no direct power requirement.
USB: None.
Function: Carries interlock signals that shut down the Pritel EDFA when unsafe conditions are detected.
Spares: Three spare cable sets stored in the "Spares Box".

Pritel interlock latching circuit
Model / description: Dedicated latching circuit for the Pritel interlock.
Power: From power outlet A13.
USB: None (driven by the Arduino logic).
Function: Implements a latching behaviour so that once an interlock trip occurs the Pritel EDFA remains disabled until explicitly reset.

2×2 switch driver
Model / description: Agiltron mini driver for FFSW-full 2×2 latching switch, USB driver, also supports TTL and push-button control (model SWDR-111111121).
Power: Powered from the rack low-voltage supply (see wiring diagram; no separate ePDU outlet listed in the inventory table).
USB: Connected to a USB hub for software control.
Function: Provides the drive and control interface for the Agiltron 2×2 optical switch, including latching and manual override modes.

YJ shutter
Model / description: Agiltron fiber shutter for the YJ comb output.
Power: From ePDU outlet 11B.
USB: Controlled via the shutter driver electronics (no explicit USB port number assigned).
Function: Opens or closes the YJ output towards the spectrograph; also controlled by the KTL keywords `LFC_YJ_SHUTTER` / `LFC_YJ_ONOFF`.
Notes: See the shutter section for software and safety details.

5.3.4 Filter cavity

(this feature is deprecated, but may be reinstated in future system modifications) Describe role of filter cavity, why it was taken out of the loop, why it might be needed again, and if so, the interplay between it and the pump laser and rep rate; would need to tune the rep rate by 1.2MHz (TBC) The Astrocomb was designed to suppress RF phase noise from the drive oscillator by passing the amplified minicomb light through a 16 GHz free spectral range (FSR) Fabry-Perot (FP) filter cavity locked to the pump laser. This is accomplished by using the fast axis light split off in the comb signal

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

chain using a polarizing beam splitter.

If not filtered, this noise will be amplified in the nth comb line as (TBD).... causing line broadening and a loss of coherence at wavelengths far from the pump. However, during initial commissioning tests it was determined that this filter stage was unnecessary because comb lines were unresolved (sub-pixel) across all observable orders as seen on the NIRSPEC spectrograph. Nevertheless, the FP filter components remain on the breadboard to allow for future implementation should it be deemed necessary. Such comb line filtering is necessary when attempting to self-reference an EO comb to achieve high SNR of the f-2f beat note. However, the NIR Astrocomb is a line-referenced instrument.

In order to minimize the insertion loss of the filter cavity, it is necessary to tune the RF drive oscillator to a frequency slightly offset from 16 GHz when the pump laser is locked to its rubidium line reference. This offset frequency was determined to be 1.2 MHz. If the filter cavity loop is to be implemented in the future, the drive oscillators will have to be returned to the vendor (Nexyn Corporation) and tuned appropriately. It is unknown whether this offset frequency will change over time and under varying environment conditions, and thus should be verified prior to making any changes to the oscillator by substituting a variable RF source for the Nexyn oscillator and maximizing the filter cavity throughput.

CW laser light is split off from the pump laser and directed through a phase modular driven by one channel of a two-channel signal generator. The modulated light is injected into a circulator, through a polarizing beam splitter (fast axis), and backinjected into the filter cavity. The reflected fast axis light returns through the PBS, into the circulator, and onto a photodetector that generates an RF signal, which is then sent to a mixer where it is divided down by mixing it with the second channel of the signal generator, phase shifted from the first by delta phi. The mixer then provides the error signal to a servo controller that tunes the voltage of the controller for the filter cavity piezo actuator to lock the cavity to the pump laser.

Control loop Piezo controller Temperature controller

5.3.5 Control system

(computers, ethernet, USB, serial, VISA protocol) FPGA(s) (deprecated)

Computer: LFC laptop

Ethernet: see 5.2.2

VISA protocol: please see 5.4

USB serial:

Table 5.7. USB hub port assignments for hub A and hub B.

USB A		USB B	
Port number	Device	Port number	Device
1	GPIB (SIM900?)	1	–
2	RF Amp 1 PS	2	–
3	RF Osc PS	3	FC TEC
4	RF Amp 2 PS	4	Spare
5	Keysight Function generator	5	Spare
6	Amonics 27 dBm	6	DAQ2408 #1
7	Amonics 13 dBm	7	Spare
8	Pritel	8	Spare
9	Waveshaper	9	Spare
10	Rb Clock	10	Spare
11	Amonics 23 dBm	11	Spare
12	Eaton LEFT	12	Spare
13	Latching Circuit	13	–
14	TP Link	14	–
15	Spare	15	RJ45-USB computerz

5.3.6 Thermal control

In the bottom of the rack is a glycol-cooled heat exchanger supplied by WMKO; the heat exchanger expels cool air to the optical bench-facing side of the rack and draws in hot air through the opposite, outward facing side. At the top the electronics rack is a fan tray to force air down [Deprecated], A chimney with inlet at the top of the rack pulls warm air down via a fan installed midway down the duct work, and expels it into the bottom of the rack at the intake to the heat exchanger. Two fans mounted approximately mid-way on the inside of the optical bench-facing side force warm air toward the intake-side of the heat exchanger. All of the shelving in the rack is perforated to allow for air flow.

The breadboards are also glycol-cooled; they are fed by copper lines plumbed from the same source on the facility interface panel (Fig. 5.18) . The chiller is located in an adjacent room in the basement. The board and rack are each equipment with a series of thermocouples to monitor temperature. Table 5.4.1.18 Shows the location and label for each of the 16 thermocouples (Type, part number) which interface to the LFC computer via two (2) McDAQ 8 port data acquisition modules (part number). Chiller temperature is set to ??? (TBC) and is monitored at he interface panel via an analog gauge.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

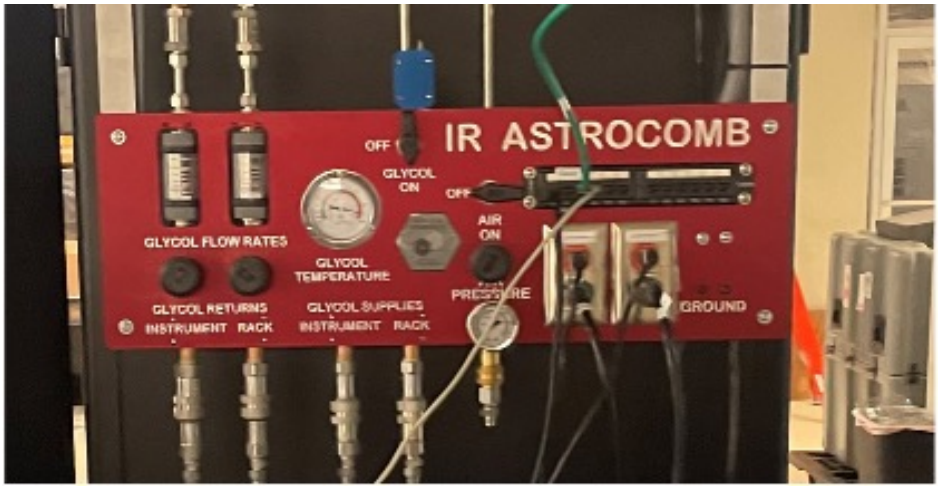


Figure 5.18. facility interface panel

5.3.7 Spectral Flattener

The NIR LFC spectral flattener is provided by Menlo Systems (Fig. 5.19, Fig. 5.20). The role of the flattener is to modify the intensity of comb lines across the full spectral bandpass to provide uniform calibration light intensity on the observatory's spectrograph. Light is injected into the spectral flattener unit through a fiber connected collimator that then directs the light via free space onto a reflective grating where it is spectrally dispersed and then directed onto a liquid Crystal on Silicon (LCoS) spatial light modulator (SLM). The individual SLM pixels rotate the polarization of incident light when a voltage is applied. The light returns through the polarizer resulting in amplitude attenuation of each channel in proportion to the applied voltage; a voltage "mask" is generated through feedback from a local spectrometer internal to the flattener unit to obtain a flat spectrum. The flattened spectrum is then directed through a filter slider assembly with user-selectable neutral density attenuators, then through a shutter, out of the instrument assembly, and into an optical fiber that supplies the desired instrument.

a dichroic filter (to be added) that splits off the comb light blue of 1480 nm and directs it 90 degrees to a collimator fiber coupler optimized for this spectral band. At this time, this channel is not being used.

Update with pump filter with wedge, and rejected light to monitor flattener power. Reliable check in case flattener software freezes

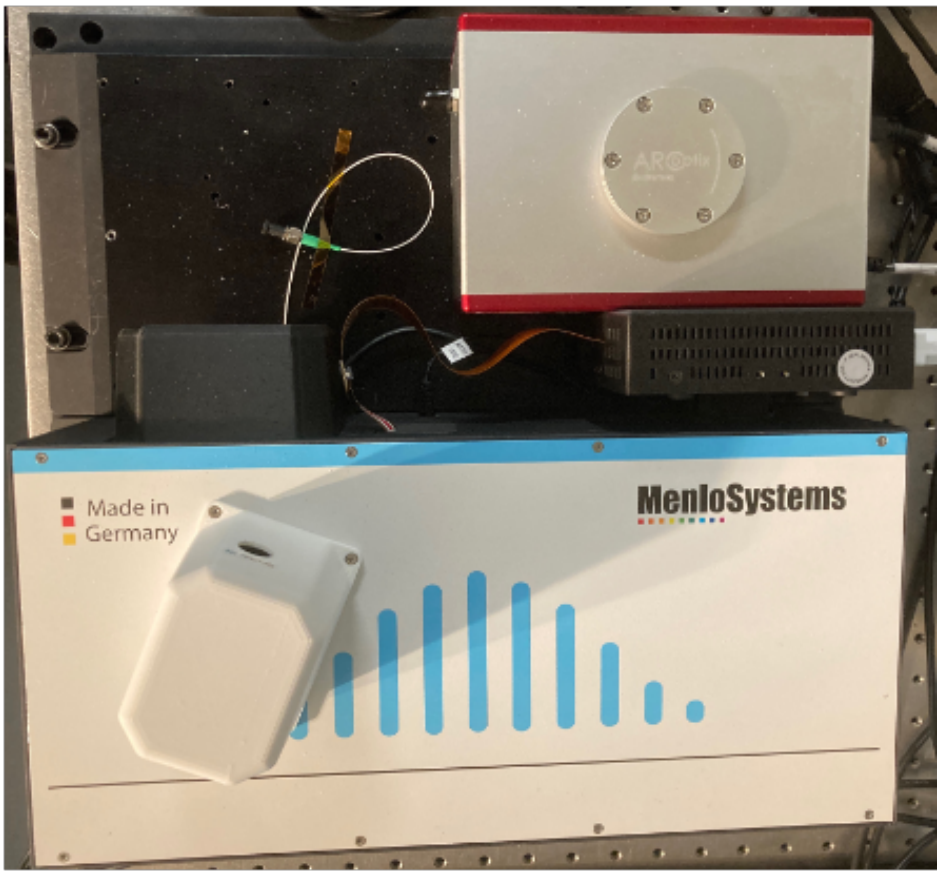


Figure 5.19. Menlo flattener outlook

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

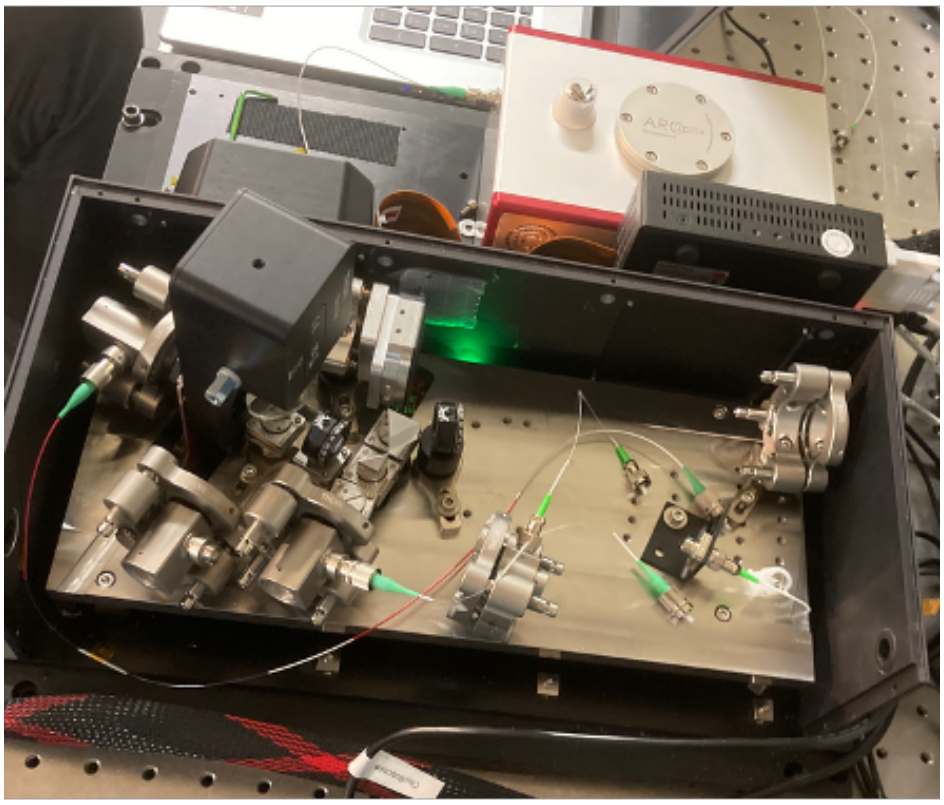


Figure 5.20. Menlo flattener inside

5.3.8 Fiber Feed

All of the Astrocomb fiber connections are made with polarization-maintaining (PM) Panda 1550 fiber. Nearly all (with exceptions noted) of the fiber connections are made with narrow-key PM FC/APC connectors. Table **??** and Figure ??? shows the fiber connections within the electronics rack, on the optical breadboards, and between the electronics rack and breadboards. Note that the latter is also reported in the document NIR Astrocomb ICD. Facility Fiber Feed Present SM28 fiber Planned ZBLAN fiber

5.3.9 Electrical

There are two vertical power strips mounted at the reach of the Astrocomb electronics rack. Each one is a 24-receptable Eaton model EMA1140-10 managed power distribution unit (PDU). Each strip has a 16A breaker. These PDUs report the power draw at any of the receptables, as well as environmental data (TBC – temperature and humidity) Both units are connected to the rack ethernet switch. The left- and right-hand strips are denoted “A” and “B”, respectively, and the outlets numbered 1 through 24 from top to bottom. Table 5.1 lists the outlet instrument assignments for each strip. See Fig. 5.21, Fig. 5.22, Fig. 5.23, Fig. 5.24. The ePDU maximum power draw see Fig. 5.25

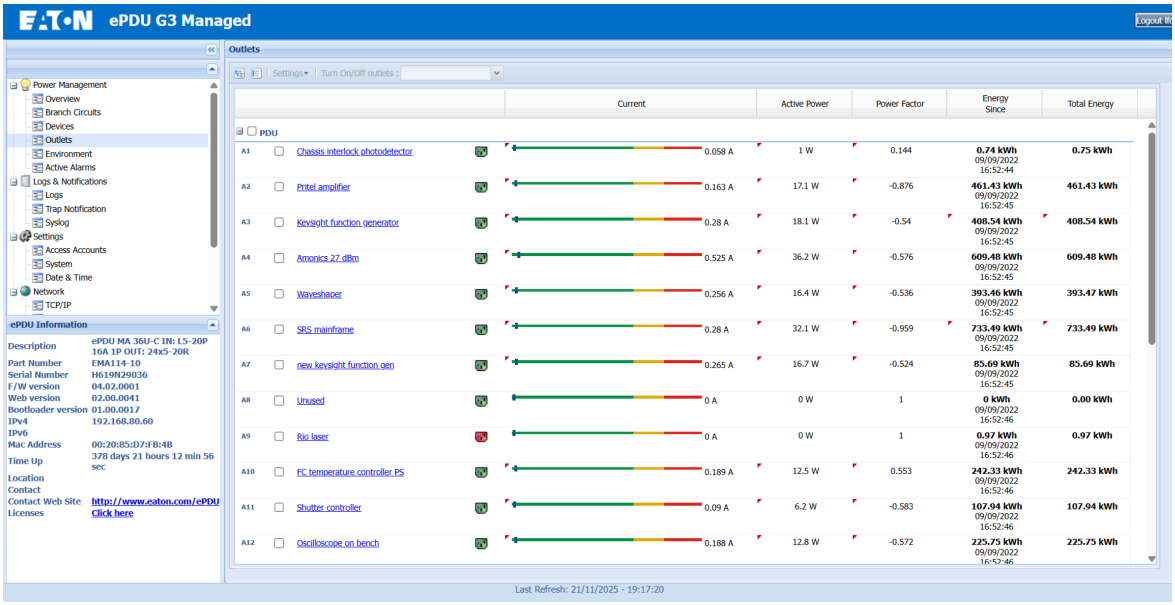


Figure 5.21. ePDU A1-A12

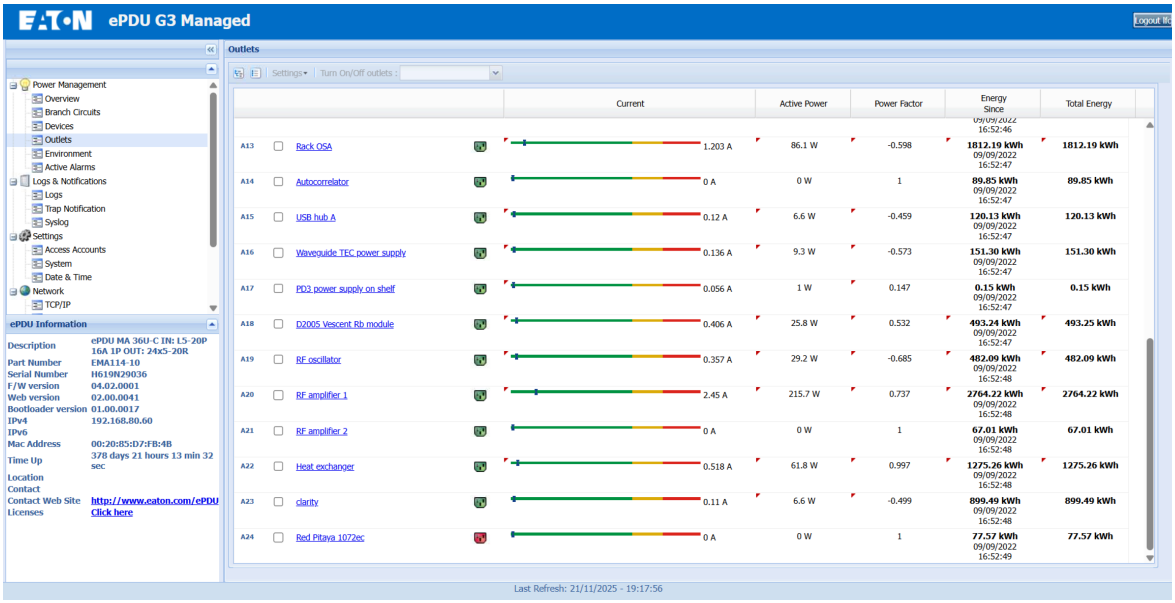


Figure 5.22. ePDU A13-A24

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

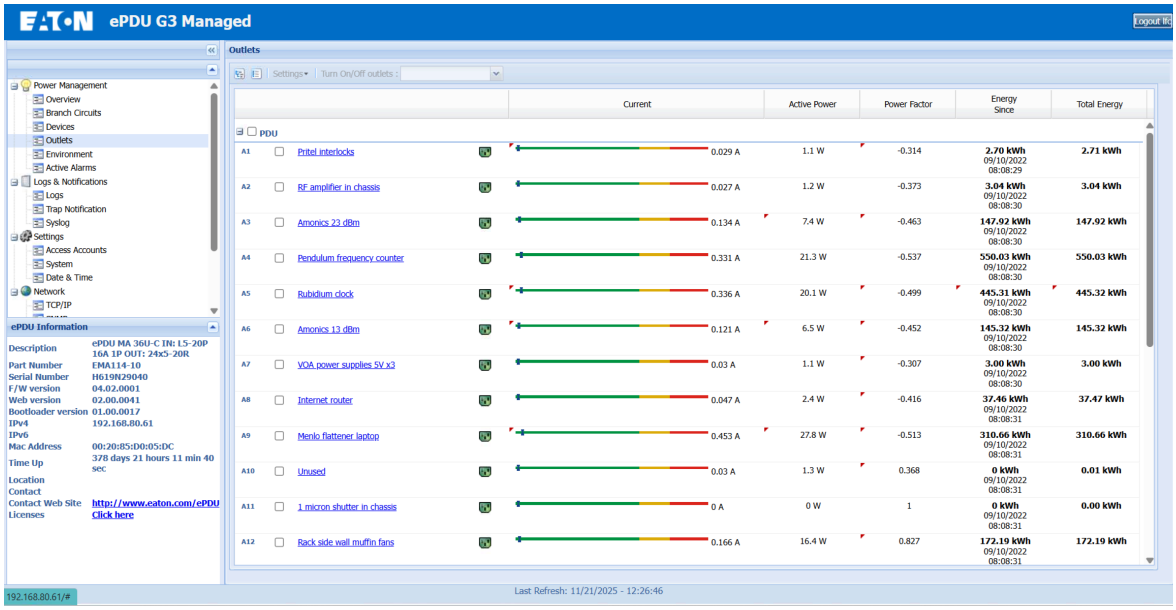


Figure 5.23. ePDU B1-B12

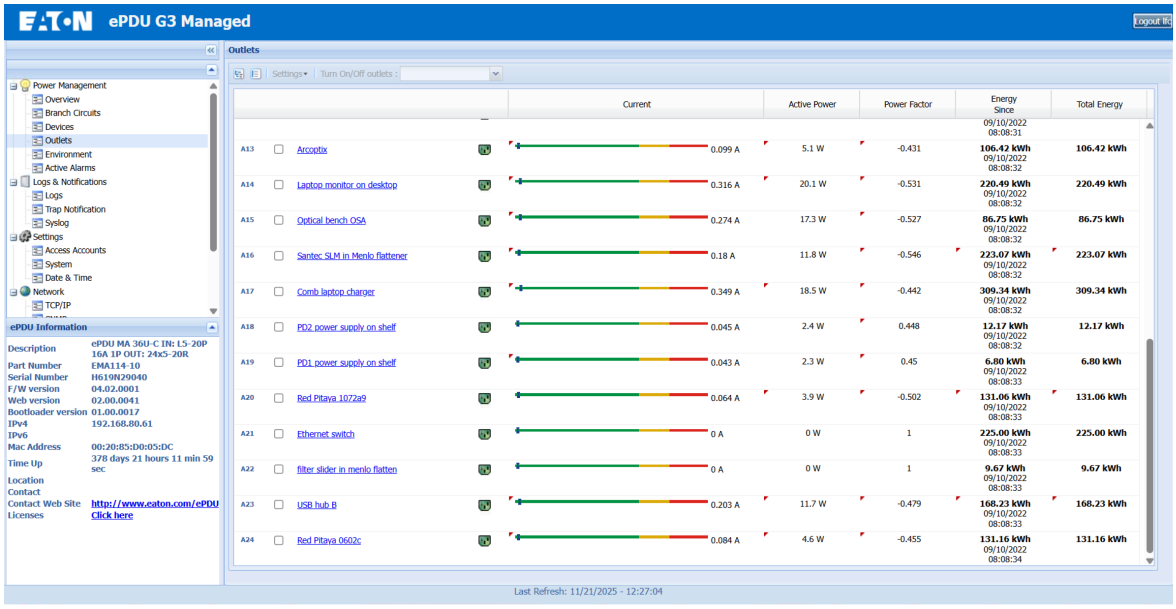


Figure 5.24. ePDU B13-B24

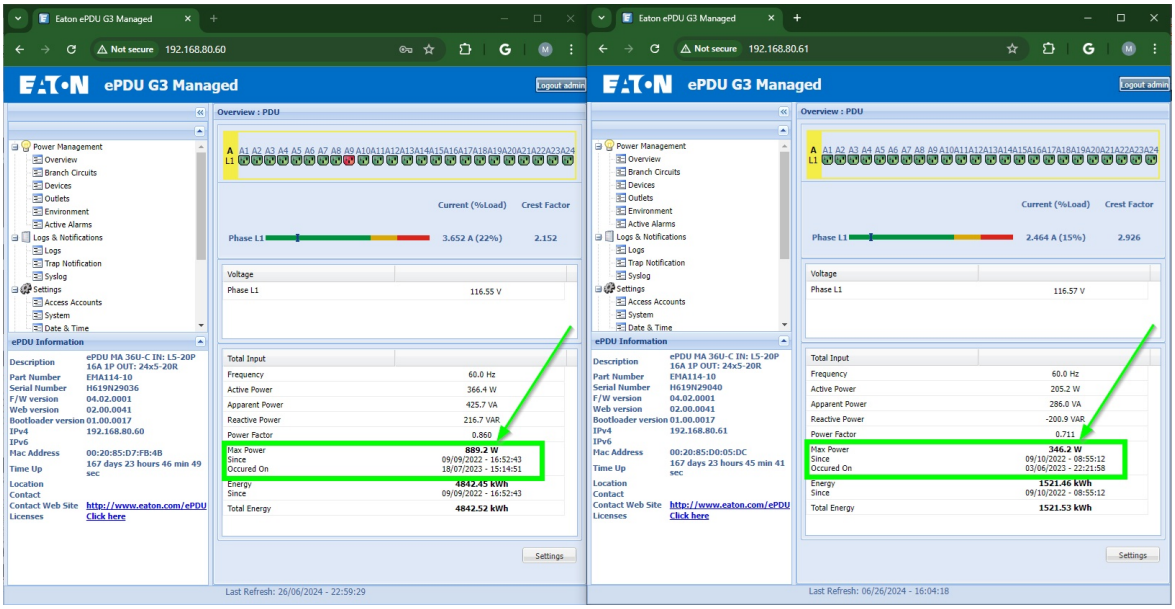


Figure 5.25. ePDU power draw

Table 5.8. LFC Electronics Rack Power Port Assignments

Strip A		**Strip B**	
Port	Assignment	Port	Assignment
1	Photodetector Power Supply for Pritel interlock	1	Relay Circuit Power Supply
2	Pritel Erbium Doped Fiber Amplifier	2	Small component chassis RF amplifier power supply
3	Keysight function generator	3	Amonics 23 dBm EDFA
4	Amonics 27dBm EDFA	4	Pendulum frequency counter
5	Finisar Waveshaper	5	SRS Rubidium clock
6	SRS Mainframe	6	Amonics 13 dBm EDFA
7	Spare	7	Small component chassis variable optical attenuators (3)
8	Spare	8	Internet router power supply
9	RIO laser power supply	9	Menlo Flattener Laptop Power Supply
10	FC Temperature controller PS	10	Spare
11	Thorlabs Shutter Controller	11	1 yJ band Fiber Shutter Controller in small component chassis
12	HP Oscilloscope on Optical Bench	12	Side-mounted Rack Muffin Fans
13	Agilent Optical Spectrum Analyzer	13	Arcoptix Spectrometer in Spectral Flattener
14	Autocorrelator (temporary)	14	LFC Computer Monitor on Desk
15	USB Hub A	15	Optical bench OSA (temporary)
16	Waveguide TEC power supply	16	Santec SLM in Menlo Flattener
17	Photodetector 3 power supply	17	LFC computer charger
18	Vescent D2-005 power supply	18	Photodetector power supply 2
19	GWInstek power supply for Nexyn RF oscillator	19	Photodetector power supply 1
20	GWInstek power supply for CTT RF amplifier	20	Red Pitaya FPGA 072A9
21	Spare RF power supply	21	Ethernet Switch
22	Heat exchanger	22	Filter slider in Menlo Flattener
23	Clarity	23	USB Hub B
24	Red Pitaya FPGA 072EC	24	Red Pitaya FPGA 0602C

Note: Two components in the rack are disconnected from power: the filter cavity piezo controller and the filter cavity TEC power supply.

5.4 Instrument operation details and comb operation process

This section describes, step by step, how to bring up the comb and how to operate each instrument involved in the EO and broadened comb chain. It is intended as a practical "from the rack to the comb" guide rather than a low-level programming reference.

If you plan to operate any instrument in this system, please read and understand this section first. If you are unsure about any detail, please ask Jinhao or Stephanie before proceeding.

5.4.1 Individual instrument operation

5.4.1.1 Clarity reference laser

The Clarity unit is a commercial frequency reference laser. A general introduction is given in the hardware section; here we focus on operational details.

- Clarity contains its own internal reference with a nominal stability at the level of about 10^{-9}. For most purposes this is sufficient as a stand-alone reference.
- After power-on the laser requires a warm-up time. The front panel reports three status phases: initial power-on, locking attempt, and finally locked. Only in the locked state should Clarity be used as a reference.
- The front-panel laser button is password protected. The current password is **5000**. This should not be changed without coordination.
- Most front-panel functions are self-explanatory. There is an option to lock on different parts of the error signal; for this application the exact lock point is not critical.
- Before cutting power to the rack, always turn off the Clarity laser activation on the front panel. Do not simply pull power while the diode is driven.

5.4.1.2 RIO seed laser

The RIO laser is the main CW seed for the EO comb.

- The RIO emits light as soon as it is plugged in and enabled; it also needs a few minutes of warm-up to reach thermal equilibrium.
- The free-running RIO is not sufficiently stable for astrocomb applications. In regular operation it is locked to a rubidium (Rb) reference, described later in this section. When locked, the stability is comparable to or better than Clarity.
- The laser is controlled exclusively via software. The two main control parameters are diode current and temperature:
  - current allows fast, small-range frequency tuning;
  - the TEC temperature provides slow, large-range tuning.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

After a tuning change, allow time for the temperature and cavity to re-stabilize.

- The RIO in the lab is known to exhibit occasional mode hops; similar behaviour may occur in the summit unit. Mode hops are not a fault of the code; they are a property of the device.

In practice, Clarity is convenient for quick standalone work, while RIO+Rb lock is used when a tighter or Rb-referenced frequency is required.

5.4.1.3 RF oscillator and RF oscillator power supply

The RF oscillator provides the 16 GHz drive for the phase and intensity modulators. Its DC power is supplied by an Instek GPD-4303S four-channel supply; only channels 2 and 3 are used in this system.

A typical status readout looks like:

```
|----------------Instek DC Power Supply GPD-4303S Status Summary-----------------
|--------------------Model: GPD-4303S, Serial No.SN:EV884423--------------------
|   ------------Channel Summary-------------
|    CH1: master, CH2: slave, mode UNRECOGNIZED?.
|    CHANNEL1: CC Mode.
|        VSET=0.000V, ISET=0.000A.
|        VOUT=0.000V, IOUT=0.000A.
|    CHANNEL2: CV Mode.
|        VSET=15.000V, ISET=3.000A.
|        VOUT=14.997V, IOUT=0.609A.
|    CHANNEL3: CV Mode.
|        VSET=0.800V, ISET=1.000A.
|        VOUT=0.800V, IOUT=0.001A.
|    CHANNEL4: CV Mode.
|        VSET=0.000V, ISET=0.000A.
|        VOUT=0.000V, IOUT=0.000A.
--------------Instek DC Power Supply GPD-4303S Status Summary Ends--------------
```

- Channel 2 powers the RF oscillator:
  - working point is about 15 V and 0.6 A;
  - the supply is configured with limits of 15 V / 3 A.
- Channel 3 provides the RF attenuation control voltage (“IM bias supply” for the RF amplitude). In normal operation it is set around 0.80–0.85 V.
- Channels 1 and 4 are unused.
- Before turning off mains power, always disable the DC supply outputs (set `activation = 0`) and only then turn off the front-panel power.

Example code:

```
rfoscPS.Vset2 = 15
rfoscPS.Iset2 = 3
rfoscPS.Vset3 = 0.8
rfoscPS.Iset3 = 1

rfoscPS.activation = 1
```

5.4.1.4 RF amplifier and RF amplifier power supply

The RF power amplifier boosts the 16 GHz signal to the level required by the EO modulators. There are two RF amplifiers mounted on the cooling panel, one of which is a spare.

- The RF amplifier modules are mounted on a cooling plate via a thermal pad. If an amplifier is replaced or moved, re-apply a thin, uniform layer of thermal compound between the amplifier base and the cooling panel before tightening the screws.
- The dedicated RF amplifier DC supply is separate from the RF oscillator supply. It is configured, in normal operation, to:
  - `VSET = 30.000 V`,
  - `ISET = 4.200 A`.
- In a healthy state the output power is about 120 W. If either the voltage or the current deviates significantly from the nominal values, immediately switch off the amplifier supply.
- A previous power outage damaged an RF amplifier when it was left at high current without proper shutdown. Treat this supply with care.
- When manually starting the comb from scratch, always:
  (1) turn on the RF amplifier supply first;
  (2) then turn on the RF oscillator supply.

  The RF amplifier supply should be brought directly to 30 V; do not slowly ramp the voltage through a large range while the RF drive is present.

Example code:

```
rfampPS.Vset1     = 30
rfampPS.Iset1     = 4.2
rfampPS.activation1 = 1
```


jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

5.4.1.5 Amonics 27 dBm EDFA

The Amonics 27 dBm EDFA pre-amplifies the seed laser before EO modulation.

- ◇ The default operating mode is APC (automatic power control) with an output around 450 mW.
- ◇ The seed light is passed through this EDFA before the EO modulation chain. Excessively high power at this stage can damage downstream components.

Example code:

```
amonic27._setChMode('apc')
amonic27.accCh1Cur    = '450mw'
amonic27.accCh1Status = 1
amonic27.activation   = 1
```

5.4.1.6 Phase modulators, intensity modulator, and phase shifter

The EO comb is generated by a chain of three phase modulators (PMs) and one intensity modulator (IM), plus a passive phase shifter.

- ◇ The three PMs and the IM are normally operated at fixed RF drive and bias points; there are no frequent adjustments once they are set up.
- ◇ The dedicated phase shifter is used to optimise the EO comb bandwidth. Typically only one combination of knob settings maximizes the comb width. If the EO comb bandwidth shrinks significantly, small adjustments of the phase shifter knobs may help; otherwise leave them untouched.

5.4.1.7 Optical spectrum analyser (OSA)

The OSA is used to view the EO comb and the broadened comb toward shorter wavelengths.

Typical usage:

```
osa.Run()

osa.wlstart   = 1553
osa.wlstop    = 1567
osa.reflevel  = -40
osa.get_trace('a')
```

- ◇ Use a narrow span around the carrier (for example 1553–1567 nm) to assess EO comb flatness and sideband structure.
- ◇ Use a wider span and higher input attenuation when viewing the fully broadened comb to avoid saturating the OSA input.

5.4.1.8 Amonics 23 dBm EDFA

The 23 dBm Amonics EDFA amplifies the EO comb before the Pritel amplifier and nonlinear broadening.

Example code:

```
amonic23.accCh1Cur    = '80mA'
amonic23.accCh1Status = 1
amonic23.activation   = 1
```

- ◇ Use just enough power to reach the desired comb bandwidth after the nonlinear fibre and Tantala waveguide.
- ◇ Monitor temperatures and interlocks when increasing this amplifier power.

5.4.1.9 Waveshaper

The waveshaper is used to transform the EO comb into a near-Fourier limited pulse by applying the appropriate spectral phase and bandpass.

- ◇ When the IM bias is locked slightly below the centre of the negative RF slope, a second-order dispersion of roughly $d_2 \approx 2$ (in the internal units) is often suitable.
- ◇ When the IM bias is locked near the positive slope, a value $d_2 \approx -3$ is typical. These numbers are only rough references; the actual optimum depends sensitively on IM bias, optical power, and temperature. Always verify against the measured comb on the Menlo flattener.
- ◇ The spectral bandwidth setting usually does not need frequent changes.
- ◇ The third-order dispersion parameter d_3 is normally set to zero; empirically, nonzero d_3 tends to worsen the pulse quality for this system.

Example code:

```
wsp.set3rdDisper(d2=2.14, d3=0.0, center=1559.8)
wsp.setBandPass(span=5)
wsp.plotStatus()
wsp.writeProfile()
```

Important implementation notes:

- ◇ The waveshaper uses its own API (not VISA). In the current Python interface you must call `disconnect()` before the next `connect()`, otherwise the device may become unreachable until the Python kernel is restarted.
- ◇ Only one thread or process can hold a connection at a time. When debugging in Jupyter, release the connection and restart the kernel when finished.

5.4.1.10 Arduino interlock and YJ shutter control

The Arduino-based interlock performs two tasks:

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

- ◇ protect the Pritel amplifier by enforcing an interlock;
- ◇ control the YJ comb shutter.

- ◇ When the system is closed or disconnected, the Arduino places the interlock in a safe *off* state, blocking the amplifier. Only when the Pritel input power is adequate and `reset_relay_latch()` has been executed can the system move to *standby* and allow Pritel activation.
- ◇ Calling `disconnect()` on the Arduino driver returns the system to the *off* state. Restarting the Python kernel does not automatically reset the hardware latch.
- ◇ The Arduino cannot be connected twice simultaneously; as with the waveshaper, you must disconnect before a new connection.

Example code:

```
arduino.reset_relay_latch()  # reset to standby if input power is OK

arduino.shut_YJ()
arduino.pass_YJ()
arduino.get_YJ_info()
```

5.4.1.11 Pritel main amplifier

The Pritel amplifier is the final high-power stage before the nonlinear fibre and the Tantala waveguide.

- ◇ Typical operating points:
  - preamplifier current: about 600 (internal units, e.g. mA);
  - main amplifier current: 3.8–4.2 A.
- ◇ The hardware limit of the main amplifier is 5 A, but operating near this limit is **very risky** for the optics and fibre. Currents above 4.2 A are strongly discouraged unless there is a clear and understood reason and appropriate supervision.

Example code:

```
ptamp.preAmp    = 600
ptamp.activation = 1

ptamp.pwrAmp    = '3.8A'   # usually 3.8 A to 4.2 A max
```

5.4.1.12 2×2 optical switch

The 2×2 switch selects whether YJ or HK comb light is sent to the instrument.

- ◇ `status = 1`: YJ comb path selected.

- ◇ `status = 2`: HK comb path selected.

Always confirm the switch status before enabling the corresponding shutter and adjusting instrument exposure.

5.4.1.13 HK shutter

The HK shutter controls whether HK comb light passes to the instrument.

- ◇ `status = 0`: shutter closed, no HK comb light.
- ◇ `status = 1`: shutter open, HK comb passes.

The HK shutter uses a dedicated serial protocol (not VISA). From the user perspective it acts like a simple on/off device.

5.4.1.14 Oscilloscope

The oscilloscope is used to observe the Rb spectroscopy and PDH locking signals.

- ◇ Channel 1: transmission (or absorption) signal through the Rb cell.
- ◇ Channel 2: PDH error signal used for locking.
- ◇ Note that the oscilloscope occasionally freezes. When this happens, it must be power-cycled via the ePDU rather than only using the front-panel power button.

Example code:

```
osc.get_trace(trace=2)
```

5.4.1.15 Function generators

There are two Keysight 33500B function generators mounted in the rack and controlled via code.

- ◇ First generator:
  - channel 1: provides the slow frequency sweep for the laser (scan used to find Rb lines);
  - channel 2: currently unused.
- ◇ Second generator:
  - channels 1 and 2: provide modulation and demodulation signals for the PDH lock.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Example code:

```
fg.set_channel_parameters(channel=1, freq=10, amp=8, offset=0, phase=0)
fg.set_channel_func(1, 'ramp')
fg.set_channel_state(1, 0)
```

5.4.1.16 VOA 1550, 2000, and 1310

These three VOAs are electrically controlled optical attenuators for the pump (1550 nm) and the YJ/HK comb paths (1310 nm and 2000 nm). In the current configuration:

- ◇ the VOAs are mounted in a small component chassis;
- ◇ the entire chassis is bypassed in the live optical loop.

As a result, these VOAs are not active in the current feedback loop and their settings do not affect the instrument. The chassis wiring is quite dense; unless you are actively reworking that subsystem, it is recommended not to open or modify it.

Example code (for reference only):

```
voa1550.atten_db = 0   # set attenuation
voa1550.atten_db       # read attenuation
voa1550.disconnect()
```

5.4.1.17 TEC controllers

TEC controllers on the breadboard regulate the temperature of the PPLN crystal and the waveguide.

- ◇ The PPLN TEC sets the phase-matching window of the 1550 nm second-harmonic generation and is used to align the doubled light to the Rb D2 transition. Once aligned, its setpoint is rarely changed.
- ◇ The waveguide TEC is typically set near 25 ℃. Because the breadboard is water-cooled and colder than room temperature, the TEC initially heats to reach 25 ℃; after the comb is on and absorption increases, the TEC will gradually switch from heating to cooling. Seeing this change is a good check that the TEC is actually regulating.

Example code:

```
tec_ppln.get_control_type()
tec_ppln.get_temp()
tec_ppln.set_temp(22)
```

A more careful ramp of the PPLN setpoint:

```
import time
import numpy as np
```

```
now_temp = tec_ppln.get_temp()
set_temp = 20.1

temp_gap = round(set_temp - now_temp, 2)

int_p = int(temp_gap // 0.5)
mod   = temp_gap % 0.5

if (int_p > 0) or (int_p == 0):
    int_p = int_p
else:
    int_p = np.abs(int_p) - 1

for i in range(1, int_p + 1):
    tec_ppln.set_temp(now_temp + i * 0.5)
    time.sleep(4)
    print(tec_ppln.get_temp())

tec_ppln.set_temp(set_temp)
time.sleep(4)
print(tec_ppln.get_temp())
```

5.4.1.18 Thermocouples and DAQ boards

Thermocouples are distributed across the astrocomb system and read out by two DAQ boards (`addr = 0` and `addr = 1`). The channel mapping is:

- **Board on the optical table** (`addr = 1`):
  - RF oscillator,
  - RF amplifier,
  - main phase modulators,
  - filter cavity,
  - breadboard glycol out,
  - breadboard glycol in,
  - compression stage,
  - Rb cell (D2-210).
- **Board in the electronics rack** (`addr = 0`):
  - rack side baffle (mid rack),
  - waveshaper (upper rack),
  - Rb clock (mid rack),

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

- Pritel (mid-upper rack),
- rack glycol out,
- rack glycol in,
- power supply shelf (bottom rack),
- unused.

Example code:

```
daq0.get_temp_all()
daq1.get_temp_all()
```

5.4.2 Combined operation details

This subsection describes how the instruments work together in the overall comb operation, from EO comb flattening to Rb locking.

5.4.2.1 Flattening the EO comb and locking the IM bias

5.4.2.1.1 Viewing the EO comb. Start by observing the EO comb spectrum:

```
osa.Run()

osa.wlstart  = 1553
osa.wlstop   = 1567
osa.reflevel = -40
osa.get_trace('a')
```

Adjust input attenuation or resolution as needed to see the comb structure clearly.

5.4.2.1.2 Setting the IM servo to manual mode. Place the intensity-lock servo into manual mode:

```
servo_IM.output_mode = 'man'   # 'man' or 'pid'
```

5.4.2.1.3 Scanning the IM bias to find a good slope. Use the servo to scan the IM bias between about -3 V and $+3$ V and locate a point on a reasonably linear slope where the EO comb appears flattest. Record the corresponding servo output voltage.

Example scan:

```
import numpy as np

scan_start = -3
scan_stop  =  3
num_steps  = 100

servo_IM.manual_output_ramp = 0.1
servo_IM.manual_output      = scan_start
```

```
r = servo_IM.manual_sweep(np.linspace(scan_start,
                                      scan_stop,
                                      num_steps))

servo_IM.manual_output = 1.4   # example chosen bias
```

5.4.2.1.4 Adjusting RF attenuation and IM bias. Next, adjust the RF attenuation voltage (channel 3 of the RF osc supply) and the IM bias simultaneously while monitoring the OSA. The goal is to obtain a reasonably flat minicomb around the carrier.

Example:

```
import time

servo_IM.manual_output = 1.4
rfoscPS.Vset3          = 0.85
time.sleep(1)

osa.Run()
time.sleep(1)
osa.get_trace('a')
```

Iterate between IM bias and RF attenuation until the comb shape is satisfactory.

5.4.2.1.5 Enabling the IM bias lock (PID mode). Once a good operating point is found, record:

- the servo *input* (error signal setpoint);
- the servo output (IM bias voltage).

These values define the `setpoint` and `outoffset` for PID operation.

Example PID configuration:

```
servo_IM.output_upperlim = 10
servo_IM.output_lowerlim = -10

# PID gain parameters
servo_IM.prop_gain = -2    # negative if locking on a falling slope
servo_IM.intg_gain = 0.1
servo_IM.outoffset = 0.2
servo_IM.set_setpoint(2.2)

servo_IM.output_mode = 'pid'   # 'man' or 'pid'
```

If you lock on a rising slope, the proportional gain should be positive; if you lock on a falling slope, it should be negative.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

5.4.2.1.6 Quick sanity check of the IM lock. After enabling PID mode, monitor the servo output and input for a short period (e.g. 20 s) to verify that the loop is converging rather than running away:

```
import time
for i in range(20):
    print(servo_IM.get_output_voltage())
    print(servo_IM.get_measure_input())
    time.sleep(0.5)
```

If the output ramps to a rail or the error does not shrink, reduce the gains and try again. Do not leave an unstable PID loop running while the comb is being used for science.

5.4.2.2 Rb lock of the RIO laser

The Rb lock stabilises the RIO frequency to a transition in the Rb D2 line via second-harmonic generation in PPLN and a PDH locking scheme.

5.4.2.2.1 Initial temperature conditions. From previous calibrations, the PPLN temperature setpoint is typically around 42 °C, but this can drift. If starting from a warm system with known working points, you can often begin searching near the last PPLN and laser temperature settings. A fully cold start requires a two-dimensional scan in both laser temperature and PPLN temperature, which is significantly slower.

5.4.2.2.2 Finding the absorption features. Use the function generator to provide a low-frequency ramp to the laser (via the appropriate input), and observe the transmission on the oscilloscope:

- ◇ channel 1: transmission through the Rb cell, showing a broad Lorentzian envelope and finer substructure;
- ◇ channel 2: PDH signal, initially not well optimised.

Turn on the modulation and demodulation paths on the second function generator, then adjust the modulation amplitude and phase until the PDH signal resembles the expected dispersive shape around the target sub-feature.

Gradually reduce the laser scan amplitude so that the sweep covers only the region around the desired fine structure. The goal is to end with a scan that runs almost entirely on the target slope.

5.4.2.2.3 Engaging the Rb servo. When the operating point is well centered on the chosen slope, switch the laser modulation from a ramp to DC and enable the Rb servo in PID mode. For example:

```
fg.set_channel_func(1, 'dc')
# fg.set_channel_parameters(channel=1, freq=10, amp=1, offset=0.95, phase=0)
# fg.set_channel_state(1, 1)
```

```
fg.get_channel_parameters(1)

servo_RB.output_mode = 'pid'
```

The Rb lock should pull the laser into the reference slope and keep it there, provided the PPLN temperature, optical power, and servo gains are in the right range.

5.4.2.3 Automatic Rb lock algorithm and periodic lock checks

The automated Rb locking routine uses a convolution-based pattern recognition method to find the correct spectral feature and narrow down the scan range.

5.4.2.3.1 Automatic lock algorithm (conceptual). see 5.3.2.2 first.

- For each scan range of interest, the system has pre-recorded Rb transmission traces. Fig. 5.26
- During automatic locking, the current oscilloscope trace is convolved with each stored template trace to identify the best match.
- Once a match is found, the scan range is narrowed and the process is repeated, each time zooming closer to the target sub-feature.
- When the scan covers only the desired slope, the corresponding laser control voltage is read out and used as the offset for enabling the Rb servo in PID mode.

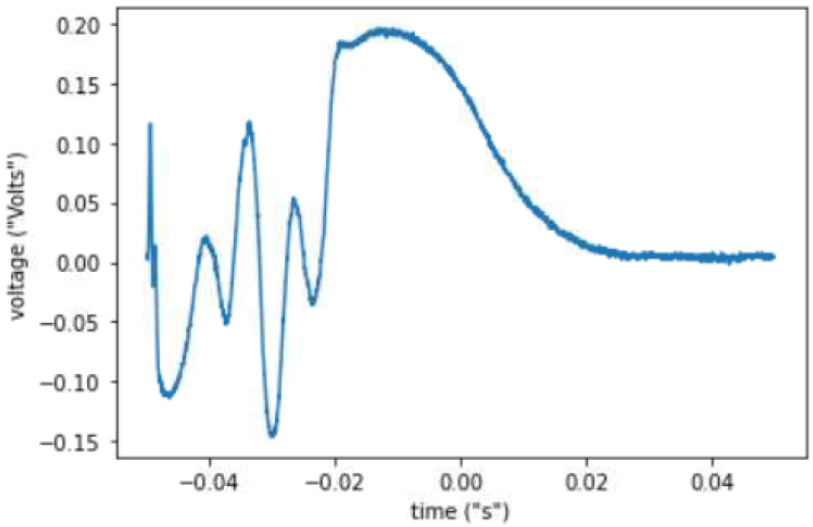


Figure 5.26. Rb transmission PDH signal (Full scan range).

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

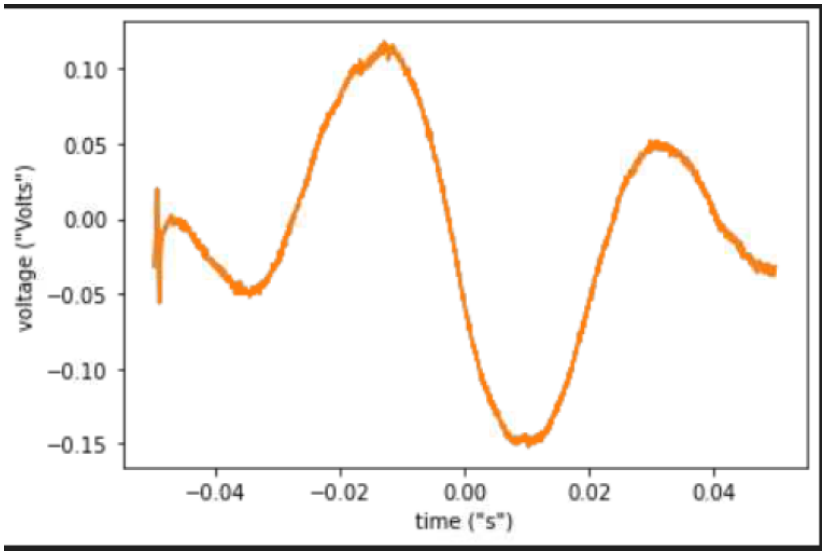


Figure 5.27. Rb transmission PDH signal (smaller scan range).

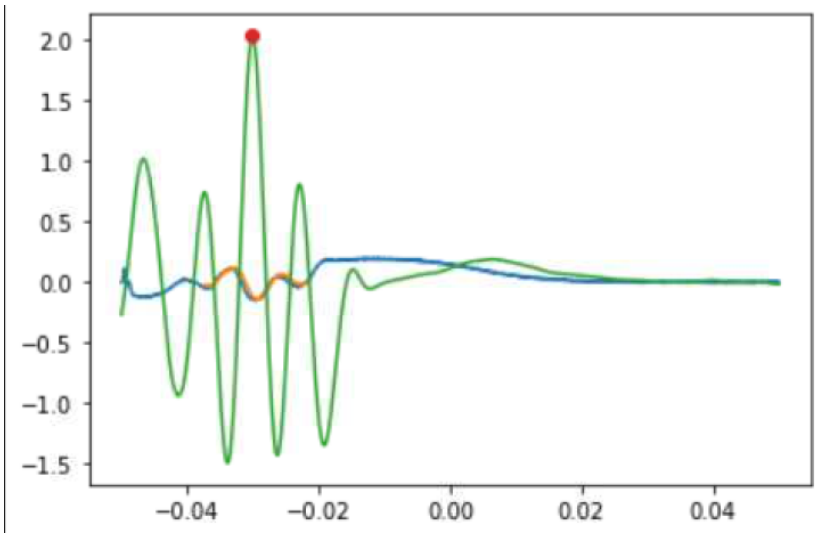


Figure 5.28. convolution value of current trace with previous stored trace. The red point shows where the maximum is.

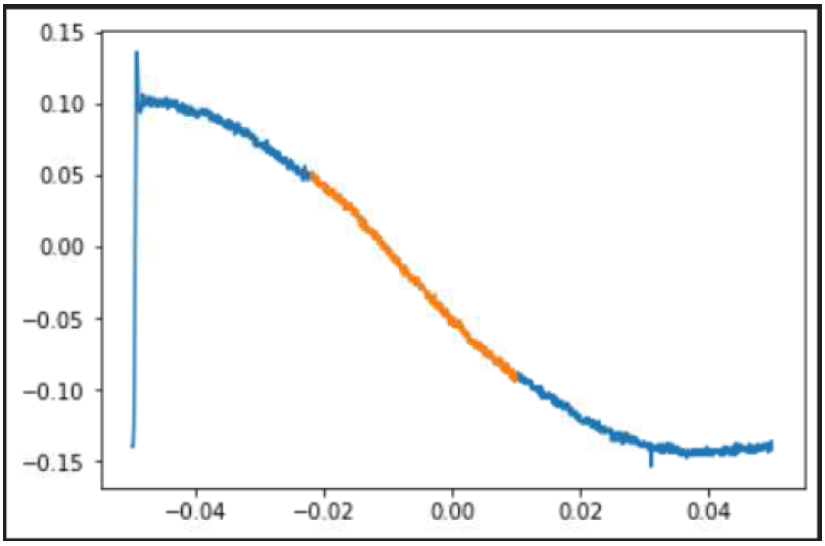


Figure 5.29. Rb transmission PDH signal. The orange part shows the target slope position.

This staged narrowing process makes the lock robust against moderate drifts in temperature and optical power.

5.4.2.3.2 Periodic Rb lock checks. The stability of the Rb lock depends on:

- external conditions (temperature, mechanical disturbances);
- servo gains and bandwidth;
- slow drifts in PPLN and laser temperature.

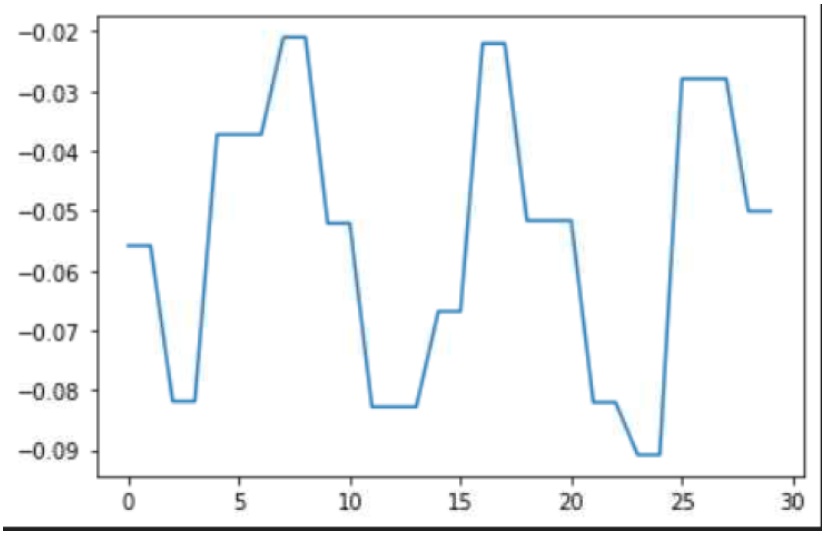


Figure 5.30. Servo output voltage time series with laser voltage perturbation.

jge2@caltech.edu, stephanie.leifer@aero.org, gdoppmann@keck.hawaii.edu, daniel.hickstein@octavephotonics.com, david.carlson@octavephotonics.com, yjkim134340@gmail.com

Because the Rb lock underpins the entire comb frequency stability, it should be checked periodically during observations. A simple and robust check is:

- ◇ apply a small, narrow-range modulation (dither) to the laser control;
- ◇ monitor the Rb servo output and error signal;
- ◇ when the lock is engaged, the servo output should show a small response at the dither frequency with opposite sign relative to the error signal (negative feedback).

If the servo output no longer responds as expected, or if the PDH signal degrades significantly, the Rb lock is likely lost or drifting. In that case, re-run the automatic locking sequence or repeat the manual procedure above.

Taken together, the instrument-level notes and combined procedures in this section describe the full chain from a CW seed laser to a flattened EO comb, spectral phase correction, nonlinear broadening, and finally a Rb-referenced astrocomb suitable for on-sky operation. Operators are strongly encouraged to follow the power-up and shutdown order given here and to keep safety margins on all high-power devices, especially the RF amplifier and the Pritel main amplifier.